Colorado State University Pueblo

School of Engineering

A Graph Neural Surrogate for Longitudinal Train Dynamics: Toward Fuel-Optimal Control of Freight Consists

George James

October 10, 2026

Abstract

Freight-train longitudinal dynamics (LTD) simulators capture nonlinear slack couplers, per-vehicle grade and curvature, actuator lag, and power-limited traction with high fidelity, but their computational cost — minutes of CPU time per kilometre of simulated track, and months of sequential compute for a whole-trip optimization study — makes them impractical for the repeated evaluation that control design and operational planning require. This thesis develops a fast, train-agnostic surrogate of longitudinal train dynamics that serves as a differentiable simulation backbone for fuel-optimal control of freight consists.

The consist is represented as a chain graph, with vehicles as nodes and couplers as edges, using a tensorized node/edge state ($\mathbf{H}$, $\mathbf{E}$) that generalizes across arbitrary train length without padding or masking. A message-passing graph neural network advances that state by one fixed 0.25 s step: a graph network simulator, rather than a learned derivative wrapped in a numerical solver. The network predicts only the mean acceleration of each vehicle over the step, together with a per-coupler shape correction; position follows from the kinematic trapezoid, and the brake and traction lag states from the exact solution of their first-order dynamics. A single network evaluation therefore replaces fifty right-hand-side evaluations of the reference integrator, and what is learned narrows to two scalars per vehicle rather than a correction on every state channel. Predictions inherit the consist's chain topology and remain differentiable end to end for downstream control. This design targets structural generalization: a model trained on one range of consist sizes and route conditions should extrapolate to unseen train lengths and corridors, rather than relying on the fixed-length token handling that sequence models require.

Training data are produced by a reference LTD simulator, developed and validated stage by stage (single-vehicle traction, linear and nonlinear slack couplers, N-vehicle assemblies, per-vehicle route sampling, and actuator/power-limited traction), driven along real-terrain route profiles derived from elevation and alignment data. The surrogate is evaluated against zero-output, hold-the-state and constant-acceleration baselines, against a physics-residual MLP, and against a transformer variant demoted to an ablation, on trajectory fidelity, trip-level summary accuracy (energy, arrival time, peak coupler force), and generalization across held-out consist sizes, grades, and corridors.

The trained surrogate then serves as the world model for a fuel-optimal control demonstration: minimizing trip energy on held-out real routes subject to speed limits and coupler-force safety constraints, using model-predictive control that exploits the surrogate's analytic gradients. Throughout, we are explicit that simulator parameters are illustrative rather than calibrated to a specific real train, so the claims concern fidelity to the reference simulator, structural generalization across consist size and route, and control performance under that

model — not fidelity to real rolling stock.

Acknowledgments

Contents

Abstract 3

Acknowledgments 5

I A First Surrogate 19

1 Introduction and Problem Motivation 21
- 1.1 Background: Longitudinal Train Dynamics . . . 21
- 1.2 Surrogate Modeling . . . 22
- 1.3 Decision Support and Fuel-Optimal Control . . . 23
- 1.4 Thesis Objective and Contributions . . . 23
- 1.5 Scope . . . 24
- 1.6 Organization of Thesis . . . 24

2 Related Work 25
- 2.1 Classical Train Longitudinal Dynamics and Simulation . . . 25
  - 2.1.1 Modeling Abstractions and Levels of Fidelity . . . 25
  - 2.1.2 Computational Schemes and Numerical Challenges . . . 26
  - 2.1.3 Benchmarking, Applications, and Limitations . . . 26
- 2.2 Data-Driven and Hybrid Modeling in Rail . . . 27
  - 2.2.1 Surrogate Models for Accelerating Physics-Based Simulation . . . 27
  - 2.2.2 Data-Driven Estimation of In-Train Forces . . . 28
  - 2.2.3 Surrogate Modeling for Operations and Optimization . . . 28
  - 2.2.4 Limitations of Existing Data-Driven and Hybrid Approaches . . . 29
  - 2.2.5 Implications for Structural Generalization . . . 29
- 2.3 Transformers for Dynamical Systems and Spatiotemporal Forecasting . . . 29
  - 2.3.1 Long-horizon Forecasting Transformers . . . 30
  - 2.3.2 Neural Operators for PDE-governed Dynamics . . . 31
  - 2.3.3 Implications for Train Dynamics and Spatiotemporal Decision Support 32
- 2.4 Graph Networks for Physical Systems . . . 32
  - 2.4.1 Relational inductive bias and message passing . . . 32
  - 2.4.2 Graph networks as learned simulators . . . 33
  - 2.4.3 Stability of learned rollouts . . . 33
  - 2.4.4 Learned coarse models and the cost of a step . . . 34

2.4.5 Continuous-time alternatives . . . 34
2.4.6 Graph learning in rail applications, and the remaining gap . . . 34
2.5 Probabilistic Forecasting and Uncertainty Calibration . . . 35
2.5.1 Foundations of Probabilistic Regression . . . 36
2.5.2 Empirical Miscalibration in Deep Models . . . 38
2.5.3 Post-hoc Calibration for Regression . . . 38
2.5.4 Distribution-Free Coverage via Conformal Prediction . . . 40
2.5.5 Proper Scoring Rules and Evaluation Metrics . . . 40

3 Problem Formulation **43**
3.1 The prediction and control problem, and the tensor format used to describe it 43
3.1.1 Physical setting and prediction task . . . 43
3.1.2 Classical longitudinal dynamics formulation . . . 44
3.1.3 Tensorized state representation . . . 45
3.1.4 Dynamics in tensor-oriented form . . . 45
3.1.5 Forecasting objective and simulator operator . . . 46
3.2 Input definitions . . . 47
3.2.1 Route profile . . . 47
3.2.2 Consist configuration . . . 48
3.2.3 Control plan . . . 48
3.3 Output definitions . . . 48
3.3.1 Trajectory-level outputs . . . 48
3.3.2 Aggregate outputs . . . 49
3.3.3 Risk metrics . . . 49
3.4 Uncertainty sources . . . 49
3.4.1 Formal treatment of uncertainty . . . 50
3.5 Supervised Learning Objective . . . 50
3.5.1 Loss term definitions . . . 50
3.6 Conceptual pipeline . . . 51
3.7 Evaluation targets . . . 51
3.8 Chapter summary . . . 52

4 Reference Simulator **53**
4.1 Stage 1: single powered vehicle . . . 53
4.2 Stage 2: locomotive and one car with a linear coupler . . . 55
4.3 Stage 3: nonlinear coupler with slack and asymmetric buff/draft . . . 57
4.4 Stage 4: three bodies and sequential coupler forces . . . 58
4.5 Stage 5: general N-vehicle assembly . . . 58
4.6 Stage 6: per-vehicle grade and curvature from a route profile . . . 58
4.7 Stage 7: actuator dynamics and power-limited traction . . . 61
4.8 Stage 8: scope relative to industrial high fidelity . . . 63
4.9 Chapter summary . . . 64

5 Route Acquisition and Terrain Data Pipeline **65**
5.1 Stage 1: Acquisition . . . 66
5.1.1 Why acquisition has to be area-first . . . 66
5.1.2 Corridor extraction as a graph problem . . . 67
5.1.3 Putting fragments back together . . . 67
5.1.4 Interactive selection . . . 68
5.2 Stage 1.5: Resampling to a uniform grid . . . 68
5.3 Stage 2: Elevation . . . 69
5.4 Stage 3: Deriving the route profile . . . 70
5.4.1 Correcting for bridges and tunnels . . . 70
5.4.2 Smoothing and differentiating together . . . 71
5.4.3 Speed limit and output . . . 72
5.5 Export . . . 72
5.6 Testing and design choices . . . 73
5.7 Chapter summary . . . 73

6 RailLab: A Desktop Research Console **75**
6.1 Why a graphical console . . . 75
6.2 Architecture . . . 76
6.3 The workflow, screen by screen . . . 76
6.3.1 Route . . . 76
6.3.2 Consist . . . 77
6.3.3 Controls . . . 77
6.3.4 Simulate and Results . . . 80
6.4 Status and limitations . . . 80
6.5 Chapter summary . . . 80

7 GPU Integrator and Dataset Build **83**
7.1 Why the reference simulator could not build the dataset . . . 83
7.2 A batched integrator on the GPU . . . 84
7.2.1 Design . . . 84
7.2.2 Agreement with the reference . . . 84
7.2.3 Step size . . . 85
7.2.4 Launch overhead and CUDA graphs . . . 85
7.2.5 Precision . . . 86
7.2.6 Throughput . . . 87
7.3 Scenario generation . . . 87
7.3.1 Driving regimes . . . 87
7.3.2 Randomized scenarios . . . 88
7.4 Dataset format . . . 88
7.5 The two corpora . . . 89
7.6 Validation . . . 90
7.7 Known limitations of the corpora . . . 91
7.8 Chapter summary . . . 92

8 Surrogate Architecture **93**
8.1 What a neural network contributes here . . . 93
8.2 The unit of prediction: one fixed step . . . 94
8.3 What to anchor the step on . . . 95
8.4 Channel by channel . . . 97
8.4.1 Actuator lags: imposed exactly . . . 98
8.4.2 Velocity: the one learned quantity . . . 98
8.4.3 Position: imposed, except for the shape of the train . . . 98
8.4.4 What the division achieves . . . 99
8.5 The network . . . 99
8.5.1 The graph and its features . . . 100
8.5.2 Encoding . . . 101
8.5.3 Message passing . . . 101
8.5.4 How many rounds: depth from the physics . . . 103
8.5.5 Decoding . . . 103
8.5.6 Size, and independence from train length . . . 104
8.5.7 Implementation . . . 105
8.6 Training objective . . . 105
8.7 Relation to graph network simulators . . . 105
8.8 Chapter summary . . . 105

9 Training and First Results **107**
9.1 Experimental setup . . . 107
9.1.1 Data . . . 107
9.1.2 Batches of one consist size . . . 108
9.1.3 Optimization . . . 108
9.1.4 Metrics . . . 108
9.1.5 Baselines . . . 109
9.2 Keeping a rollout stable . . . 109
9.2.1 Noise injection . . . 110
9.2.2 Pushforward . . . 111
9.2.3 Checking the rollout itself . . . 111
9.3 Results . . . 111
9.3.1 One step from a true state . . . 112
9.3.2 Rolled out . . . 112
9.3.3 Longer consists . . . 114
9.3.4 An unseen corridor, and a steep one . . . 114
9.3.5 Against constant acceleration . . . 114
9.4 Coupler shape on long consists . . . 115
9.4.1 What the error is . . . 115
9.4.2 A constraint that should have helped, and did not . . . 115
9.4.3 Velocity error is not the cause . . . 116
9.4.4 A failure of stability, not accuracy . . . 116
9.5 Cost of a step . . . 117
9.6 What Part I claims . . . 118

9.7 Chapter summary 119

II Interrogating the Simulator 121

10 Critique: What the First Experiment Rests On 123
10.1 A corridor that was not on the track 123

11 In-Train Forces: Draft Gear, Traction, and the International Benchmark 125

12 Air Brakes, Resistance, and the v3 Simulator 127

III The Rebuilt System 129

13 Corridors at Scale and the v3 Corpus 131
13.1 Repairing the grade field 131
13.2 Scaling beyond a handful of corridors 132
13.2.1 The batch crawl 132
13.2.2 The North American Rail Network 133
13.2.3 From network to corridors 133

14 The Surrogate, Revised 135

15 Results II 137

16 Fuel-Optimal Control Demonstration 139

17 Discussion and Conclusion 141

Bibliography 143

List of Figures

2.1 Quantile calibration: diagonal = perfect; uncalibrated model deviates; isotonic calibration corrects. 39

3.1 Conceptual pipeline from structured inputs to tensorized simulator outputs, surrogate forecasting, and fuel-optimal control. 51

4.1 Stage 1 reference run: single vehicle on level track with a smooth traction ramp. Top to bottom: position, speed, and net longitudinal force $F_{\text{trac}} - R(v)$ versus time. 55

4.2 Stage 2: two-body train with linear coupler. Speeds of lead and trailing vehicle (overlapping at this scale), coupler force on the rear car, and extension δ. . . 56

4.3 Stage 3: comparison of linear spring-damper force (Stage 2 parameters) versus nonlinear slack coupler force along the same trajectory, with extension and relative speed. 57

4.4 Stage 4: three-body train with nonlinear couplers. Speeds, coupler forces F_0 and F_1, and end-to-end stretch versus time. 59

4.5 Stage 4 optional scenario: lead-only brake pulse; speeds and coupler forces. . 60

4.6 Stage 5: N-vehicle train; selected speeds and coupler forces versus time. . . . 60

4.7 Stage 5: coupler force heatmap (force versus time and coupler index). 61

4.8 Stage 6: interpolated grade profile $\sin\theta$ versus route position, with lead and rear speeds and coupler forces versus simulation time. 62

4.9 Stage 7: lead speed with brake-command onset marker; first-order lag states z_{brk} and z_{trac} on the lead versus time. 63

5.1 Overview of the route acquisition pipeline, from raw OpenStreetMap track data to the route-tensor fields consumed by the reference simulator. 65

5.2 Railway mapping around Pueblo, Colorado, rendered from OpenStreetMap track data. Mainline corridors (orange) run through a dense tangle of yard and service track (brown) and meet at several junctions. There is no single map object corresponding to "the line through Pueblo"; corridors have to be recovered from the segments. 66

5.3 The acquisition panel of the route application. A selection mode (radius, bounding box, or relation identifier) is chosen on the right; the centre point, radius, admissible railway types, and a minimum corridor length are set on the left. The mainline-only filter excludes yards and spurs, and the bounded outward crawl described in section 5.1 is toggled at the bottom. Several Overpass endpoints are selectable because any one of them may be rate-limiting at a given moment. 68

5.4 Inspection view for a resampled route: a 49.1 km corridor rewritten onto a uniform 10 m grid, yielding 4920 points. The plan view distinguishes points that sit on an original map vertex from those interpolated across long source segments (713 of them here), so stretches where the geometry is inferred rather than mapped remain visible downstream. 69

5.5 An elevated route rendered in three dimensions, coloured by elevation along the line, with the mapped ground track shown alongside. Visual inspection at this stage catches corridors that have been stitched incorrectly or that jump between parallel alignments, before any derivative is taken. 70

5.6 Ground elevation against arc length for the same corridor, climbing from roughly 1430 m to 1740 m over 49 km. This is the raw signal from which grade is derived; the metre-scale error it carries is negligible against the elevation range plotted here but is large against the elevation *change* between adjacent 10 m samples, which is what motivates the smoothing described below. 71

5.7 The three derived fields the simulator consumes for `route_line0`, plotted against chainage in kilometres: grade $\sin\theta(s)$ as a percentage (top), the curvature proxy $\kappa(s)$ (bottom left), and the speed limit $v_{\max}(s)$ (bottom right). Fields are shown as used to build `data/v2`, with a 200 m smoothing window and a 4% grade limit. The noisier grade over the first 20 km is the urban approach, where frequent bridges, junctions, and short mapped segments degrade the elevation signal. There the grade reaches the 4% limit in several short spikes, 1.3% of the corridor's length. Beyond it the line settles onto a sustained climb near one percent. The speed limit is a step field, here rising where the line leaves restricted urban trackage. Figure 13.1 shows the same corridor after the grade field was revised. 72

6.1 RailLab's layering. Screens call services; services are the only modules that import the numerical backend, and they return plain data. The backend packages are unchanged by the application's existence and continue to run headless. 76

6.2 The CONSIST screen: a library of reusable car types (left), the ordered train under construction with per-car overrides (centre), and saved trains (right). The running summary reports car count, total mass, total length, and the number of powered vehicles. 78

6.3 The CONTROLS screen: randomization bounds and distributed-power settings on the left, with a live preview of the generated traction command per power role (top right) and the shared brake command (bottom right). The offset between the three traction traces is the configured lag of the mid-train and rear helper units behind the head end. 79

6.4 The RESULTS screen after a 120 s run of a 126-vehicle consist under a randomized control profile. Every vehicle's speed is plotted above every coupler's force, on time-linked axes. The information is carried by the spread between traces: the widening and collapsing speed envelope is slack running in and out, with the corresponding fan of coupler forces below. 81

8.1 The size of the remaining learning target under each anchor, on a logarithmic scale. (a) For velocity, every physics anchor leaves a larger target than predicting the change directly. (b) For position, anchoring on the physics reduces the target more than a hundredfold. The best choice in each panel is marked in red. Data from tables 8.1 and 8.3. 96

8.2 How each state channel is advanced over one step. A design that corrects the physics would learn a correction on every channel (centre). Here two channels are imposed exactly, position is imposed apart from the coupler shape, and the network's output narrows to one scalar per vehicle and one per coupler (right). 97

8.3 The surrogate as a whole. Node and edge features are encoded into 128-dimensional vectors, updated by five rounds of message passing along the chain, and decoded into the two learned outputs. The reconstruction step turns those into the next state exactly as in eq. (8.6). In a rollout the new state is fed back as the next input. 100

8.4 One round of message passing. (a) Each coupler updates from the two vehicles it joins. (b) Each vehicle then updates from the couplers ahead of and behind it, kept as separate inputs. (c) At the ends of the train a zero vector stands in for the missing coupler. (d) Each update is a two-layer MLP followed by layer normalization, added to the current vector through a residual connection. 102

8.5 Receptive field of vehicle i after r rounds of message passing. After r rounds it has received information from vehicles up to r couplers away, and no further. A coupler force wave travels 167–193 m/s, about three vehicles each way in one 0.25 s step at the median and five at the 99th percentile over a 17 m car pitch, so five rounds cover the physical reach of one step. 104

9.1 Rollout error against horizon on four splits, on logarithmic axes. Solid lines are the model (mean over three seeds, with the range over seeds shaded). Dashed lines are holding the starting state on the same split. (a) Velocity error stays more than an order of magnitude below holding on every split. (b) Stretch error does too on `val` and on the unseen corridor, but on long consists (`ood_size`) it rises to meet the holding baseline. 113

13.1 The same fields as fig. 5.7, for `route_line0` after the raw-elevation check, with an 800 m smoothing window and a 2.5% grade limit. The urban approach over the first 20 km is still the noisiest stretch, but the grade reaches the 2.5% limit only in two short runs, near the start and near 12.7 km, which are residual artifacts rather than terrain. 132

List of Tables

1.1 Reported computational cost of physics-based rail simulation, and the reduction achieved by a learned surrogate. Figures are quoted as reported by the cited studies; they are not normalized to a common hardware baseline. . . . 21

2.1 Representative longitudinal train dynamics (LTD) simulation tools 27

5.1 Grade statistics for the five corridors as used to build `data/v1` and `data/v2`, with a 200 m smoothing window and a 4% grade limit. 71

7.1 Cost of the NumPy reference simulator on 120 s scenarios, with a 30 s wall-clock budget per run. At $N = 130$ no run finished within budget. 84
7.2 Maximum velocity error of the GPU integrator against the ten tightly integrated reference scenarios, in double precision. 85
7.3 Effect of launch overhead on the GPU integrator, for $B = 64$ scenarios of $N = 130$ vehicles and 120 s of simulated time. 86
7.4 Cost of double precision relative to single precision for 120 s of simulated time. 86
7.5 Wall clock of the GPU integrator for 120 s of simulated time, in single precision, against batch size B (left) and consist size N (right). 87
7.6 Arrays stored for each scenario. Each file also holds the initial state, the integrator constants needed to recompute the right-hand side, trip-level summaries, and a metadata record. 89
7.7 The two corpora. `data/v2` is the training corpus; `data/v1` exists to measure the curvature term. 89
7.8 Splits of each corpus. 90
7.9 Maximum mismatch between the recomputed right-hand side and a central difference of the stored trajectory, for the position and velocity rates of one scenario ($N = 31$, cruise) re-sampled at finer output spacing. 91

8.1 Size of the remaining learning target for velocity under each anchor: the largest velocity change over the consist that the network would still have to supply, in m/s, over 480 transitions from 40 training scenarios of `data/v2`. . 95
8.2 Natural frequency of the two-vehicle coupler mode, over 60 sampled scenarios. 96
8.3 Size of the remaining learning target for position under each anchor, in metres, on the same transitions as table 8.1. 97
8.4 Largest per-step error of the trapezoid rule computed from true velocities, against predicting no change, over 30 training scenarios. 98

8.5 Parameter count of the network. None of the counts depends on the number of vehicles. . . . 104
8.6 The original graph network simulator and the model of this chapter. . . . 106

9.1 Splits of `data/v2` used in this chapter (table 7.8). No consist in the library has between 81 and 119 vehicles. . . . 108
9.2 Rollout velocity error, the largest over the consist in m/s (median over starts), with and without noise injection during training. Development runs of 3,000 updates, one seed, validation split. Without noise the model is worse than holding the state after about 20 steps. . . . 110
9.3 Rollout velocity error in m/s at four horizons, and one-step velocity error, against the size of the injected velocity noise. Development runs of 3,000 updates, one seed. Differences under about 20% are within seed variation. . . . 110
9.4 Rollout velocity error in m/s at four horizons, stretch error at 200 steps, and training time, for pushforward with 1 to 16 unsupervised steps. Development runs of 3,000 updates, one seed. . . . 111
9.5 One-step error of GNS-1, mean over three seeds, with the factor by which it improves on zero output and on constant acceleration (the baseline's error divided by the model's). . . . 112
9.6 Rollout error of GNS-1 after 10, 50 and 200 steps (2.5, 12.5 and 50 s): the largest error over the consist, median over starts, mean over three seeds. "vs hold" is the factor by which the model improves on holding the starting state at 200 steps. . . . 113
9.7 The signed per-coupler stretch error after a 200-step rollout, on a short and a long consist. The last row gives the mean absolute error in each quarter of the consist, front to rear. . . . 115
9.8 Stretch error at 200 steps in mm, with the edge head constrained to a zero consist-mean and without. Three seeds. . . . 116
9.9 Stretch error at 200 steps with and without the true velocity field forced into the rollout. . . . 116
9.10 Wall clock for 120 s of simulated time at $N = 130$ on one RTX 4090: GNS-1 rolled out in plain PyTorch, against the GPU integrator replayed as a CUDA graph in single and in double precision (tables 7.4 and 7.5). The corpora were integrated in double precision. . . . 117
9.11 What Part I claims. Chapter 10 examines the third column. . . . 118

10.1 Roughness of the raw elevation, before any smoothing, for the five acquired corridors on a 10 m grid. `route_line2` exceeds a one-metre step on nearly a fifth of its length, against 0.7–1.5% for the others. Its bridge and tunnel share is under 1%, so structure correction does not explain the difference. . . . 124

13.1 Grade statistics for the four corridors that pass the raw-elevation check, with an 800 m smoothing window and a 2.5% grade limit. . . . 131

Part I

A First Surrogate

Chapter 1

Introduction and Problem Motivation

1.1 Background: Longitudinal Train Dynamics

Longitudinal train dynamics (LTD) concern the motion and force distribution through the length of a train. A freight train is a distributed mechanical system, often consisting of tens to hundreds of individual coupled masses. The system involves complex and highly nonlinear interactions between locomotives, freight cars, and the track profile, and most of that nonlinearity sits in the couplers: slack, asymmetric stiffness in buff and draft, and damping, acting as stiff elements between very large masses. Modern simulations of these interactions rely on detailed physical models which solve differential equations for each vehicle, and the stiffness of the couplers forces the numerical solver to take small internal steps. The result is a high computational cost, and rapid scenario exploration can be impractical because of it.

Task	Reported cost	Source
Whole-trip LTD optimization study, sequential execution	∼18 months	[1]
Same study, parallel execution across multiple processors	11 days	[1]
Coupler-force evaluation, share of LTD solver runtime	∼65%	[2]
Multibody vehicle–track simulation of a 5 km segment	∼30 minutes	[3]
Learned surrogate for the same 5 km segment	∼8 seconds	[3]

Table 1.1: Reported computational cost of physics-based rail simulation, and the reduction achieved by a learned surrogate. Figures are quoted as reported by the cited studies; they are not normalized to a common hardware baseline.

Table 1.1 collects the costs reported in the literature. Two features of it matter for what follows. The first is scale: a whole-trip optimization study that must evaluate many candidate driving plans is measured in months of sequential computation. The second is where the cost concentrates. In the study of [2], roughly two thirds of the solver's runtime was spent evaluating coupler forces, which is to say on the same stiff, nonlinear interactions that force the solver's step to be small in the first place.

1.2 Surrogate Modeling

High-fidelity physics simulators are essential for understanding complex physical systems, but their cost places a hard limit on how they can be used. They are invaluable for validating theory and for studying a single scenario in detail. They are impractical wherever the same simulation must be repeated many times with slightly different inputs, which is exactly the pattern of control design, operational planning, and optimization: each candidate plan is a full simulation, and the number of candidates is large.

Surrogate modeling addresses this problem by learning an approximate model of the simulator from its own outputs, one that can be evaluated at a small fraction of the original cost. The simulator is not discarded. It generates the data from which the surrogate is trained, and it remains the reference against which the surrogate is judged. What changes is which of the two is called in the inner loop of an optimization.

How the physics enters such a model is the central design choice, and there are two broad options. A learned model can sit alongside the known equations and correct their output, or the known equations can be built into the structure of the learned model, so that the parts of the dynamics that are exactly known are never learned at all. This thesis takes the second route, and in the strongest form available for this problem. Two properties of a coupled vehicle chain make that possible.

The first is that its structure is given. A consist is a chain of vehicles joined by couplers, and the force on any vehicle depends only on the two couplers attached to it. Treated as a graph, with vehicles as nodes and couplers as edges, the train already has the shape of a message-passing neural network [4]: the coupler law is a function on edges and the force balance is an update on nodes. A model built this way has no fixed input length. The number of vehicles is a property of the input rather than of the architecture, so a single model can be applied to a train of any length, including lengths it was never trained on.

The second is that most of the state evolves by relations that are known exactly. Position is the integral of velocity. The brake and traction actuators follow first-order lags whose commands are known in advance and whose solution is available in closed form. Of the four dynamic channels carried per vehicle, only velocity, and with it the fine shape of each coupler's extension, genuinely requires a learned model. The surrogate developed here is therefore a *graph network simulator*, after [5]: it advances the state of the whole consist by one fixed step of 0.25 s, with the network predicting only the mean acceleration of each vehicle over the step and a small per-coupler correction to its extension, while position and the actuator states are reconstructed exactly. Adding the learned model to the physics would have left every channel to the network; building the physics into the step removes two channels from it entirely, and narrows what is learned to two scalars per vehicle. The physics imposed on the model goes up, not down.

The fixed step is also where the speed advantage comes from. A learned update is not bound by the stability limit of the explicit scheme it replaces [6], so it can take a step at which the reference integrator could not. Between two stored samples the reference integrator evaluates its right-hand side fifty times; the surrogate replaces all fifty with a single network evaluation. Because that ratio is fixed by the integrator's own configuration, the saving can be stated without appeal to any particular hardware comparison.

1.3 Decision Support and Fuel-Optimal Control

A fast model is useful for forecasting: it can answer what a given driving plan will do to fuel use, trip time, and in-train forces, many times over and at low cost. The more demanding use, and the one that shapes the design in this thesis, is control. A model that is also differentiable with respect to its inputs can report not only the outcome of a plan but how that outcome would change if the plan were adjusted, and that is the quantity an optimization routine needs in order to improve the plan by itself.

The target application is therefore fuel-optimal driving of a freight consist: choosing traction and braking commands that reduce trip energy while respecting speed limits and keeping coupler forces within safe bounds, using model-predictive control with the surrogate as its internal model. This places two requirements on the surrogate beyond speed. It must be differentiable end to end, which the fixed-step formulation helps rather than hinders, since no adaptive solver sits between the control plan and the predicted trajectory. And its coupler-force predictions must be trustworthy enough to serve as a safety constraint, because that is the quantity whose violation matters most.

Uncertainty enters in the same narrow role. Vehicle masses, resistance coefficients and coupler parameters are never known exactly, but this thesis does not build a full calibrated forecast of every output. Uncertainty is folded into the coupler-force safety margin that the controller is required to respect, and it is evaluated in that role (section 3.4).

1.4 Thesis Objective and Contributions

This thesis develops a fast, differentiable, train-agnostic surrogate of freight-train longitudinal dynamics, and the infrastructure needed to train and evaluate it. Its contributions are:

1. **A graph formulation of longitudinal train dynamics.** The consist is represented as node (vehicle) and edge (coupler) tensors, so that the simulator's outputs, the training targets, and the surrogate share one layout for any number of vehicles, and consist length becomes a property of the input rather than of the model.

2. **A reference simulator and a real-terrain data pipeline.** A longitudinal dynamics simulator built and validated one physical effect at a time, from single-vehicle traction through nonlinear slack couplers, per-vehicle route sampling, and power-limited traction with actuator lag; and a pipeline that turns open track-geometry and elevation data into the route profiles along which it is driven.

3. **A graph network simulator for longitudinal dynamics.** A message-passing network that advances the whole consist by one learned fixed step, with physics imposed channel by channel: position kinematic, actuator lags exact, and only the mean acceleration and a per-coupler shape correction learned. One network evaluation replaces fifty evaluations of the reference right-hand side.

4. **An evaluation of structural generalization.** A comparison against zero-output, hold-the-state, constant-acceleration and physics-residual baselines, with a transformer

variant as an ablation, on trajectory fidelity, trip-level summaries, and generalization to consist lengths and corridors not seen in training.

5. **A fuel-optimal control demonstration.** Model-predictive control using the surrogate's gradients to reduce trip energy on held-out routes subject to speed-limit and coupler-force constraints.

1.5 Scope

The simulator's parameters are illustrative rather than calibrated to a specific real train. The claims of this thesis therefore concern fidelity to the reference simulator, structural generalization across consist size and route, and control performance under that model. They are not claims about fidelity to real rolling stock, which would require measured data this work does not use.

An earlier statement of this work [7] proposed a physics-residual formulation, in which a network learns a correction to the known force-balance right-hand side; section 8.2 gives the measurements that led to its replacement by the fixed-step formulation described above.

1.6 Organization of Thesis

Chapter 2 reviews related work: classical longitudinal dynamics and its computational cost, data-driven and hybrid modeling in rail, transformer and neural-operator models of dynamical systems, graph networks for physical simulation, and probabilistic forecasting. Chapter 3 formulates the prediction and control problem, defines the node and edge tensors, and states the surrogate as a learned fixed step on that tensor state. Chapter 4 documents the reference simulator that generates the training data, chapter 5 the pipeline that builds real-terrain route profiles for it, chapter 6 RailLab, the desktop console used to explore both, and chapter 7 the batched GPU integrator and the two 10,000-scenario datasets built with it.

Chapter 8 presents the surrogate architecture and the measurements that fix its design. The chapters that follow present the training regime that keeps a single-step model stable over long rollouts, the evaluation against baselines and on held-out consists and corridors, and the control demonstration, before a discussion of implications and limitations and a concluding chapter on future work.

Chapter 2

Related Work

2.1 Classical Train Longitudinal Dynamics and Simulation

Longitudinal train dynamics (LTD) is the study of the physical interactions within a connected train body comprised of multiple linked masses, henceforth referred to as a *consist*. Classical longitudinal train dynamics is defined as the motion of rolling stock vehicles in the longitudinal dimension of the track, ignoring lateral or vertical forces in most cases. The motion of the whole train as well as the individual motions of each vehicle are accounted for in this schema [8].

Early attempts at LTD study treated a train as a continuous elastic rod with a traction force applied at one end, and enabled the study of longitudinal vibration phenomena. This later progressed to discretized mass-spring chains. Early models such as these generally ignored discontinuities, such as those arising from coupler slack, which enabled closed form solutions that required no numerical solver. Nonlinearities were identified early, such as draft gear behavior, but could not be simulated with the level of computing power at the time [9].

2.1.1 Modeling Abstractions and Levels of Fidelity

LTD can be framed as layered abstractions with increasing levels of complexity. At the greatest order-reduction level, LTD can be modeled as point mass or a single-body train model. This level of complexity can be useful for modeling energy usage or scheduling, but is highly limited in the observability of the system, i.e. many variables are unaccounted for. At the next level of abstraction, the consist can be modeled as a multi-mass system, where each vehicle is accounted for as a rigid body with one longitudinal degree of freedom. Masses are connected with modeled couplers and draft gears, which introduce nonlinear interactions. Beyond this, we can extend the DOF of the model to account for motions in the remaining spatial dimensions, which allows for observability track curve effects, in train instabilities, and even derailment scenarios; such models are formulated using the general multibody dynamics machinery of constrained rigid and flexible bodies [10]. This scale of modeling naturally trades speed for fidelity, as computation of so many parameters is highly expensive, but the high fidelity is necessary for some analyses [9].

2.1.2 Computational Schemes and Numerical Challenges

The standard computation schema for LTD treats the problem as an repeated evaluation of all force elements followed by state propagation [2]. In general, a simulator first identifies force elements (propulsion resistance, curving resistance, grade components, gravity, traction forces, dynamic brake forces, air brake forces, and coupler forces). Then, the system computes each element from the inputs (time, vehicle velocities, accelerations, and vehicle locations). Finally, velocities and positions are updated through a numerical solver. The clear modularity of this computing system allows for addition or replacement of new computing models, such as replacing a module with a learned surrogate model.

The critical difficulty with computing LTD is with the vehicle connection systems, specifically the couplers, draft gears, and drawbars. Friction draft gears exhibit extreme nonlinearities, including hysteresis and stick-slip) that are hard to both model and numerically solve [9]. The coupler force calculation accounted for approximately 65% of the computing time in the study described in [2]. That the cost concentrates so heavily in one force element is what makes a surrogate attractive here: the expensive part of the calculation is also the nonlinear, discontinuous part, and it is the part a learned model is best placed to absorb while the remaining force terms stay analytic.

2.1.3 Benchmarking, Applications, and Limitations

LTD has historically lacked standardized procedures for benchmarking LTD simulators, which limits objective comparison of tools developed by different institutions. Spiryagin et. al. introduced an international benchmarking initiative to define a mutual set of input scenarios, operating conditions, and necessary outputs to compare efficacy of various simulators [11]. The benchmarking initiative was motivated by the fact that most LTD simulation tools have evolved independently as in-house software for the many institutions that use them. This resulted in limited to no transparency in terms of numerical methods, assumptions, and performance. Participating simulators used identical train configurations, commands for traction and braking, and track profiles. Outputs were measured in high temporal resolution to enable detailed comparison. This work provides an indispensable reference point for evaluating simulation fidelity and computational performance. The work further motivates the need for reproducible, scenario-based evaluation systems.

The applications of LTD, especially rapidly evaluated scenarios, is further highlighted in the Wu-Spiryagin-Cole paper. Driving advisory systems are integrated computation engines which advise locomotive operators to take actions based on instrumented vehicle data. These systems require real time evaluation of LTD outputs. Furthermore, energy consumption calculations can be evaluated from a simulator with observable traction/braking forces and speed/distance signals. Such energy estimates are only as accurate as the model's treatment of the interactions that are hardest to capture, chiefly those at the couplers. Fatigue and safety analyses depend on time-domain histories of dynamic interactions, which can be invaluable for risk assessment.

Table 2.1: Representative longitudinal train dynamics (LTD) simulation tools

Tool	Origin	Scope	Availability
TrainDy [12]	UIC-oriented (academic/industry consortium)	Freight LTD for operations and safety analysis	Specialized / consortium
TOES™ [13, 14]	Developed by the AAR, licensed to member railroads	Comprehensive train dynamics for operating scenarios	Commercial / licensed
TEDS [15, 16]	U.S. FRA-sponsored (contractors)	Safety/risk evaluation, incident investigation, operations LTD	Publicly documented
UM Train [17]	Universal Mechanism suite	Train longitudinal dynamics within multibody ecosystem	Commercial
TDEAS [18]	Developed in China (research/industry)	LTD + energy analysis	Research / institutional
CRE-LTS [11]	Centre for Railway Engineering (CQU)	Benchmarking-ready research LTD simulator	Research / in-house

2.2 Data-Driven and Hybrid Modeling in Rail

The computational expense and architectural complexity of high-fidelity rail models have motivated a growing body of work on data-driven and hybrid modeling approaches. These methods utilize machine learning for acceleration of classical simulations, estimation of unmeasured internal quantities, or enabling of real-time decision support. In general, models maintain an interpretable physics-based backbone where possible, but this line is blurred when more advanced "black-box" models are introduced. Existing efforts, in general, fall into three overlapping categories: surrogate replacement of expensive physics submodels, data-driven estimation of in-train forces from operational data, and surrogate modeling for optimization and fleet/network level decision-making.

2.2.1 Surrogate Models for Accelerating Physics-Based Simulation

One prominent line of research focuses on embedding learned surrogate components within classical multibody or finite-element-based simulation frameworks. Instead of replacing the entire physics model with a "black-box" technique, these approaches substitute the most computationally intensive or nonlinear subcomponents with data-driven approximations trained on simulation data. For example, Nie et al. train surrogate elements on finite-element outputs and substitute them for the corresponding mechanical elements inside a multibody vehicle dynamics simulation, coupling MATLAB/Simulink with SIMPACK; they report that a Legendre polynomial regression surrogate substantially reduces computing time without sacrificing accuracy [19]. Polynomial regression, Kriging, and other response-surface techniques are commonly employed in this setting, as they are highly interpretable and allow scientists and engineers to make grounded deductions.

More recent work has emphasized system integration of machine learning surrogates into digital twin architectures. In this schema, a full multibody dynamics (MBD) model is replaced by a neural network based surrogate and packaged using standardized interfaces such as the Functional Mock-up Interface (FMI). This enables deployment as a Functional Mock-up Unit (FMU) within larger co-simulation or decision-support environments [3]. Zhou et al. report that a recurrent neural network surrogate evaluates a 5 km track segment in roughly eight seconds, against approximately thirty minutes for the equivalent multibody simulation in SIMPACK, and they demonstrate a practical pipeline for continuous model deployment. However, this line of work remains primarily focused on reproducing nominal simulator behavior rather than modeling uncertainty or difficult component interactions.

2.2.2 Data-Driven Estimation of In-Train Forces

A second major branch of data-driven rail modeling targets the estimation of in-train forces, which are difficult to measure directly in operational freight and passenger trains. Leveraging the increasing availability of onboard telemetry, particularly from Automatic Train Operation (ATO) systems, several authors have proposed supervised learning approaches that map operational signals such as speed, acceleration, and control commands to coupler force estimates. Convolutional neural networks (CNN) augmented with causal structure and self-attention mechanisms have demonstrated accurate reproduction of simulator-derived force histories under typical operating conditions [20].

Extensions of this concept incorporate multi-task learning and transfer learning to generalize force estimation across multiple couplers and train configurations. The In-trainNet framework, for example, first pre-trains a multi-task model that estimates forces at several couplers of one train configuration, then adapts that pre-trained representation to different configurations through transfer learning, substantially reducing the volume of training data each new configuration requires [21]. While these approaches represent an important step toward scalable "virtual instrumentation" of in-train forces, they are generally trained on labeled data generated by longitudinal train simulators and are validated on limited scenario sets.

2.2.3 Surrogate Modeling for Operations and Optimization

Data-driven surrogates have also been employed to accelerate optimization problems in railway operations, transcending the low-level dynamics and force estimation problems and solving high-level, or even fleet level problems. In the paper from Liu et. al., surrogate models are used to approximate the evaluation of candidate rail schedules, significantly reducing the amount of expensive computation required during optimization [22]. These methods search over multiple surrogates jointly and employ transfer learning, in this case to compensate for incomplete and imbalanced historical disruption data rather than for computational cost alone. Although effective in reducing computational burden, such surrogates are typically designed for specific operational contexts and rely on deterministic predictions of performance metrics for validation.

2.2.4 Limitations of Existing Data-Driven and Hybrid Approaches

Despite demonstrated success on several rail domains, existing data-driven and hybrid rail models exhibit several notable limitations. First, the majority of proposed surrogates are deterministic, providing point estimates without measures of predictive uncertainty. This limits their reliability when deployed outside the training distribution or when embedded within control and optimization loops where constraint violations can have significant consequences.

Second, generalization across operating schemes and consist configurations remains a persistent challenge. Force-estimation models of this kind are typically developed for a single coupler position within a single train configuration, so a separate model must be trained for each coupler and each consist of interest [20]. Subsequent work addresses this limitation directly through multi-task pre-training and transfer learning across configurations, but still requires configuration-specific adaptation data rather than generalizing to an arbitrary consist without retraining [21]. These issues are exacerbated by the reliance on simulated labels, as discrepancies between simulation and real-world behavior can compound errors in purely data-driven mappings.

Finally, while hybrid surrogate approaches retain a physics-based structure, they are generally aimed at replication of nominal simulator outputs rather than modeling of epistemic uncertainty arising from unmodeled dynamics, parameter variability, or environmental disturbances. As a result, current surrogates offer limited robustness under distribution shift, constraining their applicability to safety-critical tasks such as energy-optimal control or real-time operational decision-making. Furthermore, the lack of baseline physical interpretability limits the industry adoption of new simulator technology, as rail engineers and scientists may be skeptical about a fully black-box prediction model (such as neural network).

2.2.5 Implications for Structural Generalization

Collectively, this body of work demonstrates that data-driven methods can accelerate railway simulation substantially, while also exposing a consistent structural limitation: each surrogate is tied to something particular. A model is trained for one coupler position, one consist configuration, one track segment, or one operational context, and a change in that quantity requires retraining or configuration-specific adaptation data. The generalization sought in this thesis is therefore not primarily statistical but structural. What is required is a model whose representation of a train does not encode a particular train, so that consist length becomes a property of the input rather than of the architecture. The remainder of this chapter reviews the two families of architecture that might plausibly supply that property.

2.3 Transformers for Dynamical Systems and Spatiotemporal Forecasting

Transformer architectures demonstrate strong candidacy for general purpose sequence modeling tasks. They have been increasingly adopted for dynamical systems modeling research and spatiotemporal forecasting. Transformers enable adaptive context aggregation over long

temporal horizons by replacing recurrence with multi-head self attention. Unlike recurrent architectures, transformers do not impose a sequential dependency in their core computation. This intrinsic algorithmic feature allows transformers to be highly parallelizable during training. Transformers are particularly attractive for dynamical systems where future states often depend on long range temporal interactions, as trajectories can be represented as sequences of states or latent tokens [23].

2.3.1 Long-horizon Forecasting Transformers

A major branch of work in transformers is focused on long horizon temporal forecasting. Standard self-attention scales quadratically with sequence length, which severely limits it's applicability to long-history prediction problems where fine-temporal resolution is necessary for accuracy. Informer addresses this problem by introducing ProbSparse attention, which identifies and attends to the most informative queries. This enables reduction in the computational cost for training in long horizon scenarios, but preserves enough necessary information to retain prediction accuracy. Compared to conventional models, Informer is capable of forecasting over substantially longer periods [24].

Autoformer is an additional architecture proposed by Wu et. al. which is motivated by the observation that many real-world time series phenomena (such as energy demand, traffic flow, and economic trends) are dominated by two distinct patterns. These include a slowly varying yet long-term trend, and repeating or periodic behavior (seasonal). At great computational cost, a neural network could be used to discover these structures implicitly. Autoformer builds these phenomena directly into the model, where the input time series is first decomposed into trend and seasonal components to enable the model to address long term evolution and short term noise/fluctuations separately. Further, Autoformer replaces scaled dot-product attention with an auto-correlation mechanism that explicitly searches for repeating temporal patterns across various time windows. If recurrent structures are identified, they can be reused more efficiently. Thus, Autoformer improves prediction accuracy and stability for long time series systems by reducing the computational burden of learning every single interaction between time steps [25].

Another promising architecture is the Temporal Fusion Transformer, which are designed for practical forecasting problems where predictions depend on many different types of information, not just a single time series data history. In applications, forecasting requires a combination of static attributes that do not change over time (such as system configuration or asset type), inputs that are known in advance (such as schedules, calendars, or planned control actions), and external variables that are observed over time but not controlled (such as weather or demand signals). Temporal fusion transformers partition these inputs instead of processing them all as a single, aggregate sequence. The model features mechanisms to determine which variables are the most relevant for prediction at each point in time. This structure allows TFT to generate multi-step forecasts while also providing interpretable information about which inputs most strongly influenced the predictions and when they were important, making the model more transparent and suitable for decision-support applications [26]. These properties make the model more transparent and interpretable, as compared to a black-box model.

2.3.2 Neural Operators for PDE-governed Dynamics

For spatiotemporal physical systems, forecasting can be cast as learning a solution operator mapping initial/boundary conditions (and coefficients/forcing) to future fields. Fourier Neural Operators (FNO) address forecasting and dynamical modeling problems from predicting individual timesteps to learning the governing function for how a system evolves. Instead of operating at specific values, FNO learns how information propagates across the entire spatial domain at once. As the name suggests, it does this by transforming the system state into the frequency domain, where long-term or broad patterns can be analyzed and computed much more efficiently. Within this new domain, the model applies global updates that resemble system-wide interactions rather than local point to point updates. As these learned updates are defined independently of any spatial configuration, the resulting model can be evaluated on finer or coarser spatial grids than those seen during training. This allows FNO based models to generalize across different spatial resolutions, making them well suited as fast, flexible surrogates for complex dynamical simulation [27].

Physics-informed neural operators (PINO) expand on operator-learning models by explicitly incorporating known physical laws into the training process. In many engineering or scientific systems, the governing equations for how the system evolves over time are at least partially known. PINO leverages this knowledge by penalizing violations of the underlying physical formulation (the differential equations governing the system) during training, rather than relying solely on error measurements from data. In practice, the model may be trained using available simulation or measurement data at a relatively coarse spatial or temporal resolution, while simultaneously evaluating how well predictions satisfy the governing physical equations at a finer resolution. The resulting model is a generalizable forecaster that respects known physical constraints. The framework allows for the creation of models even when data is limited or expensive to obtain [28].

Beyond representations that rely only on Fourier representations, Cao investigates how attention mechanisms (which were originally created to model sequences) can be used to directly learn the rules of evolution for dynamical systems [29]. The study compares Fourier-based representations, which emphasize global frequency patterns, with Galerkin-style representations, which are rooted in classical numerical methods that approximate solutions using weighted basis functions. In analyzing how self-attention interacts with these different representations, the work shows that no single formulation is universally optimal. The choice depends on the structure of the underlying system and the types of interactions that dominate its dynamic evolution. This perspective reframes operator learning as an architectural design problem, where decisions about how to represent space, enforce structure, and aggregate information are as important as the learning algorithm itself.

More recently, attention-based models have been adapted specifically for learning the evolution of systems governed by partial differential equations. In this line of work, Transformer-style architectures are reformulated to learn solution operators that map initial conditions, system parameters, and external inputs directly to future system states. As opposed to advancing the state forward step by step in time, the model learns a global input–output relationship that captures how the entire state evolves. The attention mechanism enables the model to aggregate information across spatial locations and time, allowing it to represent long-range interactions that are common in physical systems. Utilizing Transformer architec-

tures within an operator-learning framework and evaluating them on canonical PDE benchmarks, this work demonstrates that attention-based models can serve as effective and general surrogates for complex spatiotemporal dynamics, further validating the use of Transformer-style mechanisms in scientific and engineering modeling contexts [30].

2.3.3 Implications for Train Dynamics and Spatiotemporal Decision Support

These elements motivate hybrid modeling strategies for systems like train longitudinal dynamics where dynamics are structured (physics, constraints, conservation-like relationships) yet exhibit long-range dependencies (consist interactions, route profile memory, control plans). Forecasting-oriented Transformers (Informer/Autoformer/TFT) supply scalable mechanisms for long-horizon sequence prediction and covariate fusion [24–26], while neural-operator perspectives (FNO/PINO and attention-based operator learners) suggest a principled route to learning dynamics as operators that map route/consist/control inputs to trajectories or distributions over outcomes [27–30]. These approaches collectively justify treating dynamical response prediction as (i) sequence-to-sequence forecasting with structured attention and (ii) operator learning with physics-aware regularization. Both, however, share an assumption that sits awkwardly with the problem of this thesis. A sequence model requires a token layout fixed in advance, and a neural operator a discretization over which the solution field is represented; in either case the number of vehicles in a consist must be accommodated by padding, masking, or resampling onto a common grid. Since consist length is precisely the quantity over which generalization is sought, encoding it as an architectural constant works against the objective. The following section reviews a family of architectures for which it need not be.

2.4 Graph Networks for Physical Systems

The architectures of the previous section treat a dynamical system as a sequence, and pay for that choice with a fixed token layout. An alternative line of work treats a physical system as what it structurally is: a set of entities and the interactions between them, that is, a graph. For a train this is not an analogy. A consist is a chain of vehicles joined by couplers, and the force on any vehicle depends only on the two couplers attached to it. The relational structure is given by the mechanics rather than inferred from data, which is precisely the setting in which graph-based architectures have the strongest claim.

2.4.1 Relational inductive bias and message passing

The interaction network of [31] introduced the pattern that underlies most of what follows. Objects are represented as nodes and their pairwise interactions as edges; a learned relation function computes an effect for each edge from the pair of objects it connects, and a learned object function then updates each node from the aggregated effects of the edges incident on it. The correspondence with longitudinal train dynamics is close enough to be worth stating explicitly: the coupler constitutive law $F_i = \phi_i(\delta_i, \dot{\delta}_i)$ is an edge function evaluated on the

two vehicles a coupler joins, and the force balance $m_i \dot{v}_i = F_{\text{ext},i} + F_{i-1} - F_i$ is a node update that sums the incident edge forces. The interaction network was also shown to generalize to systems with different numbers of objects than those seen in training, which is the earliest form of the property this thesis requires of consist length.

[32] consolidated several such architectures under the name *message passing neural networks*, factoring them into a message function computed on edges, an update function applied at nodes, and a readout that produces graph-level outputs. [4] generalized the construction further into the *graph network* block and, more usefully for the present argument, framed architecture selection as a choice of relational inductive bias: the structure built into a model encodes an assumption about which entities interact, and a model whose structure matches the system inherits that assumption rather than having to learn it. A chain graph over vehicles and couplers is exactly such an assumption, and it is one that holds by construction for any consist length.

2.4.2 Graph networks as learned simulators

[5] applied this machinery to simulation directly, in the work that gives the present architecture its name. Their graph network simulator represents the state of a physical system as particles at the nodes of a graph, encodes node and edge features into a latent space, applies a fixed number of message-passing rounds, and decodes a per-node acceleration which is then advanced by a fixed integrator update. The framework was demonstrated across fluids, rigid solids and deformable materials, and the authors report that models trained on single-step prediction with a few thousand particles remain accurate over thousands of timesteps and at an order of magnitude more particles than were present during training. Two of their conclusions bear directly on the design adopted later in this thesis: that the number of message-passing steps is one of the principal determinants of long-term accuracy, and that the learned quantity is an acceleration rather than a state, with the integration step imposed rather than learned.

[33] extended the approach from particles to meshes. Where a particle simulator must rebuild its graph from spatial proximity at every step, a mesh supplies the graph as a given, fixed topology, and the model passes messages over the mesh connectivity. A consist sits at the simpler end of this spectrum still: its graph is a path, its topology is known in advance from the consist manifest, and no neighbour search or mesh adaptation is required at any point.

2.4.3 Stability of learned rollouts

A model trained to take one step and then evaluated over many encounters a well-documented difficulty. During training it sees only states produced by the reference solver, whereas during a rollout it is fed its own output, so its errors carry it into regions of state space it was never trained on. [5] address this by corrupting the training inputs with random-walk noise, and identify that corruption, together with message-passing depth, as one of the two main determinants of long-horizon performance. [34] formalize a complementary technique, the *pushforward trick*, in which the model is advanced several steps on its own predictions without gradient and only the subsequent step is supervised, so that the perturbation seen

during training is drawn from the model's own error distribution rather than an assumed one; the same work proposes temporal bundling, predicting several steps jointly, as a means of reducing the number of distribution-shift events over a trajectory. [35] state the underlying principle in more general terms, showing that placing the learned component inside the solver loop during training, so that it observes inputs reflecting its own previous corrections, yields stable rollouts over several hundred recurrent evaluations. Chapter 9 returns to these techniques, both of which are used directly in this thesis.

2.4.4 Learned coarse models and the cost of a step

A distinct motivation for learned simulation is not fidelity but step size. [6] train learned models to reproduce turbulent dynamics at spatial and temporal resolutions substantially coarser than those of the generating simulation, and report that a learned model on a coarse grid can be more accurate than a classical solver on the same grid. The significance for the present work is that a learned update is not bound by the stability constraints of the explicit scheme it replaces, and may therefore take a step at which that scheme would fail outright. The saving is then proportional to the number of solver evaluations the learned step displaces, a quantity fixed by the reference integrator's own configuration rather than by any hardware comparison.

A related but distinct strategy retains the classical solver and learns a correction within it. [36] coarsen the spatial grid and use a learned component to recover accuracy comparable to a substantially finer baseline, reporting speedups of one to two orders of magnitude. The two approaches differ in what is kept: the former replaces the integrator over the coarse step, the latter preserves it and corrects its output.

2.4.5 Continuous-time alternatives

A further possibility is to learn the derivative rather than the step. [37] parameterize the right-hand side of an ordinary differential equation with a neural network and obtain the state trajectory by numerical integration, with gradients computed either through the unrolled solver or by the adjoint method. The formulation is attractive for physical systems because it separates the learned dynamics from the choice of integrator and admits irregular output spacing.

Its behaviour on stiff systems is a documented limitation. [38] examine neural ODEs on classical stiff problems and show that the scale separation characteristic of stiffness impedes training, proposing a set of mitigations in response. Since stiffness in a coupled vehicle chain arises from the coupler itself, acting as a stiff element between large masses, this is a property of the present system rather than an incidental one, and section 8.3 quantifies it for the consists considered here.

2.4.6 Graph learning in rail applications, and the remaining gap

Graph-based learning has begun to appear in railway engineering, though not yet for the problem addressed here. [39] construct a graph neural network surrogate for a train–track–bridge coupled system, modelling the interaction between a vehicle and the structure beneath

it; the coupling is vertical and lateral rather than longitudinal, and the relevant interaction is between a vehicle and the supporting structure rather than between vehicles. [40] apply a spatio-temporal graph network to the coupler and draft gear system of heavy-haul trains using in-service sensor data, which is closer to the present subject matter, but the task is condition monitoring and anomaly detection rather than forward prediction, and the graph is constructed over sensor channels with edges inferred by thresholding a mutual-information matrix rather than given by the mechanical topology. Outside rail, [41] apply the graph network simulator pattern to structural dynamics, setting the interaction range between mesh regions on wave-propagation grounds, an argument that recurs in chapter 8 of this thesis.

Taken together with the data-driven rail work reviewed in section 2.2.4, this leaves a specific gap. Learned graph surrogates have been applied to particles, meshes, structural vibration and vehicle–structure interaction; within rail they have been used for diagnosis from measurements and for vertical and lateral vehicle–structure coupling. They have not been applied to the forward longitudinal dynamics of a coupled consist, where the graph is a chain given by the consist itself and the edge law carries a deadband. The estimators of in-train forces discussed earlier operate on the correct physical axis but are regressions from telemetry fixed to a particular coupler position and consist configuration, rather than dynamics models that advance a state under a control plan. It is that combination, a train-agnostic forward model on the longitudinal axis with the graph supplied by the mechanics, that the remainder of this thesis develops.

2.5 Probabilistic Forecasting and Uncertainty Calibration

In engineered systems, point predictions alone are not sufficient for supporting reliable decision-making. In safety-critical domains, decisions must account for both the expected system behavior as well as the likelihood and severity of rare or adverse outcomes. In such environments, uncertainty is not a nuisance to be minimized, but a fundamental quantity that must be modeled, quantified, and communicated explicitly. Probabilistic forecasting provides a well-studied framework for this purpose by representing predictions as distributions rather than a single value output.

Probabilistic forecasts allow for estimation of quantities relevant to decision making such as prediction intervals, quantiles, and exceedance probabilities. These quantity estimations become directly actionable in downstream tasks such as risk-aware control, constraint satisfaction, and safety margins. For example, rather than predicting a single value representing a failure or violation, a probabilistic model can estimate the probability that a failure will happen.

A central concept of probabilistic forecasting is calibration, and refers to the statistical consistency between predicted probabilities and observed empirical frequencies. A probabilistic model is well-calibrated if events predicted to occur with probability p indeed occur approximately p fraction of the time in practice. Calibration is essential in safety-critical applications, where overconfident predictions can lead to gross underestimation of risk and

potentially catastrophic decisions. Poor calibration undermines trust in probabilistic outputs, regardless of the apparent sharpness or sophistication of the predictive model.

However, good calibration alone is not sufficient for a strong probabilistic forecasting model. Trivially wide prediction intervals or overly diffuse distributions may be perfectly calibrated yet provide little useful information for decision-making. This motivates the complementary notion of sharpness, which measures the concentration or informativeness of probabilistic forecasts. Sharpness is a property of the predictive distribution itself, independent of the realized outcomes. Sharpness reflects the model's ability to make precise predictions. The fundamental tension between calibration and sharpness is well established in the forecasting literature: improving one often degrades the other. Evaluation of probabilistic models must therefore consider both aspects jointly rather than optimizing either in isolation.

In practice, many modern machine learning models produce uncertainty estimates with poor calibration, even when they achieve strong point prediction accuracy. This issue is exacerbated in deep learning models, which are often highly overconfident and sensitive to distributional shift. For safety-critical systems, this behavior is unacceptable, as it may systematically underestimate tail risk. Consequently, conservative uncertainty estimates are often preferred, even at the expense of reduced sharpness, provided that statistical guarantees can be established.

2.5.1 Foundations of Probabilistic Regression

Probabilistic regression aims to characterize the conditional distribution of a target variable given observed inputs, rather than producing a single point estimate. This perspective has a long history in statistics and has gained renewed importance in modern machine learning as uncertainty quantification has become central to decision-making in safety-critical and risk-sensitive applications. Existing approaches to probabilistic regression can be broadly categorized into quantile-based methods, Bayesian formulations, and likelihood-based probabilistic models, each of which encodes uncertainty in a fundamentally different manner.

Quantile regression, introduced by Koenker and Bassett [42], generalizes classical least-squares regression by estimating conditional quantiles of the response variable rather than its conditional mean. By minimizing an asymmetric absolute loss function, often referred to as the pinball or check loss, quantile regression provides a direct and distribution-free mechanism for estimating specific quantiles of the conditional outcome distribution.

This formulation has made quantile regression particularly attractive in applications where tail behavior and asymmetric risk are of interest. Unlike likelihood-based approaches, quantile regression does not require explicit assumptions about the parametric form of the underlying distribution, and quantiles can be estimated independently at arbitrary probability levels. However, classical quantile regression also exhibits important limitations. Independently estimated quantiles are not guaranteed to be mutually consistent, leading to the possibility of quantile crossing, and the method does not provide formal guarantees on coverage. As a result, estimated quantiles may systematically under- or over-cover the true conditional distribution, especially under model misspecification or distributional shift.

Bayesian neural networks represent a fundamentally different approach to probabilistic regression, in which uncertainty is modeled through probability distributions over model

parameters rather than through direct estimation of output quantiles. In this framework, predictive uncertainty arises from marginalization over a posterior distribution on network weights. Variational inference methods have been proposed to make Bayesian neural networks tractable for large-scale models, most notably the Bayes by Backprop approach of Blundell et al. [43], which approximates the posterior distribution over weights using parametric variational families.

Approximate Bayesian techniques such as Monte Carlo dropout further reduce computational complexity by showing that dropout training already corresponds to variational inference in a deep Gaussian process, so that leaving dropout active at test time yields Monte Carlo samples from an approximate predictive posterior [44]. These methods have been widely adopted due to their ease of implementation and compatibility with standard deep learning architectures. Bayesian neural networks and their approximations are often interpreted as capturing epistemic uncertainty arising from limited data or model uncertainty.

Despite their conceptual appeal, Bayesian neural networks do not inherently guarantee calibrated predictive distributions. The quality of uncertainty estimates depends strongly on the choice of prior, the expressiveness of the variational approximation, and the optimization procedure. In practice, approximate inference error and overparameterization can lead to miscalibrated predictions, particularly in regions of the input space that are sparsely represented in the training data.

In addition to quantile-based and Bayesian approaches, many modern regression models produce probabilistic outputs by directly parameterizing a predictive likelihood. Common formulations assume a parametric distribution, such as a Gaussian with learned mean and variance, and train the model using maximum likelihood objectives. More expressive variants, including mixture density networks, extend this approach by modeling multimodal conditional distributions.

Likelihood-based probabilistic regressors offer strong expressive power and are straightforward to integrate into deep learning pipelines. However, their probabilistic validity depends critically on the correctness of the assumed likelihood model. When model assumptions are violated, learned uncertainty estimates may be systematically biased, resulting in overconfident or underconfident predictions. Empirical evidence suggests that such models can achieve high point prediction accuracy while producing poorly calibrated uncertainty estimates, limiting their reliability in downstream decision-making.

The existing literature on probabilistic regression provides a diverse set of tools for uncertainty-aware prediction. Quantile regression offers a simple and distribution-free approach to estimating conditional quantiles but lacks formal coverage guarantees. Bayesian neural networks provide a principled framework for modeling epistemic uncertainty but are sensitive to approximation quality and modeling assumptions. Likelihood-based probabilistic regressors are flexible and scalable but often exhibit miscalibrated uncertainty in practice.

These limitations motivate subsequent work on uncertainty calibration and distribution-free guarantees. In particular, methods that explicitly evaluate and correct calibration, as well as techniques that provide finite-sample coverage guarantees independent of model correctness, have emerged as important complements to expressive probabilistic regression models.

2.5.2 Empirical Miscalibration in Deep Models

Despite their expressive power and strong performance in point prediction tasks, modern deep learning models frequently produce probabilistic predictions with poor calibration. This phenomenon has been systematically documented by Guo et al. [45], as they demonstrated that contemporary neural networks tend to produce overconfident results, even when trained with likelihood-based objectives and regularization techniques. In both classification and regression settings, predicted probabilities often fail to correspond to empirical frequencies. This damages trust in the model to produce reliable estimates in downstream decision-making.

A major contribution of this thread of work is the formalization and adoption of calibration diagnostics for neural networks. Metrics such as expected calibration error (ECE) and reliability diagrams provide both quantitative and visual tools to analyze whether predicted confidence levels align with observed outcomes. Empirical results indicate that increasing model capacity, depth, and training duration can contribute to miscalibration, despite improvements in predictive accuracy. These findings challenge the assumption that probabilistic outputs produced by deep models are inherently reliable.

Miscalibration issues are not confined to only deterministic neural networks. Bayesian neural networks and approximate Bayesian methods (such as variational inference and Monte Carlo dropout) have also been shown to produce miscalibrated predictive distributions. Approximation error in posterior inference, sensitivity to prior specification, and artifacts of optimization can all contribute to overconfident uncertainty estimates. As a result, Bayesian formulations alone do not guarantee calibrated predictions, particularly in high-dimensional or data-sparse schemes.

The presence of miscalibration in deep probabilistic models has significant implications for safety-critical applications. Overconfident predictions can lead to systematic underestimation of risk, while underconfident predictions may result in overly conservative behavior and reduced efficiency. These observations motivate the need for uncertainty quantification methods that go beyond expressive modeling and explicitly address calibration, evaluation, and robustness. Subsequent work has therefore focused on calibration-aware learning objectives, distribution-free guarantees, and evaluation frameworks that jointly consider calibration and sharpness, which are reviewed in the following sections.

2.5.3 Post-hoc Calibration for Regression

The empirical miscalibration observed in modern probabilistic models (especially deep models) has motivated the development of post-hoc calibration methods that operate independently of the underlying predictive model. Kuleshov et al. [46] formalized calibration for regression by defining a notion of distributional consistency based on predictive cumulative distribution functions. Under this framework, a probabilistic regressor is calibrated if the empirical frequency of outcomes falling below predicted quantiles matches the nominal quantile levels.

Expanding on this definition, Kuleshov et al. proposed a general recalibration procedure that maps predicted cumulative probabilities to calibrated probabilities using held-out validation data. This approach is model-agnostic and can be applied to a wide range of

probabilistic regressors, including Bayesian neural networks and likelihood-based deep models. Empirical results demonstrate substantial improvements in calibration with minimal degradation in predictive sharpness, making calibrated regression a practical and widely applicable correction mechanism.

However, post-hoc calibration methods rely on the assumption that calibration error is stable across training and deployment distributions. While effective in correcting average miscalibration on validation data, these techniques do not provide formal guarantees on predictive coverage and may fail under distributional shift or in sparsely sampled regions of the input space. As a result, post-hoc calibration improves empirical reliability but does not eliminate the need for uncertainty estimation methods with stronger statistical guarantees.

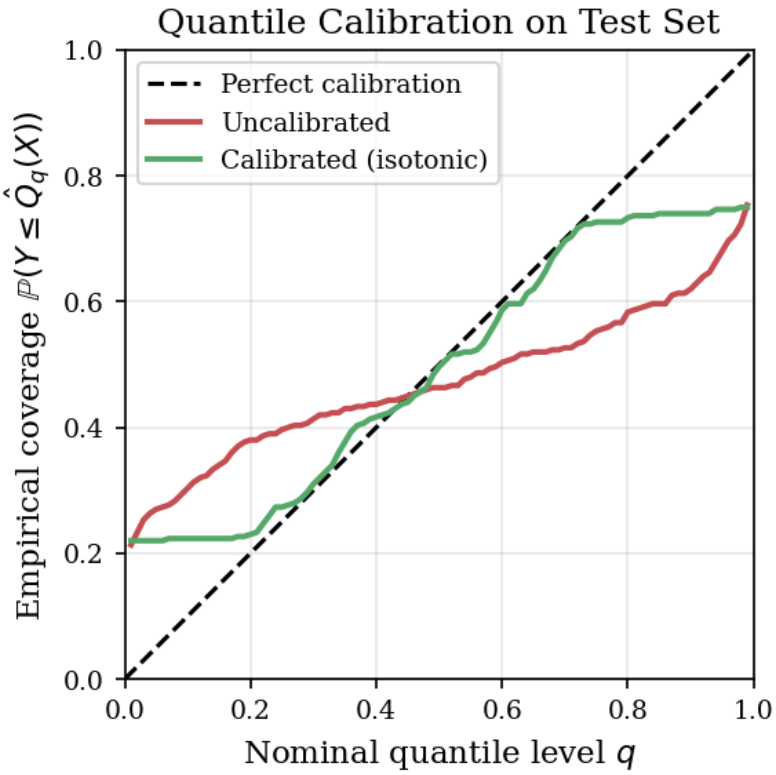


Figure 2.1: Quantile calibration: diagonal = perfect; uncalibrated model deviates; isotonic calibration corrects.

Figure 2.1 illustrates the distinction. A well-calibrated model produces quantiles such that when we predict the q-quantile $\hat{Q}_q(X)$, the fraction of held-out outcomes $Y \leq \hat{Q}_q(X)$ is approximately q. The diagonal represents this ideal. An uncalibrated model may deviate: for example, an overconfident model (underestimated variance) yields empirical coverage below the diagonal at high nominal levels. Post-hoc calibration methods such as isotonic regression can correct this by learning a monotone mapping from predicted to calibrated quantile levels on a validation set.

2.5.4 Distribution-Free Coverage via Conformal Prediction

Conformal prediction offers an alternative approach to uncertainty quantification by providing finite-sample, distribution-free coverage guarantees under minimal assumptions. Introduced by Shafer and Vovk [47], conformal prediction constructs prediction sets that contain the true outcome with a specified probability, provided the data are exchangeable. Crucially, these guarantees hold regardless of the correctness of the underlying predictive model.

Recent work has integrated conformal prediction with quantile regression to produce calibrated prediction intervals that adapt to heteroscedasticity in the data. Romano et al. [48] proposed conformalized quantile regression, which adjusts estimated conditional quantiles to ensure exact marginal coverage. This method preserves the flexibility of quantile regression while enforcing rigorous statistical guarantees, making it particularly attractive for safety-critical applications.

The primary limitation of conformal prediction methods is their inherent conservatism. Coverage guarantees are typically marginal rather than conditional, and the resulting prediction intervals may be wider than necessary for individual inputs. This trade-off between coverage guarantees and predictive sharpness reflects a fundamental tension in uncertainty quantification. Regardless, conformal prediction provides a robust and model-agnostic mechanism for uncertainty estimation, complementing calibration-based approaches and motivating hybrid methods that balance expressiveness, calibration, and statistical rigor.

2.5.5 Proper Scoring Rules and Evaluation Metrics

Evaluating probabilistic forecasts requires metrics that jointly assess both calibration and sharpness. Proper scoring rules provide a principled framework for this purpose by assigning a numerical score to a predicted distribution such that the expected score is minimized when the predictive distribution matches the true data-generating distribution. Gneiting and Raftery [49] formalized this framework and emphasized that calibration and sharpness must be considered together rather than as competing objectives.

Commonly used proper scoring rules include the logarithmic score, the continuous ranked probability score (CRPS), and interval-based scores. Unlike point-error metrics, proper scoring rules penalize both overconfident and overly diffuse predictions, making them well suited for comparing probabilistic models. As a result, they have become standard tools for evaluating probabilistic forecasts across a wide range of applications, including time series prediction and uncertainty-aware regression.

In the context of safety-critical forecasting, proper scoring rules provide an objective means of assessing whether probabilistic predictions are both reliable and informative. Their use complements calibration diagnostics and coverage-based guarantees, offering a unified evaluation framework for probabilistic regression models.

Synthesis and Implications

Taken together, the literature reviewed in this chapter establishes what a surrogate of longitudinal train dynamics must provide and where existing approaches fall short. Classical

longitudinal dynamics models supply well-understood physical structure, but their cost, concentrated in the stiff and nonlinear coupler interactions, keeps them out of optimization and control loops. Data-driven rail surrogates show that this cost can be reduced substantially, yet each is tied to something particular: a coupler position, a consist configuration, a track segment, or an operational context, and a change in that quantity requires retraining or configuration-specific adaptation data.

Sequence models and neural operators offer general machinery for learning dynamics, but both fix the size of their input in advance, so consist length, the quantity over which generalization is sought, becomes an architectural constant. Graph networks remove that constraint. Their structure matches the mechanics of a consist directly, with couplers as edges and the force balance as a node update, and the graph network simulator lineage shows that a learned fixed step can be stable over long rollouts and is not bound by the step-size limit of the scheme it replaces. Neural ODEs, the continuous-time alternative, face a documented difficulty on stiff systems, and a coupled vehicle chain is stiff by construction. What has not been done is to apply this machinery to the forward longitudinal dynamics of a coupled consist.

The probabilistic forecasting literature bears on a narrower question in this thesis than it would in a forecasting study. It establishes that expressive models do not produce reliable uncertainty estimates by default, and that calibration and coverage must be checked rather than assumed. Because the downstream use of the surrogate is control, uncertainty enters here as a coupler-force safety margin that the controller must respect, not as a calibrated distribution reported over every output, and it is evaluated in that role.

The gap this thesis addresses is therefore a surrogate that is train-agnostic by construction, imposes the physics it knows exactly rather than learning it, and is differentiable end to end so that it can serve as the model inside a fuel-optimal controller. Chapter 3 formulates that problem, and the surrogate itself is a graph network simulator on the consist's chain graph.

Chapter 3

Problem Formulation

3.1 The prediction and control problem, and the tensor format used to describe it

This chapter defines the problem this thesis solves and the mathematical objects used to describe it. The physical picture is longitudinal train dynamics (LTD): a chain of vehicles connected by couplers, moving along a route, pushed and slowed by traction, braking, resistance, grade, and the forces passed through the couplers themselves. None of this physics is new. What is new is how it is packaged. This thesis represents the coupled-vehicle system as a small set of tensors, structured arrays, so that the simulator's outputs, the model's training targets, and the surrogate model built in later chapters all share the same layout, no matter how many vehicles are in the train.

The rest of this chapter proceeds as follows. Section 3.1.1 states, in plain terms, what is being predicted and why it matters. Section 3.1.2 restates the underlying physics in its usual form. Sections 3.1.3 and 3.1.4 introduce the node and edge tensors and rewrite the dynamics in that format. Section 3.1.5 states the learning objective, including the requirement that the learned model be differentiable so that it can later be used to choose control actions, not just predict outcomes. The reference simulator that generates training data from this formulation is documented separately, in Chapter 4.

3.1.1 Physical setting and prediction task

Picture a freight train moving along a route. Before it runs, or while planning how it should run, a few questions matter: how much fuel will this trip use, will it arrive on time, and will the forces between cars ever get dangerously high. A more useful question is how the answers to those change if the driving plan is adjusted slightly.

Answering that last question well is the real goal of this thesis. A model that can only report what happened in one specific run is a forecasting tool. A model that can also say how the outcome would shift if traction or braking commands were changed, and can do so cheaply and repeatedly, becomes something a control algorithm can use to choose better driving plans on its own. That is the target: a model fast and smooth enough to sit inside an optimization loop that picks fuel-efficient throttle and brake commands, while staying

within speed limits and coupler-force safety limits.

To make this concrete, every scenario is described by three things: a route profile (the track's grade, curvature, and speed limits along its length), a consist configuration (the ordered list of vehicles, their masses, resistance properties, and coupler characteristics), and a control or operating plan (the traction and braking commands applied over time, possibly adjusted by a driver or automation model). Given these three inputs, the quantities of interest include:

- **Motion:** the position, velocity, and acceleration of every vehicle.
- **In-train forces:** the force at each coupler, which determines whether neighboring cars are pulling apart (draft) or pushing together (buff).
- **Energy and fuel:** the work or power used over the run.
- **Schedule:** trip time and arrival at scheduled points.
- **Safety margin:** how close any of the above comes to an operating limit.

This thesis treats the physics simulator described in Chapter 4 as ground truth when its parameters are fixed. Uncertainty in those parameters, or in the environment, is folded in as well (section 3.4), but its role here is narrower than a full probabilistic forecast: it mainly shows up as a safety margin that the control algorithm has to respect, rather than as a result reported for its own sake.

3.1.2 Classical longitudinal dynamics formulation

Classical longitudinal modeling represents the train as a chain of N vehicles indexed $i = 0, \ldots, N-1$ from lead to tail, connected by $N-1$ couplers. Let $x_i(t)$ and $v_i(t) = \dot{x}_i$ denote the longitudinal position and velocity of vehicle i along the route, and let m_i denote its mass. Between vehicles i and $i+1$, coupler i is associated with a kinematic extension δ_i (relative displacement from nominal spacing, positive in draft in the convention used below) and a coupler force F_i transmitted to the trailing vehicle in the pair. Resistance, grade, and optional curvature terms enter as external longitudinal forces on each vehicle; traction and braking appear as controlled inputs, possibly after actuator dynamics. Summarizing the external and control contributions on vehicle i by $F_{\text{ext},i}(t)$ and the coupler incidence by the force on the rear of i minus the force on the front, the translational dynamics take the familiar form

$$\dot{x}_i = v_i, \qquad m_i \dot{v}_i = F_{\text{ext},i}(t) + F_{i-1}(t) - F_i(t), \tag{3.1}$$

with $F_{-1} = F_{N-1} = 0$ when end forces are absent. The coupler forces F_i are determined by constitutive laws $F_i = \phi_i(\delta_i, \dot{\delta}_i, \ldots)$ linking kinematics to internal forces; slack, asymmetric buff and draft stiffness, and damping enter here (chapter 4). When actuator lag or power limits are modeled, eq. (3.1) is augmented by additional state variables per vehicle (for example brake and traction command states) driven by exogenous commands. This is the standard description of a coupled multi-body train, and it holds regardless of the tensor notation introduced next.

3.1.3 Tensorized state representation

For learning, and for software that must handle variable train length and rich per-vehicle features, it is convenient to organize the same physics into two matrices that make the vehicle-coupler graph explicit. In this thesis, a *node* (vehicle) tensor $\mathbf{H}(t) \in \mathbb{R}^{N \times d_n}$ collects d_n channels associated with each vehicle at time t. Typical channels include longitudinal position and velocity, internal actuator or command states when those are part of the dynamic model, and static attributes replicated on each row (mass, Davis coefficients, traction capability flags, force limits, and similar parameters) so that a single array suffices for batched computation and for models that read local vehicle features. An *edge* (coupler) tensor $\mathbf{E}(t) \in \mathbb{R}^{(N-1) \times d_e}$ holds d_e channels per coupler, such as extension δ_i, relative speed $\dot{\delta}_i$, algebraic coupler force F_i, nominal spacing, slack half-width, and stiffness and damping parameters for buff and draft segments. The assignment of channels is fixed in the reference implementation; the important point is the separation between vehicle-centered quantities in $\mathbf{H}$ and coupler-centered quantities in $\mathbf{E}$.

Along a discrete time grid $t_0, \ldots, t_{T-1}$, simulated trajectories can be stored as a *rollout tensor* $\mathbf{X} \in \mathbb{R}^{T \times N \times d}$, whose time slice $\mathbf{X}_{k,:,:}$ is the node feature matrix at t_k (so one may take $d = d_n$ and identify $\mathbf{X}_{k,:,:}$ with $\mathbf{H}(t_k)$ after any bookkeeping for channels). Coupler histories are stored analogously as $\mathbf{E}_{\text{hist}} \in \mathbb{R}^{T \times (N-1) \times d_e}$.

This layout keeps the structure of the physical problem visible: each coupler row depends only on the two vehicle rows next to it, mirroring the chain shape of eq. (3.1). That chain shape is also why this thesis treats the train as a graph, with vehicles as nodes and couplers as edges, rather than as a fixed-length sequence. A graph-based model can process trains of any length directly, passing information between neighboring vehicles one coupler at a time, without needing to pad shorter trains or mask out unused positions the way sequence models do. Handling trains of different lengths this way, as a direct consequence of the representation rather than as an engineering workaround, is central to the train-agnostic goal of this thesis: the same trained model should work on a train it has never seen, of a length it has never seen, without modification.

3.1.4 Dynamics in tensor-oriented form

The tensor view is a reorganization of state and parameters, not a change to the underlying force balances. Let $\mathbf{H}_{\text{dyn}}(t)$ denote the dynamic channels within $\mathbf{H}(t)$ (for example positions, velocities, and actuator states in the extended reference model), and let $\mathbf{p}$ collect fixed parameters, including static columns of $\mathbf{H}$ and of $\mathbf{E}$ where those encode coupler constants. The differential part of the evolution can be written compactly as

$$\frac{\mathrm{d}}{\mathrm{d}t} \operatorname{vec}(\mathbf{H}_{\text{dyn}}(t)) = \mathcal{F}_n\big(\mathbf{H}(t), \mathbf{E}(t), \mathcal{U}(t), \mathcal{R}; t\big), \tag{3.2}$$

where $\mathcal{U}$ denotes the control input signals, $\mathcal{R}$ the route profile, and $\mathcal{F}_n$ the assembled right-hand side obtained from eq. (3.1) and the actuator relations. Coupler kinematics link $\mathbf{H}_{\text{dyn}}$ to the extension and rate channels in $\mathbf{E}(t)$; constitutive laws determine the force channels. In the reference integrator used for data generation in this thesis, several edge channels (including δ_i, $\dot{\delta}_i$, and F_i) are *reconstructed algebraically* at each evaluation from vehicle positions and

velocities and from coupler parameters, rather than integrated as separate states. Thus the edge tensor is in part an explicit bookkeeping of quantities that satisfy relations of the form

$$\mathbf{E}(t) = \mathcal{G}_e\big(\mathbf{H}(t), \mathbf{p}\big), \tag{3.3}$$

for the algebraic channels, while eq. (3.2) carries the true differential degrees of freedom. If additional internal coupler or draft-gear states were introduced in future work, eq. (3.3) would be supplemented by differential equations for the corresponding rows of $\mathbf{E}(t)$, i.e. a nontrivial $\mathcal{F}_e$ on those components alongside $\mathcal{F}_n$.

The surrogate model built later in this thesis does not learn $\mathcal{F}_n$, and it does not learn a correction to it. It works one *step* at a time rather than one derivative at a time: given the state at t_k, it predicts the state at $t_{k+1} = t_k + h$ directly, for a fixed step h matched to the spacing at which trajectories are stored. Writing $\mathbf{s}_k$ for the dynamic state at time t_k, the model takes the form

$$\mathbf{s}_{k+1} = \mathcal{S}_\theta\big(\mathbf{s}_k, \mathbf{E}(t_k), \mathcal{U}, \mathcal{R};\, h\big), \tag{3.4}$$

in which eq. (3.2) appears only as the relation the training *data* satisfy, not as a term the model evaluates at run time.

Within that step the four dynamic channels are not treated alike, and the division of labour is the substance of the design. Position obeys $\dot{x}_i = v_i$ exactly, so it is recovered kinematically from the velocities at the two ends of the step rather than learned. The brake and traction lag states obey $\dot{z} = (u - z)/\tau$ with the command u known in advance, which is integrable in closed form across the step; giving them to a network would be asking it to learn a first-order lag it can simply be told. Only velocity is genuinely learned: the network predicts the *mean* acceleration of each vehicle over the step, together with a small correction on the per-coupler extension, which the kinematic reconstruction cannot recover from endpoint velocities alone.

Two consequences are worth stating here, because they shape everything that follows. The first is that two of the four dynamic channels leave the network entirely and a third is mostly imposed, so what is learned narrows to two scalars per vehicle rather than a correction on every channel: the physics in this model is structural, not additive. The second is that the step is the unit of computation. The reference integrator takes many internal steps between stored samples, and the surrogate replaces all of them with a single evaluation, which is where its speed advantage comes from and why that advantage can be quoted from the integrator's own configuration rather than from a hardware comparison. Sections 8.3 and 8.4 give the measurements that fix which channel is anchored on what; the point here is only that the formulation is a *learned step* on the tensor state, with physics imposed channel by channel.

3.1.5 Forecasting objective and simulator operator

The learning problem is to predict, under computation and uncertainty constraints, the outputs described in section 3.1.1 from the structured inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$. The reference physics implements a deterministic operator

$$F : (\mathcal{R}, \mathcal{C}, \mathcal{U}) \mapsto \mathbf{y}, \tag{3.5}$$

where $\mathbf{y}$ collects trajectories (for example stacks of $\mathbf{H}(t_k)$ and $\mathbf{E}(t_k)$), scalar summaries such as energy and trip time, and any derived risk indicators. The triple $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ denotes the same structured inputs as the route profile $\mathcal{R}$, consist $\mathcal{C}$, and control plan $\mathcal{U}$ detailed in section 3.2. When uncertain parameters and environment are modeled by a random vector $\xi \sim p(\xi)$, the same operator is written $F(\mathcal{R}, \mathcal{C}, \mathcal{U}; \xi)$ and outputs become random (section 3.4).

The goal of this thesis is not just to match individual trajectories point by point. The surrogate needs to be accurate enough that its outputs, especially coupler forces and trip energy, can be trusted for planning and for staying within safety margins. Where uncertainty matters, it mainly shows up as a margin that the surrogate and the downstream control algorithm must respect, rather than as a full calibrated probability distribution reported over every output.

The surrogate introduced later in this thesis approximates F by iterating the learned step of eq. (3.4), using a message-passing graph neural network that operates directly on the node and edge tensors defined above (section 3.1.4). Because the surrogate must eventually be used to choose control actions, not just predict outcomes for a fixed plan, it is also built to be differentiable: it can be evaluated many times in a batch, and the gradient of its outputs with respect to the control plan can be computed directly. The step formulation helps here rather than hindering. A fixed update around a single network evaluation leaves no adaptive solver in the gradient path, so the sensitivity of a trajectory to the control plan is obtained by differentiating a known, fixed composition of steps. That property is what later makes it possible to use the surrogate as the model inside a fuel-optimal control routine, instead of only as a fast forecasting tool.

3.2 Input definitions

3.2.1 Route profile

A route profile $\mathcal{R}$ provides the exogenous fields sampled by each vehicle as it moves along the track. In the reference simulator, $\mathcal{R}$ is a piecewise representation over distance s that supports interpolation of grade, curvature, and speed constraints:

- grade field $\theta(s)$ (or $\sin\theta(s)$ in implementation),
- curvature proxy $\kappa(s)$ when enabled,
- operational constraints such as speed limits $v_{\max}(s)$.

Given vehicle positions $x_i(t)$ from the node tensor, route-dependent forces are evaluated per vehicle at $s = x_i(t)$. This keeps the forcing consistent with the tensorized state layout while preserving the classical physics of section 3.1.2.

Route profiles used in this thesis are built from real elevation and track alignment data rather than invented from scratch, so that the grades and curves seen during training and evaluation resemble those of an actual rail corridor. Extra scenarios are produced by mildly perturbing these real profiles, for example in grade severity or adhesion, rather than by generating routes with no real-world basis. The pipeline that turns raw map and elevation data into these profiles is documented in chapter 5.

3.2.2 Consist configuration

A consist configuration $\mathcal{C}$ defines the ordered vehicle chain and coupler parameters. It is represented as two aligned tables that map directly to tensor channels:

- **Vehicle attributes** for $i = 0, \ldots, N-1$: mass, Davis coefficients, traction/brake limits, and actuator constants.
- **Coupler attributes** for $j = 0, \ldots, N-2$: nominal spacing, slack half-width, buff/draft stiffness, and damping parameters.

Static vehicle attributes are stored as fixed channels of the node tensor $\mathbf{H}$, and static coupler attributes as fixed channels of the edge tensor $\mathbf{E}$. This organization is exactly the data schema used by the simulator and by the surrogate model.

3.2.3 Control plan

A control plan $\mathcal{U}$ specifies exogenous command signals over time (or equivalent distance-indexed schedules mapped to time during simulation). For each powered or braked vehicle, commands are represented as channels $u_{\mathrm{trac},i}(t)$ and $u_{\mathrm{brk},i}(t)$ driving the actuator dynamics documented in chapter 4. The simulator therefore receives structured inputs as the triplet $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ introduced in eq. (3.5), and produces tensorized trajectories and derived summaries.

3.3 Output definitions

The output of one simulation run is a structured object

$$\mathbf{y} = \Big(\mathbf{X},\, \mathbf{E}_{\mathrm{hist}},\, \mathbf{s},\, \mathbf{r}\Big), \tag{3.6}$$

where $\mathbf{X}$ and $\mathbf{E}_{\mathrm{hist}}$ are rollout tensors, $\mathbf{s}$ contains scalar summaries, and $\mathbf{r}$ contains risk-oriented statistics.

3.3.1 Trajectory-level outputs

At discrete times t_k, the node rollout is $\mathbf{X}_{k,:,:} \in \mathbb{R}^{N \times d_n}$ and the edge rollout is $\mathbf{E}_{\mathrm{hist},k,:,:} \in \mathbb{R}^{(N-1) \times d_e}$. These tensors contain the channels defined in section 3.1.3: per-vehicle motion and actuator states in $\mathbf{X}$, and per-coupler extension, relative speed, and force channels in $\mathbf{E}_{\mathrm{hist}}$. Classical quantities are recovered directly from selected channels, e.g. $x_i(t_k)$, $v_i(t_k)$, and coupler force $F_j(t_k)$. For notation, the coupler force channel may be written as

$$F_j(t) = f_j\big(\mathcal{R}, \mathcal{C}, \mathcal{U}, t\big), \tag{3.7}$$

where f_j is induced by the simulator dynamics and constitutive laws.

3.3.2 Aggregate outputs

Aggregate outputs are deterministic functionals of rollout tensors for fixed parameters. Typical summaries used throughout this thesis include

$$E_{\text{trip}} = \int_0^T P_{\text{trac}}(t)\,\mathrm{d}t, \tag{3.8}$$

$$T_{\text{arr}} = \inf\{t :\ x_{\text{lead}}(t) \geq s_{\text{target}}\}, \tag{3.9}$$

$$F_{\max} = \max_{t,j} |F_j(t)|. \tag{3.10}$$

All such metrics are post-processing maps on $(\mathbf{X}, \mathbf{E}_{\text{hist}})$ and therefore remain consistent with the tensorized representation.

3.3.3 Risk metrics

For uncertainty-aware forecasting, risk is expressed as probabilities and quantiles of tensor-derived observables. For threshold τ and quantity Z (for example $F_{\max}$ or $|F_j(t)|$), the exceedance probability is

$$\mathbb{P}(Z > \tau). \tag{3.11}$$

Likewise, predictive quantiles satisfy $\mathbb{P}(Y \leq Q_p) = p$ for selected levels p. Both quantities are evaluated on variables computed from node/edge rollouts, so no separate non-tensor output definition is required. In this thesis these risk metrics are used mainly for coupler-force safety, as discussed next.

3.4 Uncertainty sources

Not every input to the simulator is known exactly. Vehicle masses, resistance coefficients, coupler stiffness, the exact shape of the route, and even how precisely a command is executed all carry some uncertainty. This section lists where that uncertainty comes from and how this thesis handles it. Because the main use of the surrogate in this thesis is fuel-optimal control, not general-purpose forecasting, uncertainty is treated mainly as something that narrows the safe operating margin the control algorithm must respect, rather than as a full predictive distribution reported for its own sake.

Uncertainty is introduced through a random vector ξ that perturbs physically meaningful components of the structured input and model:

1. **Vehicle and coupler parameters:** mass, resistance coefficients, slack and stiffness values, brake/traction effectiveness.
2. **Route and environment:** grade/curvature profile errors, adhesion-related effects, and external disturbances.
3. **Operational realization:** command timing and execution variability.
4. **Model discrepancy:** residual mismatch between the reference simulator and real operations.

3.4.1 Formal treatment of uncertainty

Let $\xi \sim p(\xi)$ denote the joint uncertainty law. Then the simulator operator from eq. (3.5) becomes

$$\mathbf{y}(\xi) = F(\mathcal{R}, \mathcal{C}, \mathcal{U}; \xi), \tag{3.12}$$

and every tensor channel and derived metric becomes a random variable. For example, the exceedance probability for coupler interface j is

$$\mathbb{P}_\xi\big(F_j(t;\xi) > \tau\big) = \int_{\{\xi:\, F_j(t;\xi)>\tau\}} p(\xi)\, \mathrm{d}\xi. \tag{3.13}$$

In practice, these probabilities are approximated by Monte Carlo simulation or by probabilistic surrogates trained on ensembles generated from the same tensorized simulator. The role this plays in the thesis is narrow: $\mathbb{P}_\xi(F_j > \tau)$ is used to define a safety margin that the fuel-optimal control routine is required to respect, for example by keeping the estimated coupler force below its limit with high probability, rather than as a general-purpose calibrated forecast reported across every output variable.

3.5 Supervised Learning Objective

Training data are generated by the reference simulator documented in chapter 4:

$$\mathcal{D} = \left\{ \left(\mathcal{R}^{(k)}, \mathcal{C}^{(k)}, \mathcal{U}^{(k)}, \mathbf{X}^{(k)}, \mathbf{E}^{(k)}_{\text{hist}}, \mathbf{s}^{(k)}\right) \right\}_{k=1}^{M}. \tag{3.14}$$

The surrogate f_θ maps structured inputs to trajectory and summary predictions in the same tensor layout, by iterating the learned step of eq. (3.4). A generic training objective is

$$\mathcal{L}(\theta) = \lambda_X \mathcal{L}_X + \lambda_E \mathcal{L}_E + \lambda_S \mathcal{L}_S, \tag{3.15}$$

with losses on the node channels, on the edge channels, and on the trip-level summaries. Supervision is on the learned quantities only: the imposed channels of section 3.1.4 are reconstructed exactly and contribute no error to fit.

3.5.1 Loss term definitions

Rollout losses. $\mathcal{L}_X$ and $\mathcal{L}_E$ compare predicted and reference tensor channels over time, vehicle/coupler index, and feature channels (with masking for variable train length where needed).

Summary loss. $\mathcal{L}_S$ penalizes errors on scalar outputs such as trip energy, arrival time, or peak coupler force.

On physical consistency. No separate physics-consistency penalty appears in eq. (3.15), and none is needed. Under the channel-wise construction of section 3.1.4 the kinematic relation between position and velocity, and the actuator lag dynamics, hold by construction at every step rather than approximately and on average. A penalty term is the right tool when a model is free to violate a relation and must be discouraged from doing so; here the model cannot express the violation in the first place.

On the safety margin. The coupler-force margin that the control routine relies on (section 3.3.3) is treated as an evaluation target rather than as a separate distributional head. This follows the narrower role assigned to uncertainty in section 3.4: what the controller needs is a trustworthy estimate of the peak force a plan will produce, and that is a question about the accuracy of the predicted force channel, not about calibrating a distribution over it.

Remark. Section 8.6 instantiates these terms for the graph neural network used in this thesis, and chapter 9 adds the perturbation scheme that makes a single-step model stable when iterated over long horizons.

3.6 Conceptual pipeline

The full workflow, from generating data to using the trained model for control, follows the same tensor objects at every step:

1. **Structured inputs**: Specify $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ and, for uncertainty studies, sample ξ.
2. **Reference simulation**: Run the tensorized LTD simulator (chapter 4) to obtain $(\mathbf{X}, \mathbf{E}_{\text{hist}}, \mathbf{s})$.
3. **Dataset assembly**: Store many such runs as supervised pairs for training and validation.
4. **Surrogate training**: Fit f_θ to predict rollout tensors and derived summaries, including the coupler-force safety margin.
5. **Forecasting**: Use f_θ for fast scenario evaluation in the same output space as the simulator.
6. **Control**: Use f_θ and its gradients inside an optimization routine, such as model-predictive control, to choose traction and braking commands that reduce fuel use while respecting speed limits and coupler-force safety margins.

Figure 3.1: Conceptual pipeline from structured inputs to tensorized simulator outputs, surrogate forecasting, and fuel-optimal control.

3.7 Evaluation targets

Evaluation is defined on the same outputs as the training objective:

Trajectory fidelity. How closely predicted vehicle motion and coupler forces track the reference simulator, channel by channel (for example MAE/RMSE on velocity and coupler-force channels).

Summary accuracy. Errors on trip-level numbers such as total energy used, arrival time, and peak coupler force.

Safety-margin quality. How well the surrogate's coupler-force margin estimates hold up, since this is what the control routine relies on to stay within limits.

Generalization across trains and routes. Accuracy on consist lengths and route corridors not seen during training. Since the surrogate is meant to work on trains it was never explicitly trained on, this is not a side check; it is one of the central claims of the thesis.

Operational utility. Inference-time speedup relative to the reference simulator, and performance as the model inside the fuel-optimal control routine, compared against a simple constant-speed driving plan.

3.8 Chapter summary

This chapter defined the prediction and control problem this thesis addresses. Classical longitudinal dynamics provides the physical foundation (eq. (3.1)). This thesis recasts that same system into node and edge tensors $\mathbf{H}(t)$ and $\mathbf{E}(t)$ (sections 3.1.3 and 3.1.4), a graph-shaped representation that lets a single model handle trains of any length. The simulator F maps structured inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ to structured outputs $\mathbf{y}$: trajectory tensors, trip-level summaries such as energy and arrival time, and safety-relevant quantities such as peak coupler force (eq. (3.5)). We distinguished deterministic outputs under fixed parameters from outputs under random parameter uncertainty ξ (eq. (3.12)).

The learning task is hard for four reasons: the inputs are high-dimensional and structured, the outputs span different scales and types, physical consistency has to be preserved, and safety-relevant quantities need a notion of margin rather than a single number. This thesis addresses the last of these narrowly, folding uncertainty into a coupler-force safety margin rather than building a full calibrated forecast of every output (section 3.4).

The next chapter documents the reference simulator that generates the data used throughout the rest of this thesis. Chapter 8 introduces the surrogate model itself: a graph neural network that advances the tensor state one fixed step at a time, with position and actuator dynamics imposed rather than learned, and that is differentiable end to end, so that it can later serve as the model inside a fuel-optimal control routine.

Chapter 4

Reference Simulator

Chapter 3 laid out the prediction and control problem and the tensor format used to represent a train. This chapter builds the tool that actually produces the numbers: a reference simulator of longitudinal train dynamics. Its job is simple to state even though the physics inside it is not. Given a train made of coupled vehicles, a route, and a set of driving commands, the simulator plays out what happens over time and records it. Every training example used later to teach the surrogate model comes from running this simulator once, so in that sense this chapter describes the data generator for the rest of the thesis.

The simulator was built up gradually, one physical effect at a time, so that each addition could be checked before the next was layered on. It follows a Jupyter notebook that constructs the model stage by stage using only NumPy, SciPy's `solve_ivp`, and Matplotlib, and the same model is mirrored in a structured Python package that exposes the node and edge tensors from chapter 3 alongside the classical flat state used for integration.

The parameters used, vehicle mass, resistance coefficients, coupler stiffness, traction limits, are realistic in shape but are not tuned to match any specific real locomotive or railcar. The goal at this stage is a simulator whose structure is right, not one whose numbers are certified against a real test train. This distinction matters when interpreting later results: the surrogate and the control experiments built on top of this simulator are validated against the simulator itself, not against real-world rail data. Chapter 10 returns to this distinction and asks how far the simulator described here can be trusted. Time integration uses adaptive Runge-Kutta or implicit methods as needed when stiff coupler dynamics appear, and the figures in this chapter are exported directly from the notebook at representative numerical settings.

4.1 Stage 1: single powered vehicle

The starting point is a single point mass m with longitudinal position $x(t)$ and velocity $v(t) = \dot{x}$. The equation of motion aggregates traction, braking, Davis running resistance, grade, and an optional curve resistance into a single scalar force balance:

$$m\dot{v} = F_{\text{trac}}(t) - F_{\text{brk}}(t) - R(v) - G(x) - C_{\kappa}(x, v), \tag{4.1}$$

with $\dot{x} = v$. Davis resistance is implemented in the usual magnitude-direction form,

$$R(v) = \big(A + B|v| + Cv^2\big)\,\mathrm{sgn}(v), \tag{4.2}$$

with $\mathrm{sgn}(v)$ set to zero at $|v| < \varepsilon$ to avoid chatter at standstill. Note that the coefficient C in eq. (4.2) is the Davis quadratic term and is unrelated to the curve resistance C_κ. Grade enters through $G(x) = mg\sin\theta(x)$ with θ the local track inclination.

Curve resistance is written, as is standard in railway practice, as a *specific* resistance w_c: a fraction of vehicle weight fixed by the curve radius $R = 1/|\kappa|$ and independent of speed. The implementation uses the linear (AREMA) form,

$$C_\kappa(x, v) = c_\kappa\, mg\, w_c\big(\kappa(x)\big)\,\mathrm{sgn}(v), \qquad w_c(\kappa) = \lambda\,|\kappa|, \tag{4.3}$$

with $\lambda \approx 0.699$, the AREMA figure of 0.8 lb per ton per degree of curve expressed in terms of κ, and c_κ a dimensionless multiplier for which $c_\kappa = 1$ recovers the standard value. The term is disabled by setting $c_\kappa = 0$, which is the case in the earliest runs and in any scenario that does not require it. On the corridors used here the force is small but not negligible: at the 95th-percentile curvature (radius $\approx$ 840 m) it is roughly 800 N on a loaded 100 t car, comparable to that car's Davis resistance at 15 m/s and about 8% of the force of a 1% grade.

Two aspects of eq. (4.3) are stated explicitly because both were settled by measurement rather than by convention. The first is the absence of a speed dependence. Earlier revisions used a proxy proportional to $mv^2|\kappa(x)|$; that form is retained in the implementation so results recorded before this change can be reproduced, but it is not the model of record. Curve resistance does not scale with speed, so a coefficient matched to eq. (4.3) at one speed is wrong at every other — one calibrated at 15 m/s is nine times too small at 5 m/s and 2.8 times too large at 25 m/s. The discrepancy is in the exponent rather than the constant, so no choice of scale repairs it. The second is the choice of w_c. The Röckl formula, $w_c = 650/(R - 55)$ N/kN for $R \geq 300$ m and $500/(R - 30)$ below, is the more familiar empirical statement and agrees with eq. (4.3) to within about 15% over the 300–2000 m range these routes occupy. It is available in the implementation but is not the default, because it is discontinuous at $R = 300$ m, where its two branches differ by roughly 30%. This force balance is the generator of the training data, so a jump in it becomes a jump in the trajectories the surrogate learns from, one the learned fixed step would have to reproduce whenever a vehicle crosses $R = 300\,\mathrm{m}$ within a step. The controller, in turn, takes its gradients from the surrogate rather than from this force balance, so it is the surrogate's behaviour near such a jump that the controller would see. The reasoning is the same as that behind the tanh blend in eq. (4.12). The distinction being drawn is between a jump and a kink: piecewise-linear interpolation of the route field already makes $\partial C_\kappa/\partial x$ discontinuous at every route node, and the coupler deadband of Stage 3 does the same at slack transitions, but in both cases the force itself remains continuous. Röckl would instead make the force discontinuous, which is a stronger defect and, unlike the other two, one the model does not need.

Traction and brake commands may be specified as arbitrary functions of time; the first stage uses a smooth "ramp" in time so that F_{trac} rises continuously from zero after a delay, rather than stepping instantaneously, which eases numerical stiffness and mirrors the later actuator-lag extension. Figure 4.1 shows position, speed, and the plotted residual $F_{\mathrm{trac}} - R(v)$

on level track with braking and curvature removed, confirming that the integrator and force signs behave as intended before couplers are introduced.

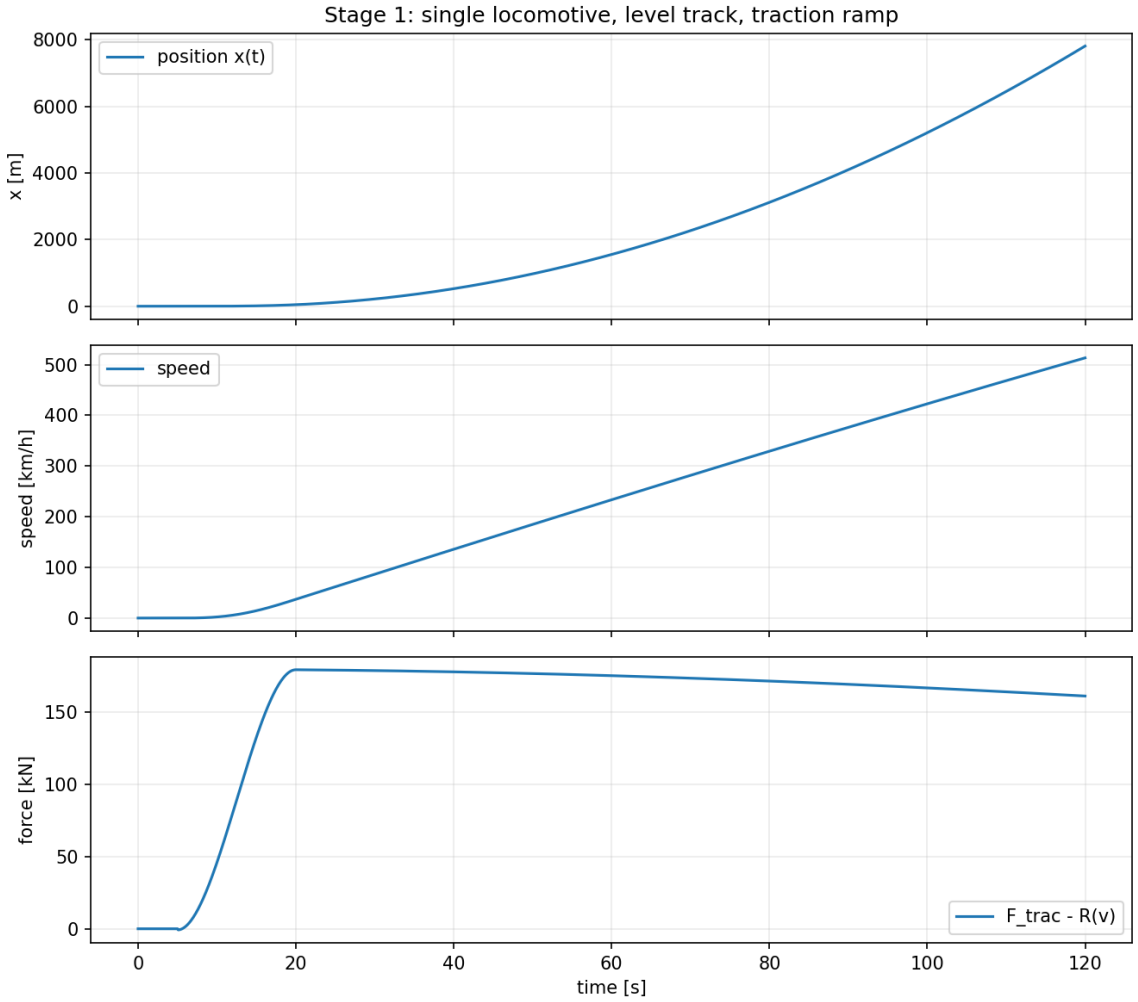


Figure 4.1: Stage 1 reference run: single vehicle on level track with a smooth traction ramp. Top to bottom: position, speed, and net longitudinal force $F_{\text{trac}} - R(v)$ versus time.

4.2 Stage 2: locomotive and one car with a linear coupler

A second mass m_1 is attached behind the lead vehicle m_0. Nominal centre-to-centre spacing is L_0. Coupler extension (positive in draft) is

$$\delta = (x_0 - x_1) - L_0, \qquad \dot{\delta} = v_0 - v_1. \tag{4.4}$$

A linear spring-damper coupler exerts force $F = k\delta + c\dot{\delta}$ on the rear car along the direction of travel; the lead feels $-F$. With G_i and R_i evaluated per vehicle,

$$m_0\dot{v}_0 = F_{\text{trac}} - F_{\text{brk},0} - R_0(v_0) - G_0(x_0) - F, \tag{4.5}$$
$$m_1\dot{v}_1 = -F_{\text{brk},1} - R_1(v_1) - G_1(x_1) + F. \tag{4.6}$$

The state vector is (x_0, x_1, v_0, v_1). For stiff (k, c) the two speeds track almost identically on a speed-time plot even though the coupler force and extension carry the load path; fig. 4.2 shows the characteristic transient in F and δ as slack is taken up and the train settles into quasi-uniform acceleration under traction.

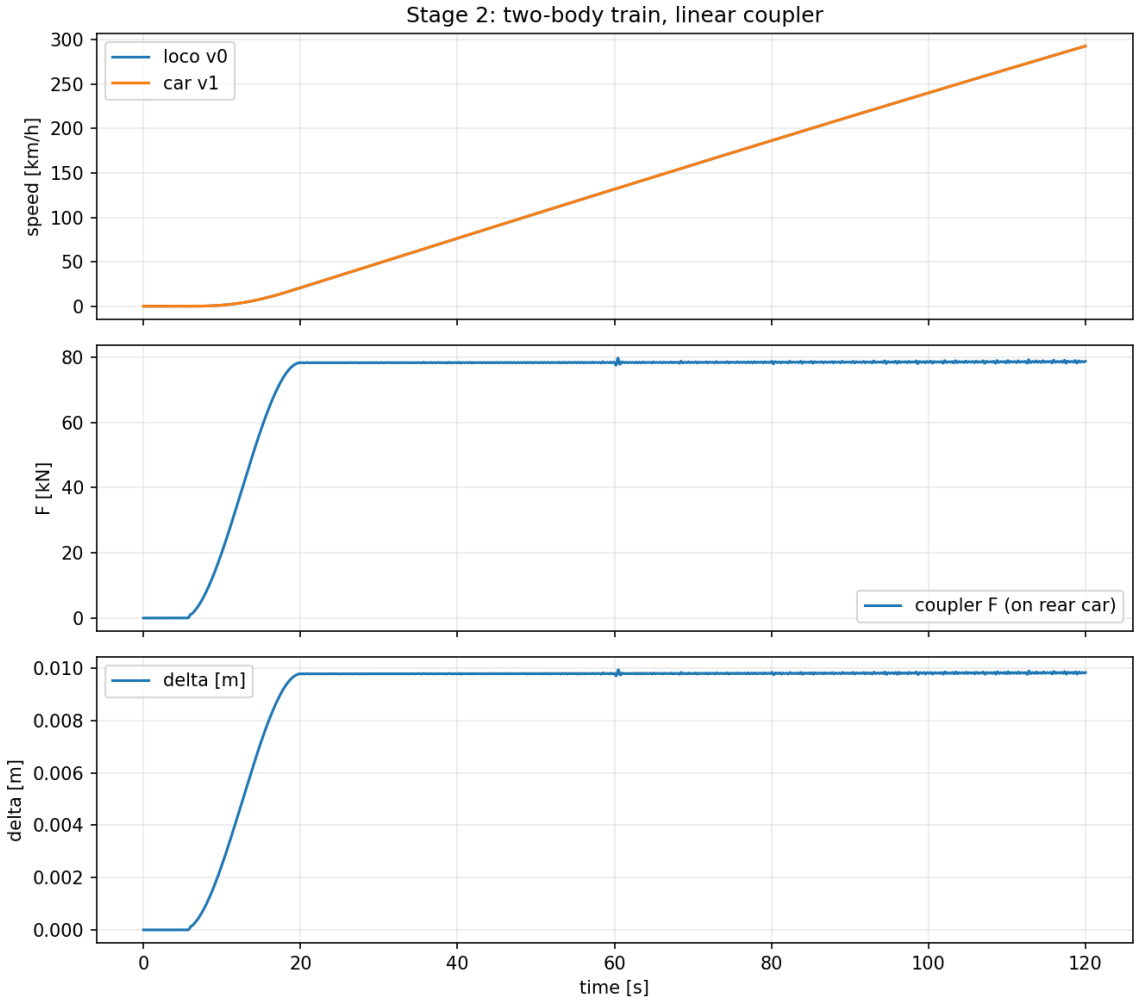


Figure 4.2: Stage 2: two-body train with linear coupler. Speeds of lead and trailing vehicle (overlapping at this scale), coupler force on the rear car, and extension δ.

4.3 Stage 3: nonlinear coupler with slack and asymmetric buff/draft

The linear model is retained for comparison, but the simulated dynamics replace F by a piecewise law with slack half-width s. For $|\delta| \leq s$ the coupler force is zero; for $\delta > s$ (draft) stiffness and damping $(k_\mathrm{d}, c_\mathrm{d})$ apply to the over-travel $\delta - s$, and for $\delta < -s$ (buff) a possibly different pair $(k_\mathrm{b}, c_\mathrm{b})$ applies to $\delta + s$. The same two-body equations eq. (4.6) hold with this $F(\delta, \dot{\delta})$. Figure 4.3 overlays the *linear* force $k\delta + c\dot{\delta}$ that would correspond to Stage 2 parameters against the *actual* nonlinear force along the trajectory, together with δ and $v_0 - v_1$. The deadband produces intervals of near-zero force and sharper loading when contact engages, which motivates slack-aware modelling for impact-style longitudinal loads.

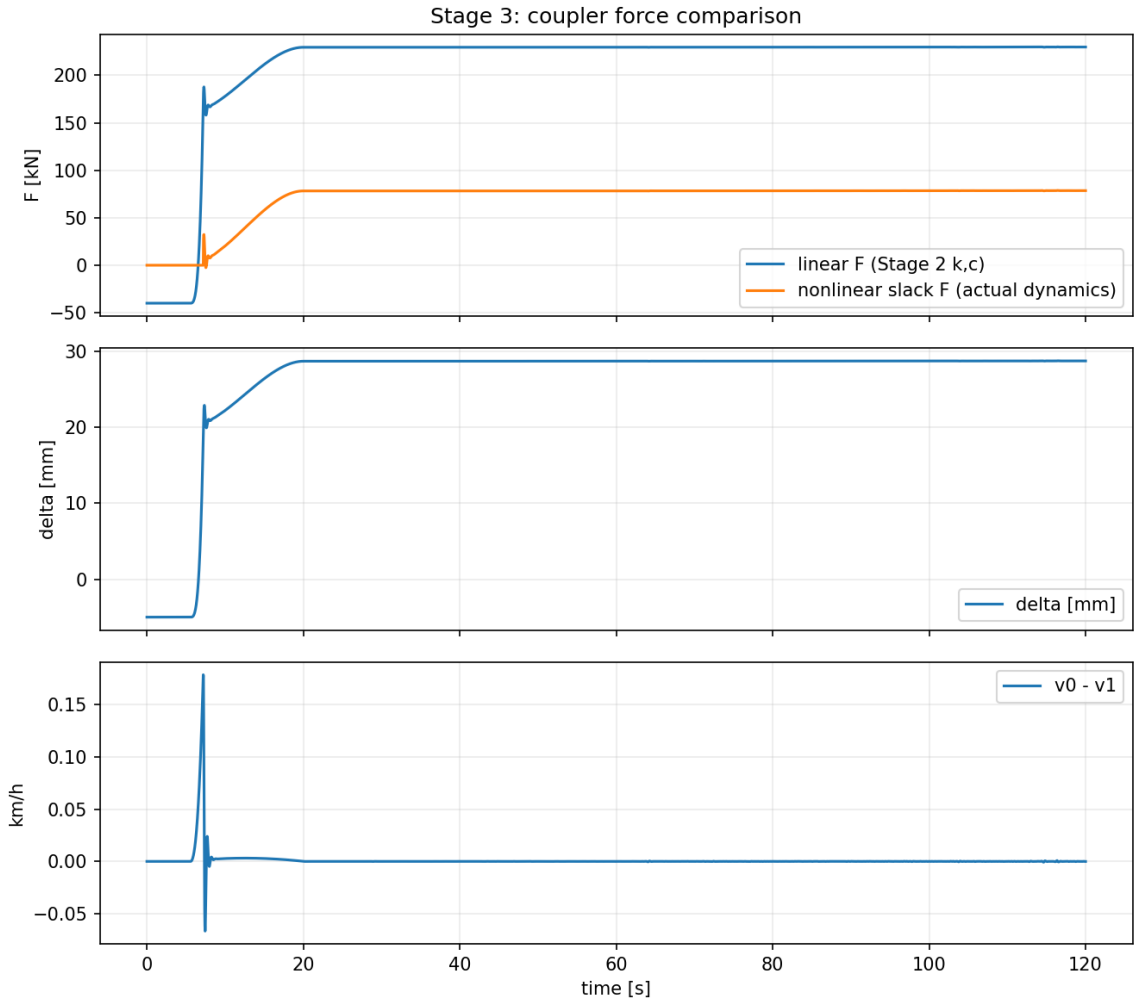


Figure 4.3: Stage 3: comparison of linear spring-damper force (Stage 2 parameters) versus nonlinear slack coupler force along the same trajectory, with extension and relative speed.

4.4 Stage 4: three bodies and sequential coupler forces

With vehicles indexed $0, 1, 2$ front to rear, coupler $j \in \{0, 1\}$ connects j to $j + 1$ and F_j denotes the force on the *rear* vehicle in that pair. The recursions generalise naturally:

$$m_0 \dot{v}_0 = F_{\text{trac}} - R_0 - G_0 - F_0, \tag{4.7}$$
$$m_1 \dot{v}_1 = -R_1 - G_1 + F_0 - F_1, \tag{4.8}$$
$$m_2 \dot{v}_2 = -R_2 - G_2 + F_1, \tag{4.9}$$

with each F_j obtained from the nonlinear coupler law applied to $(\delta_j, \dot{\delta}_j)$ where $\delta_j = (x_j - x_{j+1}) - L_{0,j}$. End-to-end stretch relative to nominal spacing, $(x_0 - x_2) - (L_{0,0} + L_{0,1})$, summarises how the consist elongates in draft. Figure 4.4 shows speeds, both coupler forces, and stretch for a traction-only run. An optional variant applies a time-localised service-brake force on the lead only while traction continues, illustrating redistribution of draft forces (fig. 4.5).

4.5 Stage 5: general N-vehicle assembly

The state is stacked as $\mathbf{y} = (x_0, \ldots, x_{N-1}, v_0, \ldots, v_{N-1})^\top \in \mathbb{R}^{2N}$. For each coupler $j = 0, \ldots, N - 2$, δ_j and $\dot{\delta}_j$ are defined as in eq. (4.4) with vehicle j ahead of $j + 1$. The force F_j on the rear vehicle follows the slack model with per-coupler parameters. The right-hand side for vehicle i is

$$m_i \dot{v}_i = F_{\text{trac},i} - F_{\text{brk},i} - R_i(v_i) - G_i(x_i) - C_i + F_{i-1}^{\text{in}} - F_i^{\text{out}}, \tag{4.10}$$

with the convention $F_{-1}^{\text{in}} = F_{N-1}^{\text{out}} = 0$, and with traction and braking possibly zero on unpowered cars. Data structures (vehicle and coupler parameter records, and a flat route profile) group masses, Davis triples, coupler geometry, and slack/stiffness data for reuse. This is also the stage where the simulator stops being tied to a fixed train length: the same code path handles any N, which is what lets it later generate training data across a wide range of consist sizes rather than one fixed configuration. Figure 4.6 plots selected speeds and coupler forces for a consist with one powered lead and identical trailing cars; fig. 4.7 shows a space-time style heatmap of all coupler forces, illustrating the monotonic decay of steady draft force from head to tail when only the lead provides traction.

4.6 Stage 6: per-vehicle grade and curvature from a route profile

Grade and curvature are no longer evaluated at a single global x; each vehicle samples $\sin\theta(x_i)$ and optionally $\kappa(x_i)$ at its own position. The profile is stored as piecewise-linear nodes and interpolated (e.g. linearly in arc length). Then $G_i = m_i g \sin\theta(x_i)$, and eq. (4.3) applies per vehicle with m and κ evaluated at that vehicle: $C_{\kappa,i} = c_\kappa \, m_i g \, \lambda \, |\kappa(x_i)| \, \text{sgn}(v_i)$. This introduces spatial dispersion along the train: the head and tail can experience different grades simultaneously, which excites relative motion and coupler cycling even when speed

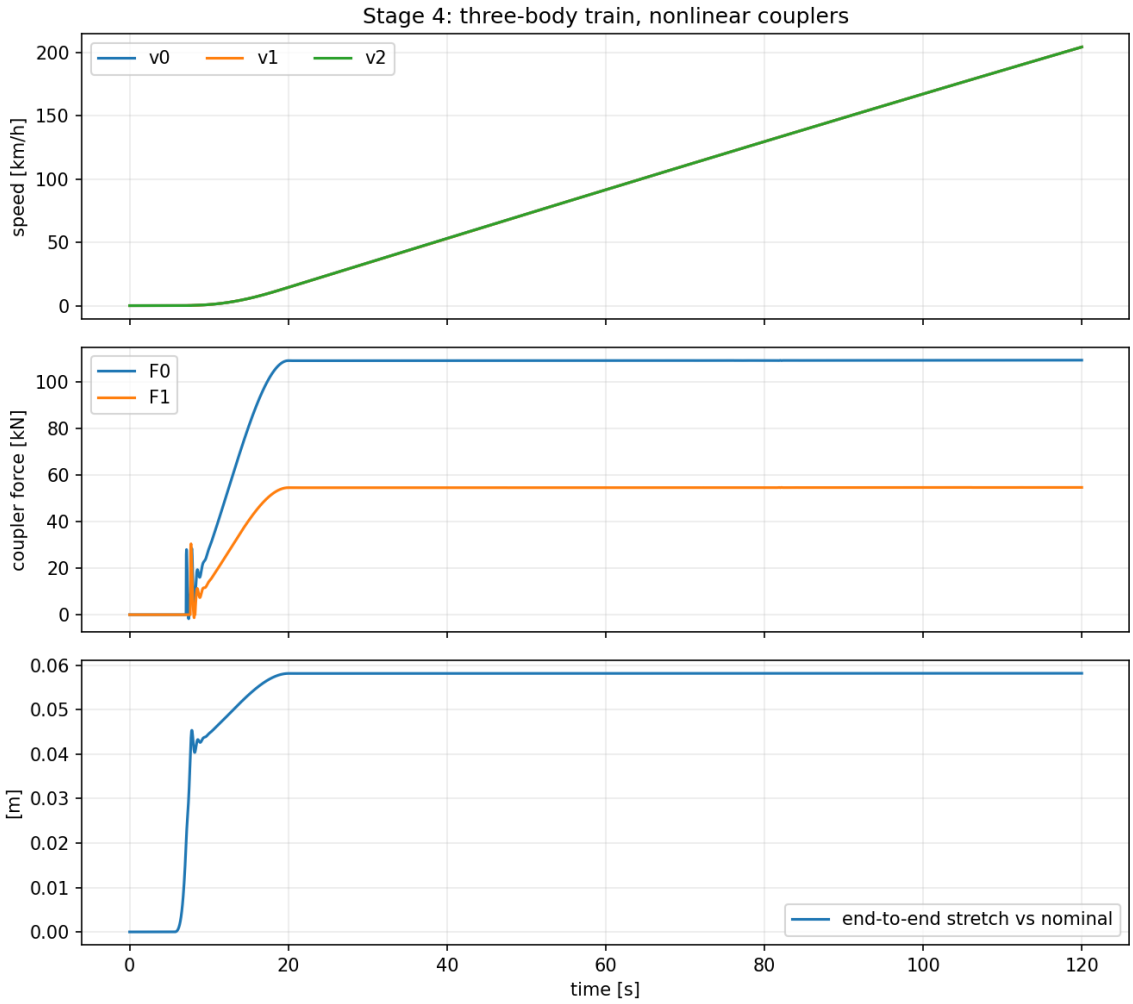


Figure 4.4: Stage 4: three-body train with nonlinear couplers. Speeds, coupler forces F_0 and F_1, and end-to-end stretch versus time.

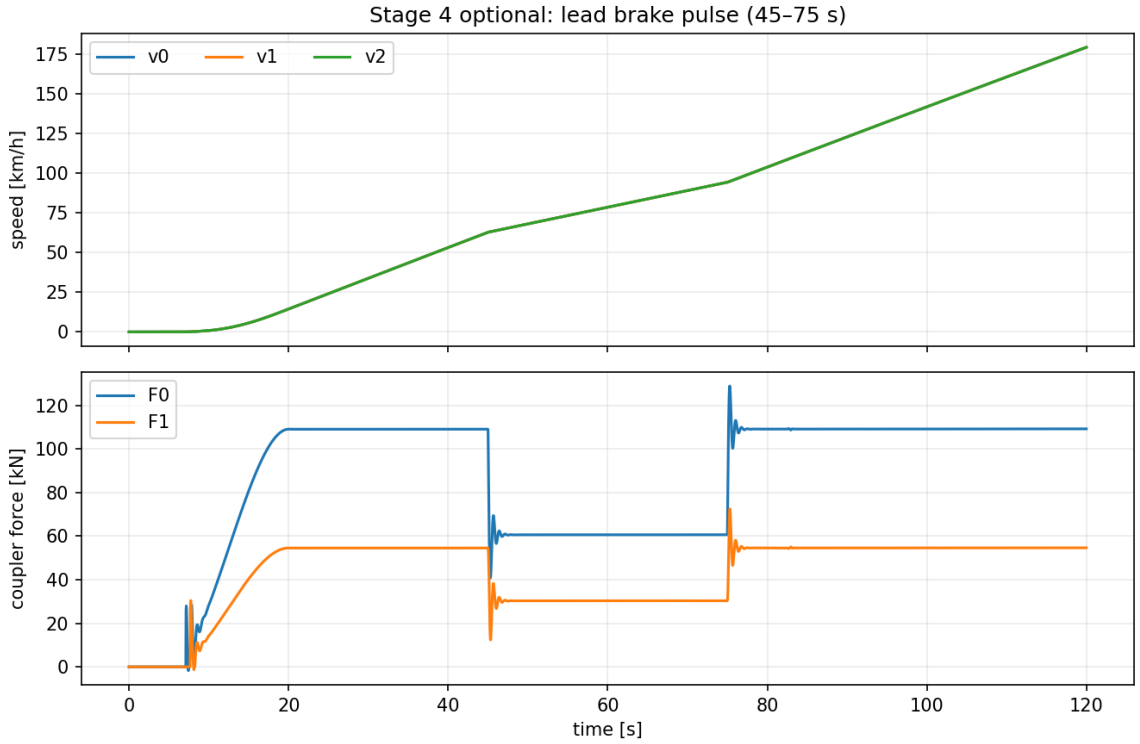


Figure 4.5: Stage 4 optional scenario: lead-only brake pulse; speeds and coupler forces.

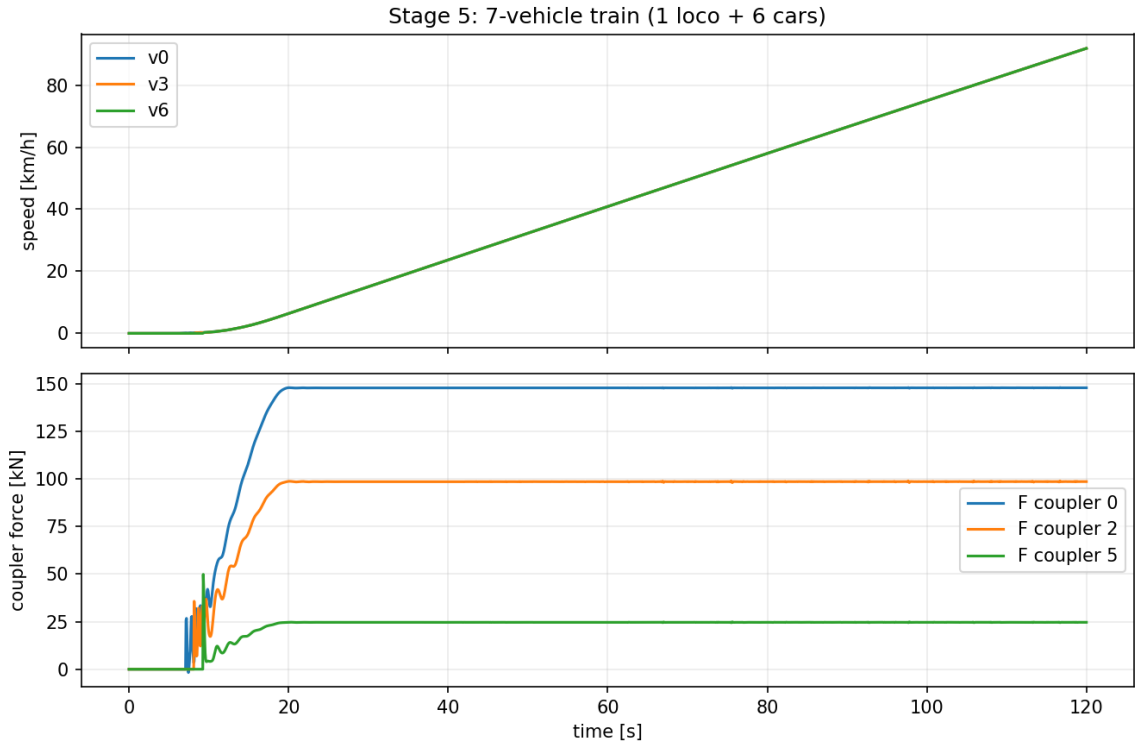


Figure 4.6: Stage 5: N-vehicle train; selected speeds and coupler forces versus time.

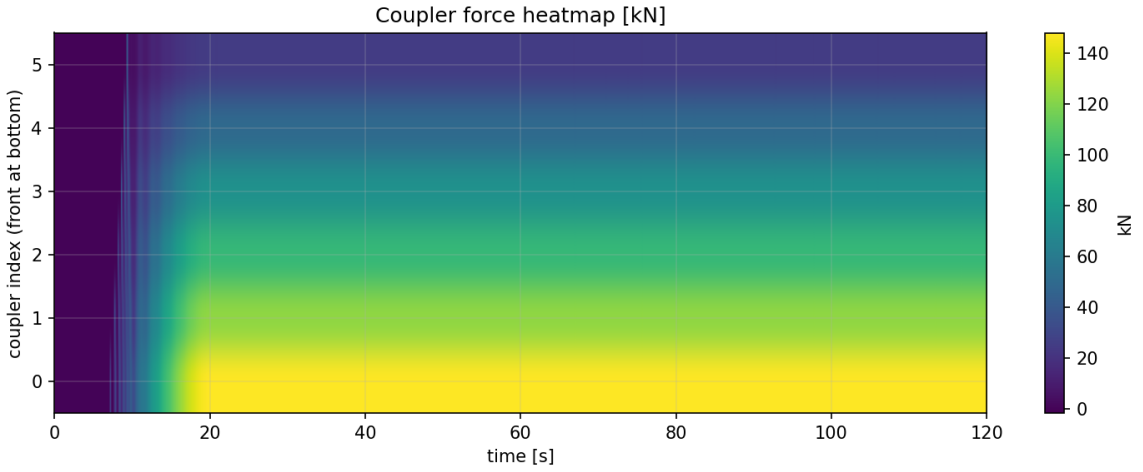


Figure 4.7: Stage 5: coupler force heatmap (force versus time and coupler index).

traces remain nearly overlaid. Figure 4.8 shows the prescribed $\sin\theta$ along route distance in the top panel and the time evolution of lead and rear speed and selected coupler forces below; the horizontal axes of the lower panels are simulation time, not chainage, so the spatial profile should be read as a fixed map through which the train moves.

4.7 Stage 7: actuator dynamics and power-limited traction

The state is augmented by first-order lag variables for commanded brake level $z_{\mathrm{brk},i}$ and commanded traction level $z_{\mathrm{trac},i}$ on each vehicle, driven by exogenous commands $u_{\mathrm{brk},i}(t)$ and $u_{\mathrm{trac},i}(t)$:

$$\dot{z}_{\mathrm{brk},i} = \frac{u_{\mathrm{brk},i} - z_{\mathrm{brk},i}}{\tau_b}, \qquad \dot{z}_{\mathrm{trac},i} = \frac{u_{\mathrm{trac},i} - z_{\mathrm{trac},i}}{\tau_t}. \tag{4.11}$$

Achieved brake force is capped by a vehicle limit, $F_{\mathrm{brk},i} = \min(\max(z_{\mathrm{brk},i}, 0), F_{\mathrm{brk,max},i})$, and is applied *against the direction of travel* rather than in a fixed direction:

$$F_{\mathrm{brk},i}^{\mathrm{applied}} = F_{\mathrm{brk},i}\,\tanh\!\left(\frac{v_i}{\varepsilon_b}\right), \tag{4.12}$$

with ε_b a small blend width. The tanh is preferred to $\mathrm{sgn}(v_i)$ because it keeps the right-hand side smooth through $v_i = 0$, which both eases the integrator and keeps a jump in force at standstill out of the training data, where the learned step would otherwise have to reproduce it; at operating speed the factor is unity to machine precision. Without this term a vehicle braked to a standstill is accelerated in reverse without bound, since the brake would keep contributing a negative force at $v_i = 0$. For powered vehicles, achieved traction is the continuous power cap

$$F_{\mathrm{trac},i} = \min\!\left(\max(z_{\mathrm{trac},i}, 0),\ \frac{P_{\max}}{\max(|v_i|, \varepsilon)}\right), \tag{4.13}$$

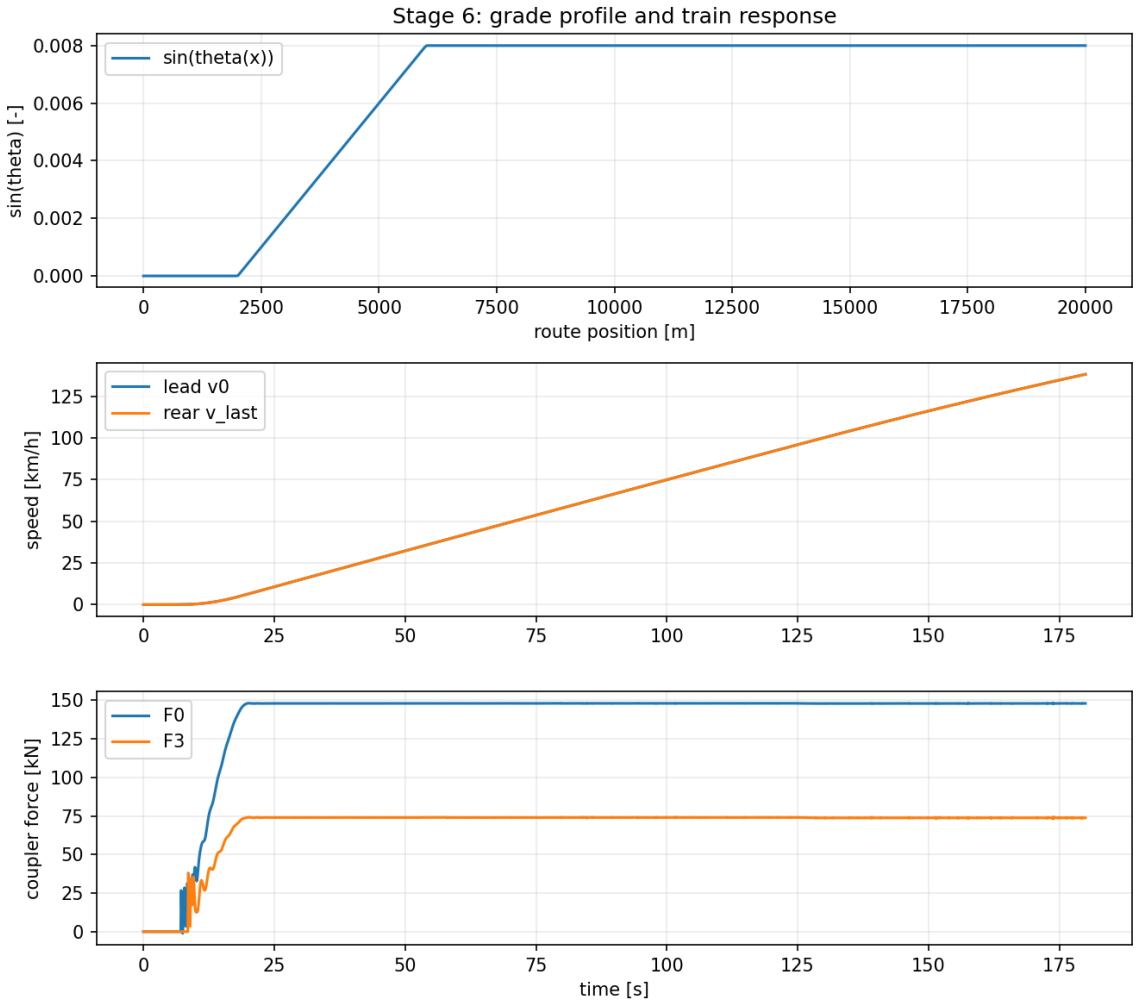


Figure 4.8: Stage 6: interpolated grade profile $\sin\theta$ versus route position, with lead and rear speeds and coupler forces versus simulation time.

with $P_{\max}$ an illustrative constant power ceiling and ε a small speed regulariser. These substitutes replace instantaneous step forces in eq. (4.10). Figure 4.9 shows lead speed together with the lead's z_{brk} and z_{trac} for a scenario that overlaps traction command with braking command; the plotted states should not be confused with achieved $F_{\text{trac},i}$ and $F_{\text{brk},i}$, which also reflect eq. (4.13) and the brake saturation. This stage is what makes the control commands in chapter 3 realistic rather than instantaneous; a full driver model or anti-slip logic is outside the present scope.

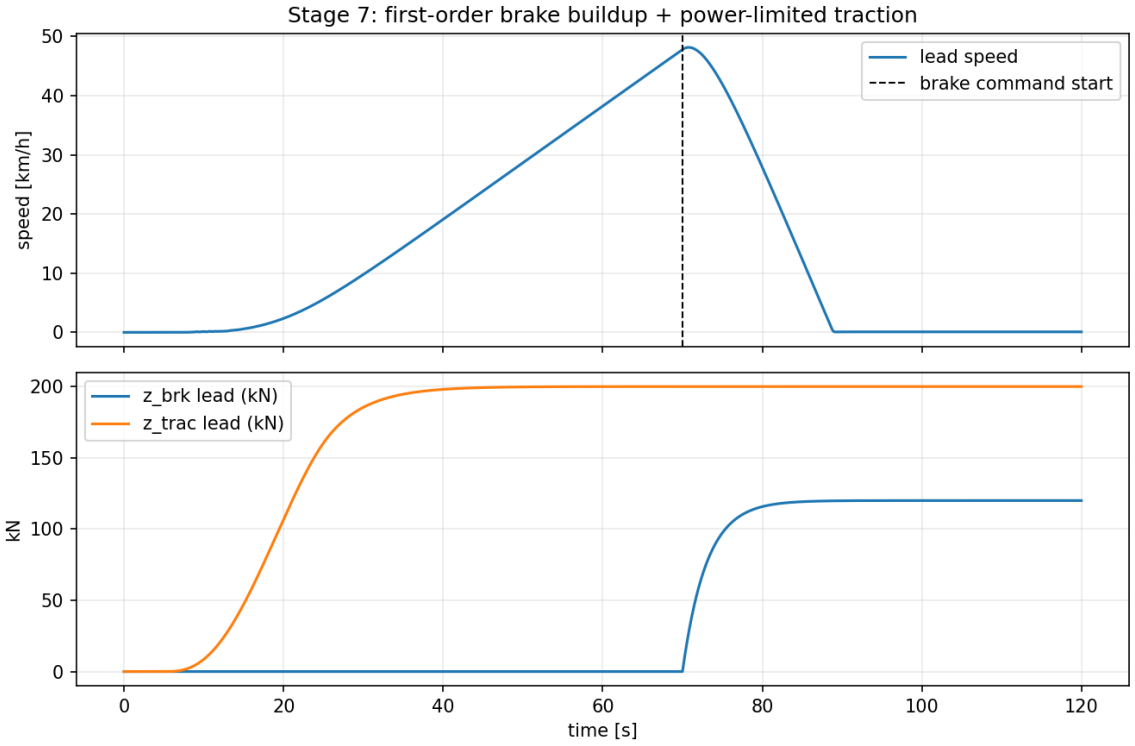


Figure 4.9: Stage 7: lead speed with brake-command onset marker; first-order lag states z_{brk} and z_{trac} on the lead versus time.

4.8 Stage 8: scope relative to industrial high fidelity

This build-up closes with a structured discussion of effects commonly associated with "high fidelity" in operational or research simulators: hysteretic draft gear, pneumatic brake pipe dynamics, distributed traction and adhesion limits, stochastic parameters, and calibration to field data. Those modules are not part of the reference code path summarised here; they mark the boundary between the intermediate-fidelity simulator used for the first learning experiments in this thesis and a fully industrial toolchain. Part II returns to the first two: a friction draft gear in chapter 11 and brake pipe propagation in chapter 12. Packaging this right-hand side as a batched GPU integrator for dataset generation is described in chapter 7.

4.9 Chapter summary

This chapter documented the reference simulator, built up one physical effect at a time: a single vehicle under traction and resistance, a linear coupler, a nonlinear slack coupler with asymmetric buff and draft, a general chain of N vehicles, per-vehicle grade and curvature sampled from a route profile, and actuator lag with power-limited traction. The result is a reproducible, intermediate-fidelity simulator that matches the multi-body, route-coupled structure of the problem stated in chapter 3, and that can generate consist configurations, routes, and control plans in the volume needed to train a surrogate model. Chapter 5 builds the real-terrain route profiles along which it is driven.

Chapter 5

Route Acquisition and Terrain Data Pipeline

Section 3.2 defined the route profile $\mathcal{R}$ as a piecewise field of grade, curvature, and speed limit sampled along a track. Chapter 4 then treats that field as an input it is simply handed. This chapter explains where that input actually comes from.

One of the locked decisions behind this thesis is that routes are built from real terrain rather than invented from scratch (section 3.2). The reasoning is straightforward: a surrogate model is only as convincing as the data it is trained on, and a grade profile that looks nothing like a real railway line is a weak foundation for later claims about generalization. Satisfying that decision, however, is not a small task. It requires pulling raw track geometry from an open mapping database, turning that geometry into clean, continuous corridors, attaching real elevation data to it, and converting the result into the exact numerical fields the simulator and surrogate expect. This chapter documents the pipeline that does that.

The pipeline is organized as four stages plus an export step, each with its own storage folder and its own tab in an accompanying application:

$$\text{acquire} \rightarrow \text{resample} \rightarrow \text{elevation} \rightarrow \text{profile} \rightarrow \text{export}$$

It is built to run in batches: a route is fetched and checked, good routes are collected, and a dataset is generated for that batch, rather than attempting to process an entire national rail network in one pass.

Five corridors were acquired this way, and the two datasets of chapter 7 are built on them. This chapter describes the pipeline as it stood when those datasets were built. Chapter 10 returns to the elevation of one of the five corridors, and chapter 13 to the larger corridor set that replaced them.

Figure 5.1: Overview of the route acquisition pipeline, from raw OpenStreetMap track data to the route-tensor fields consumed by the reference simulator.

5.1 Stage 1: Acquisition

5.1.1 Why acquisition has to be area-first

Railway lines are not tidy, self-contained objects in OpenStreetMap (OSM) [50]. Track is mapped as many short, disconnected segments called ways, and while OSM does define route relations that are supposed to group these ways into a single named line, coverage for United States freight corridors is incomplete and a given relation does not reliably correspond to one clean, walkable corridor. Waiting for well-formed relations to exist would leave large parts of the network unusable.

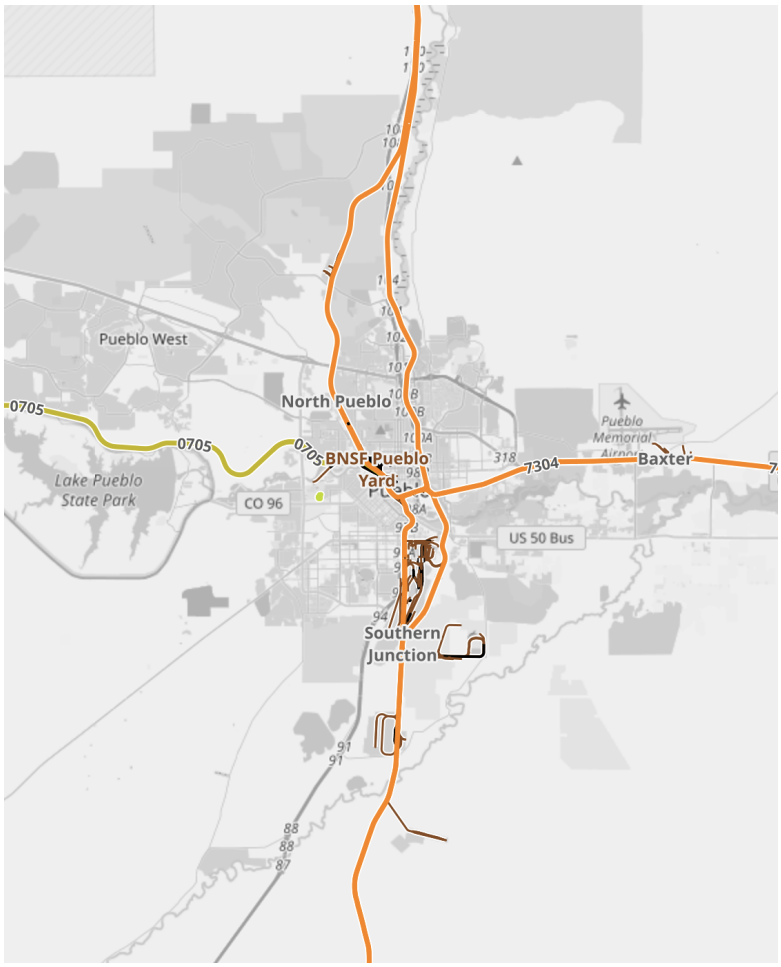


Figure 5.2: Railway mapping around Pueblo, Colorado, rendered from OpenStreetMap track data. Mainline corridors (orange) run through a dense tangle of yard and service track (brown) and meet at several junctions. There is no single map object corresponding to "the line through Pueblo"; corridors have to be recovered from the segments.

The pipeline therefore treats selection as area-first. A user specifies a bounding box or a radius around a point, all track ways inside that area are pulled, and the resulting tangle of segments is split back into corridors algorithmically rather than relying on OSM's own

grouping. Fetching by relation ID is kept as a secondary path for the cases where a good relation happens to exist.

5.1.2 Corridor extraction as a graph problem

Once an area's track ways are pulled, they are assembled into a graph whose vertices are the endpoints and junctions of the track and whose edges are the individual way segments. Every vertex falls into one of three roles: a *continuation* node has degree two and simply passes a line through; a *line-end* has degree one and marks where a piece of track terminates; a *junction* has degree three or higher and marks a real branch point. Corridors are then extracted as maximal paths through continuation nodes, breaking wherever a line-end or a junction is reached. This is what turns an unordered pile of map fragments into a set of distinct, traceable lines.

For the radius-based query mode, an optional bounded outward crawl extends this further: starting from the area first pulled, the pipeline follows each line's open ends past the edge of the original radius, issuing additional queries just beyond them, until each line reaches a natural end or a hard cap on distance, iteration count, or total ways is hit. This captures lines that only pass through the queried area rather than starting or ending inside it, while the cap keeps the crawl from expanding into the fully connected national network.

5.1.3 Putting fragments back together

OSM splits a single physical rail line into many ways wherever a junction, a bridge, or a change in tagging occurs, and sidings introduce additional junctions that fragment a line even further. A long corridor can therefore arrive as dozens of disconnected pieces. A grouping step re-links collinear fragments, across both real junctions and small unshared endpoint gaps, using endpoint proximity together with heading continuity (whether the fragments point the same direction where they meet). Nothing is merged or discarded in this step; fragments keep their identity and remain reversible, they are simply labeled with a shared line identifier and an order along that line, so later stages can stitch them into one ordered polyline. Ways tagged as sidings, spurs, yards, or crossovers are flagged as service track and kept out of this mainline grouping, though they can still be included in a batch if a user specifically wants them.

A related routine builds a robust centerline directly from a set of member ways, whether those ways come from a relation fetch or from a grouped corridor. It stitches unordered and reversed ways into one ordered polyline using shared endpoint nodes, with a coordinate-based fallback when node identifiers do not line up, and it explicitly detects gaps (gaps in space) and branches (points where more than one continuation is possible) rather than silently picking one. Duplicate junction vertices are removed, per-vertex tags such as speed limit, bridge, tunnel, cutting, and embankment are carried along, and geodesic arc length is computed for the whole line.

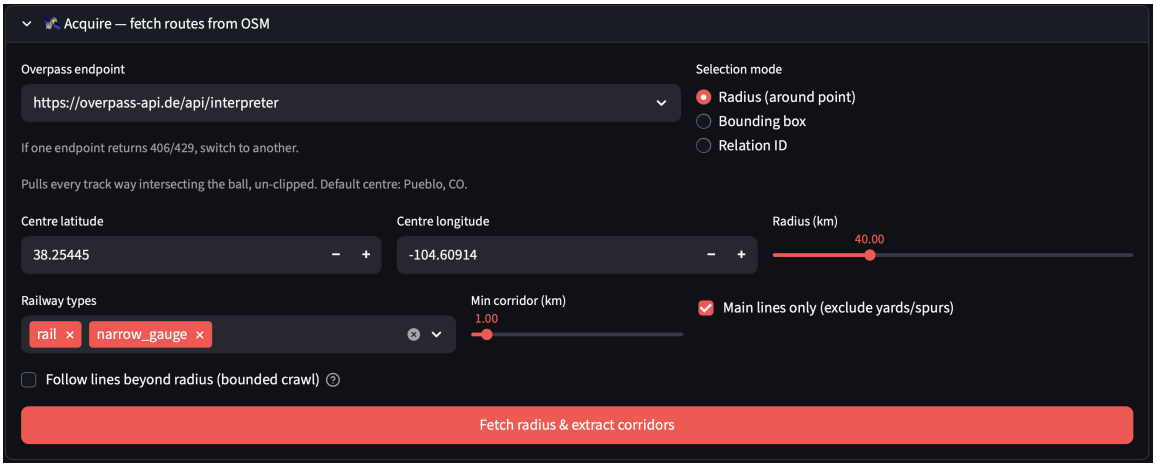


Figure 5.3: The acquisition panel of the route application. A selection mode (radius, bounding box, or relation identifier) is chosen on the right; the centre point, radius, admissible railway types, and a minimum corridor length are set on the left. The mainline-only filter excludes yards and spurs, and the bounded outward crawl described in section 5.1 is toggled at the bottom. Several Overpass endpoints are selectable because any one of them may be rate-limiting at a given moment.

5.1.4 Interactive selection

An accompanying Streamlit application exposes both acquisition modes. In area mode, a user enters a bounding box, the app fetches and extracts all corridors in it, and shows them color-coded on a map alongside a sortable table and per-corridor quality checks (gap count and size, branch points, how much of the line has known speed limits, and what fraction runs over bridges or through tunnels). Corridors selected from this view are added to a batch for export. In relation mode, a single named route is fetched and inspected the same way. For the common case of just wanting a handful of routes near a particular place, the simplest path is radius mode with the outward crawl enabled: a user clicks a line on the map, its full extent highlights, and it is written to storage with every vertex's structural flags and speed-limit coverage already attached.

5.2 Stage 1.5: Resampling to a uniform grid

OSM digitizes track adaptively with respect to horizontal curvature: dense vertices on curves, sparse vertices on straight tangents. That density pattern is a poor match for what the next stage needs, since it under-samples exactly the straight, low-curvature stretches where the vertical profile, grade, still needs to be captured accurately. The resampling stage rewrites each stored route onto a fixed arc-length grid (10 meter spacing by default) by linearly interpolating longitude and latitude along the line's arc length.

Two details matter here. First, step-valued features, speed limit, way identifier, and the bridge, tunnel, cutting, and embankment flags, are not smoothly interpolated; each resampled point simply inherits the value from whichever source segment it falls inside, and

a sample is inserted exactly at the point where such a feature changes, so a speed-limit step or a bridge entrance is not blurred across a grid cell. Second, the resampling step validates its own output: coincident vertices are dropped, arc length is enforced to be strictly increasing, and each point is tagged with the length of the source segment it came from and a flag marking whether it was interpolated across an unusually long or bridged gap in the original data.

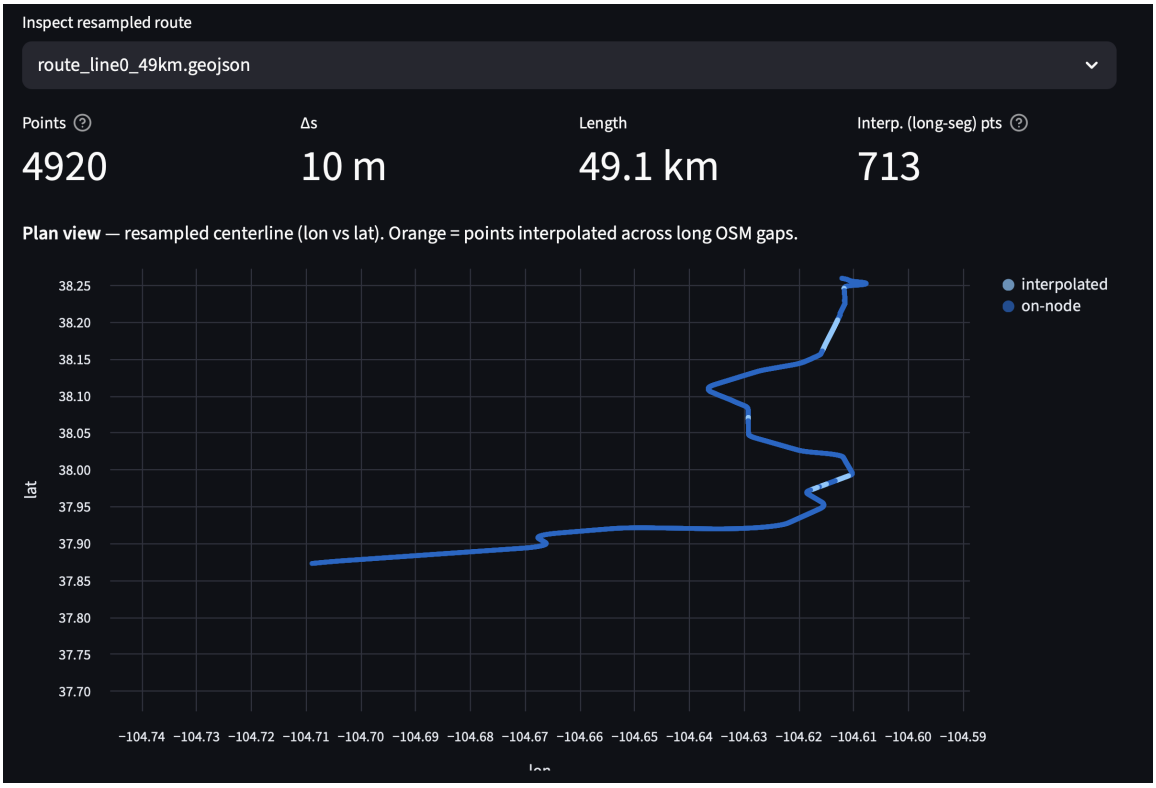


Figure 5.4: Inspection view for a resampled route: a 49.1 km corridor rewritten onto a uniform 10 m grid, yielding 4920 points. The plan view distinguishes points that sit on an original map vertex from those interpolated across long source segments (713 of them here), so stretches where the geometry is inferred rather than mapped remain visible downstream.

5.3 Stage 2: Elevation

Each resampled point is next given a ground elevation from the USGS 3DEP digital elevation model (DEM) [51], producing a three-dimensional route as a sequence of longitude, latitude, and elevation triples. Elevation is queried in batches against 3DEP's image-service endpoint, since it accepts many points per request, with a slower per-point service used as a fallback for any vertex the batch call cannot resolve. Batches hold 1000 points, the most the service returns in one response, and are sent as POST requests, since a request of that size is too long to carry in a URL. Up to sixteen batches run concurrently. Because each request is limited by network latency rather than by server work, throughput scales almost linearly

with the number of concurrent requests, reaching about 16,000 points per second. A batch that fails is reported as a failure rather than silently passed to the per-point fallback, which would otherwise hide a broken batch path behind a pipeline that still works, only several hundred times more slowly. Requests retry with backoff when the service is busy or briefly unavailable. Any point elevation cannot be determined for is left as a missing value rather than guessed, and every earlier per-vertex field from acquisition and resampling is preserved alongside the new elevation data.

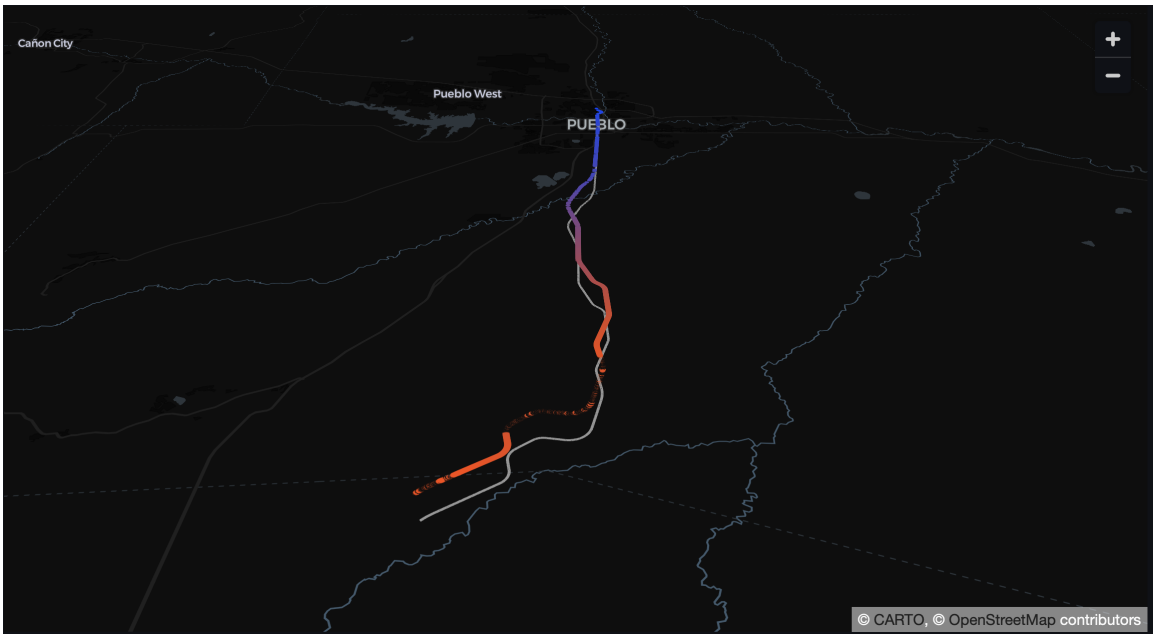


Figure 5.5: An elevated route rendered in three dimensions, coloured by elevation along the line, with the mapped ground track shown alongside. Visual inspection at this stage catches corridors that have been stitched incorrectly or that jump between parallel alignments, before any derivative is taken.

5.4 Stage 3: Deriving the route profile

This is the stage that produces the exact fields the reference simulator and the surrogate consume: the grade field $\sin\theta(s)$, the curvature proxy $\kappa(s)$, and the speed limit $v_{\max}(s)$ introduced in section 3.2.

5.4.1 Correcting for bridges and tunnels

On a bridge or inside a tunnel, the DEM reports the elevation of the structure or the ground above it, not the elevation of the rail itself, and it sometimes reports no value at all. These stretches are identified using the structural flags carried since acquisition, discarded, and filled in by linear interpolation across the gap.

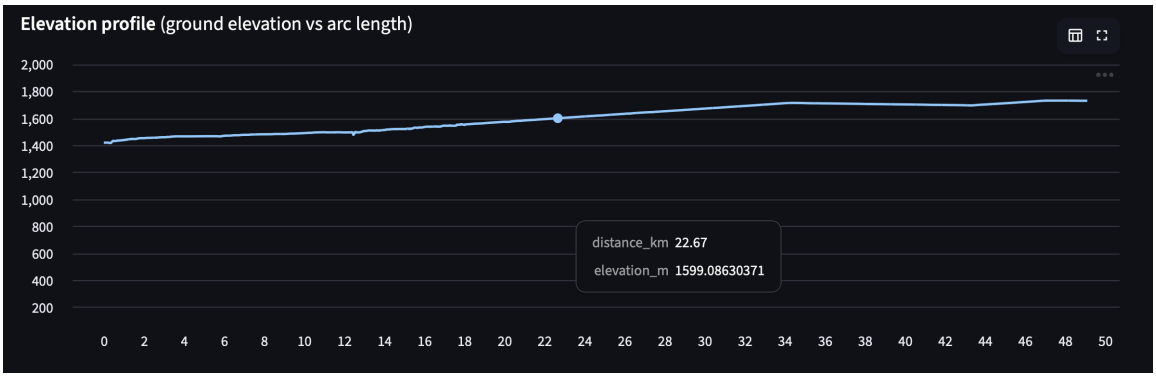


Figure 5.6: Ground elevation against arc length for the same corridor, climbing from roughly 1430 m to 1740 m over 49 km. This is the raw signal from which grade is derived; the metre-scale error it carries is negligible against the elevation range plotted here but is large against the elevation *change* between adjacent 10 m samples, which is what motivates the smoothing described below.

5.4.2 Smoothing and differentiating together

Raw DEM elevation carries roughly a meter of error, which is small compared to elevation itself but large compared to elevation's rate of change. Differentiating the raw signal directly would produce a grade estimate that is almost entirely noise. Instead, a Savitzky-Golay filter [52] is applied, which fits a local polynomial through a moving window of points and yields a smoothed elevation value and its analytic derivative from the same operation, rather than smoothing and differentiating as two separate, error-compounding steps. Grade is then computed as $\sin\theta = \sin(\arctan(\mathrm{d}z/\mathrm{d}s))$, with a clamp on the maximum grade. The corpora of chapter 7 were built with a 200 m window and a limit of 4%, which is well above the ruling grades of freight main lines. Table 5.1 gives the resulting grade statistics for the five corridors. `route_line2` is the steepest of them, which is why it was chosen for the steep-grade evaluation split (section 7.5).

Corridor	rms grade	95th percentile	Maximum	Share at limit
`route_line0` (49 km)	1.03%	1.90%	4.00%	1.3%
`route_line2` (33 km)	2.46%	4.00%	4.00%	22.7%
`route_line3` (29 km)	1.01%	1.99%	4.00%	0.1%
`route_line4` (27 km)	0.87%	1.86%	4.00%	0.3%
`route_line5` (22 km)	0.83%	1.71%	3.50%	0.0%

Table 5.1: Grade statistics for the five corridors as used to build `data/v1` and `data/v2`, with a 200 m smoothing window and a 4% grade limit.

Curvature is derived the same way. The horizontal coordinates are projected into a local planar frame, and the first and second Savitzky-Golay derivatives of that planar path give

$$\kappa = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}, \tag{5.1}$$

clamped to a minimum turning radius for the same reason grade is clamped.

5.4.3 Speed limit and output

Speed limit is converted from the map's tagged value into meters per second where available, and falls back to an estimate based on the Federal Railroad Administration's track classes [53] where OSM coverage is missing. The finished profile is written in two forms: a GeoJSON file for visual inspection, and a compact array file holding exactly the fields `route_s`, `route_sin_theta`, `route_kappa`, and `route_vmax`, matching the route-tensor layout used for dataset generation (chapter 4). As a final check, the array file is used to construct an actual route-profile object and sample it, confirming the file behaves correctly before it is trusted for later stages.

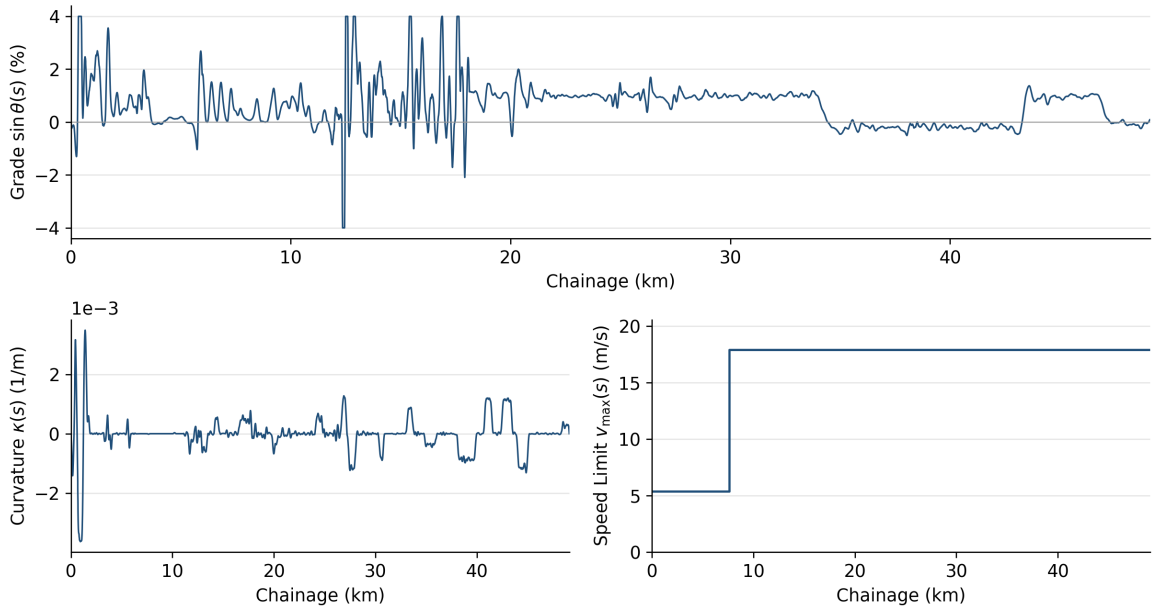


Figure 5.7: The three derived fields the simulator consumes for `route_line0`, plotted against chainage in kilometres: grade $\sin\theta(s)$ as a percentage (top), the curvature proxy $\kappa(s)$ (bottom left), and the speed limit $v_{\max}(s)$ (bottom right). Fields are shown as used to build `data/v2`, with a 200 m smoothing window and a 4% grade limit. The noisier grade over the first 20 km is the urban approach, where frequent bridges, junctions, and short mapped segments degrade the elevation signal. There the grade reaches the 4% limit in several short spikes, 1.3% of the corridor's length. Beyond it the line settles onto a sustained climb near one percent. The speed limit is a step field, here rising where the line leaves restricted urban trackage. Figure 13.1 shows the same corridor after the grade field was revised.

5.5 Export

Finished routes are collected into a single export bundle: one array file per route, an index summarizing each route's location, endpoints, length, and grade and curvature statistics,

and a manifest recording what each channel means and how the route was produced. This bundle is the handoff point between route acquisition and dataset generation; its array files drop directly onto the reference simulator's route-profile input with no further conversion.

5.6 Testing and design choices

The line-stitching logic, the part of the pipeline responsible for turning disordered, possibly reversed map fragments into one correct ordered corridor, is unit tested offline against a fixture built from a captured query response, so its correctness does not depend on network access or on live map data staying unchanged.

A few choices run through the whole pipeline. Acquisition and assembly are pure functions wherever they do not require a live network call, which is what makes offline testing possible in the first place. Data quality problems are surfaced rather than silently patched: gaps are bridged but their size is reported, branch points are flagged rather than resolved automatically, and disconnected fragments are dropped and listed rather than dropped quietly, so a user decides whether a given route is clean enough to use. Finally, the core pipeline, acquisition, resampling, elevation fusion, and profile derivation, depends only on NumPy and an HTTP request library; the Savitzky-Golay smoothing and differentiation is a small NumPy implementation of its own rather than a dependency on a larger scientific computing package, so the whole path can run and be tested without a heavier install.

5.7 Chapter summary

This chapter documented the pipeline that turns raw OpenStreetMap track data and public elevation data into the route profiles this thesis relies on: area-based acquisition and graph-based corridor extraction, fragment stitching back into whole lines, resampling onto a uniform grid, elevation fusion, and Savitzky-Golay-based derivation of grade, curvature, and speed limit. Five corridors were built this way, with a 200 m smoothing window and a 4% grade limit, and they carry the datasets of chapter 7. The output, arrays of `route_s`, `route_sin_theta`, `route_kappa`, and `route_vmax`, is exactly what the reference simulator in chapter 4 consumes and what later chapters use to generate training data for the surrogate. Because this pipeline draws on real corridors rather than invented ones, it is also what makes it possible to hold out whole real corridors for the generalization experiments described in later chapters, rather than only holding out synthetic variations of the same underlying route.

Chapter 6

RailLab: A Desktop Research Console

Chapter 4 built the reference simulator and chapter 5 built the pipeline that feeds it real terrain. Both are libraries: they are driven from notebooks and scripts, and everything they need, a consist, a route, a set of driving commands, arrives as Python objects assembled by hand. That is a workable way to produce a figure for a chapter. It is a poor way to explore, and exploration is most of what the work in the following chapters actually consists of.

This chapter documents *RailLab*, a desktop application built over the same backend. It is not a deliverable of the thesis in its own right, and none of the numerical claims in later chapters depend on it. It is a research instrument, and it is documented here for the same reason the simulator's staging is documented: the way a tool shapes what questions get asked is part of the method.

6.1 Why a graphical console

Three problems motivated building it.

The first is the cost of a question. Asking "what happens to coupler forces if this train is thirty cars longer on this particular grade" requires, in script form, editing a consist definition, re-resolving car parameters, rebuilding a scenario, re-running the solver, and re-plotting, with the answer arriving a few minutes and several opportunities for error later. Most such questions are never asked, not because they are uninteresting but because the friction exceeds the curiosity. Collapsing that loop to a dropdown and a button changes which questions get asked at all.

The second is the gap between a number and a shape. A simulator returns arrays. The failure modes that matter here, a run-in that the coupler model resolves unphysically, a grade artifact surviving the smoothing stage in chapter 5, a control profile whose brake command overlaps traction in a way no operator would produce, are visible immediately in a plot and nearly invisible in a summary statistic. Later chapters lean on trip-level error metrics; those metrics are trustworthy only in proportion to how much of the underlying trajectory behavior has been looked at directly.

The third is reproducible bookkeeping. The experiments in later chapters need many consists run against comparatively few routes, and every artifact involved, a car type, a train, a control profile, a route, has to be nameable, saveable, and reloadable so that a run

can be repeated exactly. Building an interface around those artifacts forced them to become first-class, serializable objects in the backend rather than variables in a notebook cell, which is a benefit the headless path inherits.

6.2 Architecture

The governing constraint is that the user interface never imports the numerical stack. The backend packages, `simulator2` and `routegen`, are unchanged by the application's existence; the application reaches them only through a thin service layer. Figure 6.1 summarizes the arrangement.

Figure 6.1: RailLab's layering. Screens call services; services are the only modules that import the numerical backend, and they return plain data. The backend packages are unchanged by the application's existence and continue to run headless.

The layering has three consequences that matter more than the tidiness. The backend still runs headless, so notebooks, the batch dataset generator, and the figures in this document have no dependency on the interface toolkit. The services contain no interface code at all, so each is unit-testable without a display. And the boundary is a natural insertion point: the surrogate model and the controller developed in later chapters attach as additional services without any existing screen changing.

Within the application, each screen owns one stage of the workflow and communicates through shared service objects rather than by reaching into its neighbors. The CONSIST and SIMULATE screens share one consist service, so a train edited on one appears immediately on the other; CONTROLS and SIMULATE likewise share a control-profile service. Signals from a finished run are fanned out by the window shell to the RESULTS screen and the status indicator, so neither screen holds a reference to the other.

Long solver calls run off the interface thread. Pressing GO constructs a scenario, hands it to a worker object moved onto a dedicated thread, and returns immediately; the worker emits either a result or an error message when the integration finishes. Because the reference simulator integrates the whole time span in one adaptive solver call, there is no intermediate hook to report progress against or to interrupt, and the stop control is correspondingly disabled. This is a real limitation rather than an oversight: interrupting or streaming a run requires a stepwise integrator, which the reference implementation deliberately does not use.

6.3 The workflow, screen by screen

6.3.1 Route

The route screen browses the output of chapter 5. Each processed route is listed, and selecting one loads its per-vertex arrays and renders three chainage-linked plots, elevation, grade, and curvature, alongside a summary panel: length, vertex count, elevation range, total climb and descent, grade range, maximum curvature and the corresponding minimum

radius, speed-limit range, and the fraction of the line for which the map actually supplied a speed limit. Routes lacking the compact array file the simulator consumes are listed but marked, and the simulate screen will not offer them.

That last field, speed-limit coverage, is the reason the panel exists in this form. It is a data-quality property, not a physical one, and it is exactly the sort of thing that is easy to forget when a route is loaded programmatically and impossible to overlook when it is printed next to the grade range every time the route is opened.

6.3.2 Consist

The consist screen is where trains are assembled, and it implements the three-way split the data model was designed around: a library of reusable car types, an ordered train under construction, and a shelf of saved trains (fig. 6.2).

A car type is a named template, a locomotive, a loaded grain hopper, a full water tank car, carrying mass split into tare and lading, Davis resistance coefficients, traction and braking ceilings, physical length, and a coupler type. Types are stored one per file as plain text records, so the library is inspectable and version-controllable outside the application. A train is an ordered list of references to those types, which means a train file stays small regardless of length and a correction to a car type propagates to every train that uses it.

Per-car overrides are the escape hatch. Any individual car in a train may override selected fields of its type, tare mass, lading, any Davis coefficient, either force ceiling, while inheriting the rest. This makes "the same train but with one unusually heavy car" expressible without polluting the library with a near-duplicate type. A running summary along the bottom reports car count, total mass, total length, and how many vehicles are powered.

Figure 6.2 shows a consist produced by the procedural generator rather than assembled by hand: a mixed-manifest train of 133 cars, roughly 9,756 tonnes, 1698 metres long, with four powered units at the head. The generator writes through the same save path the interface uses, so generated and hand-built trains are indistinguishable downstream, which is what makes the large training sweeps of later chapters possible without a second code path.

6.3.3 Controls

The controls screen generates the driving commands. A control profile is a train-agnostic recipe: piecewise command curves expressed as a *fraction* of each vehicle's own force ceiling rather than in newtons, so one profile drives any consist. Curves are defined by knots and interpolated with the same smoothstep shape the simulator's traction ramp uses, which keeps commands continuous and avoids the stiffness that instantaneous steps introduce.

Profiles are generated randomly under user-specified bounds: duration, number of knots per curve, traction and brake fraction ranges, and the treatment of distributed power (fig. 6.3). Distributed power units are addressed by role, head-end, mid-train, and rear helper, resolved from each car's label, and may be driven synchronously with a configurable lag behind the head end, independently, or not at all. The preview plots the resulting traction command for each role and the brake command below it; the visible role-to-role offset in the figure is the lag. Generation is seeded, so a profile is reproducible from its configuration alone.

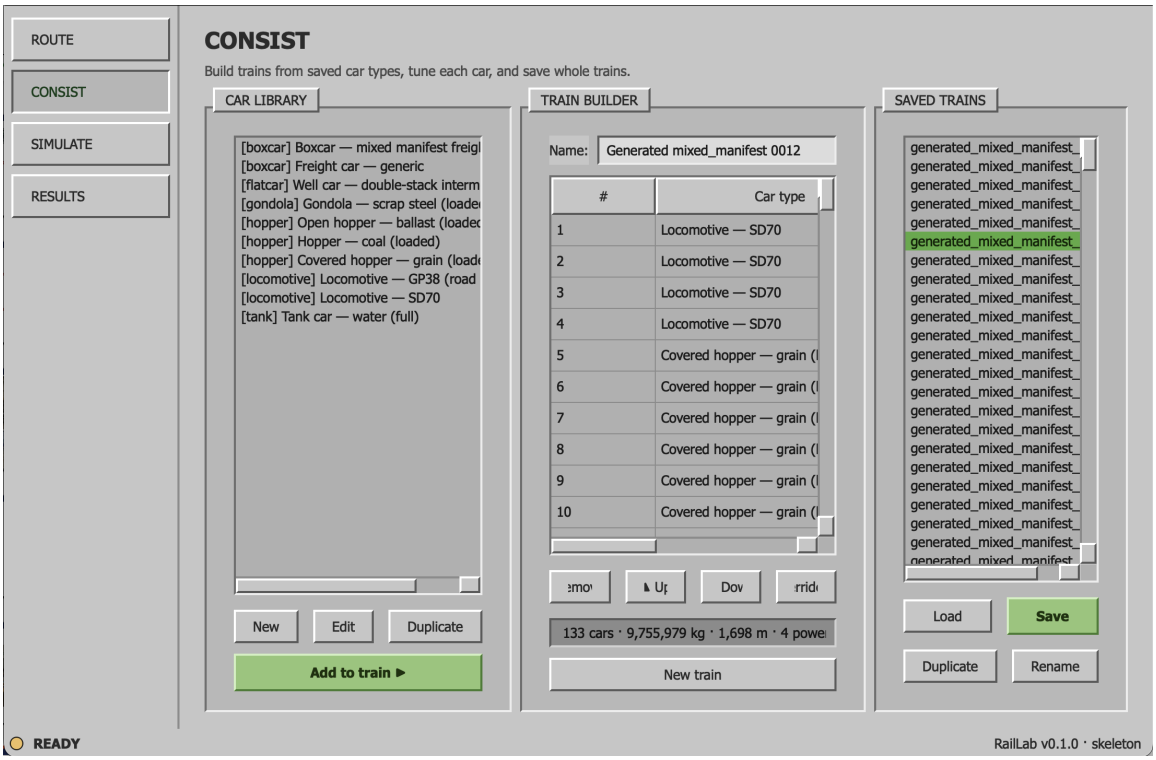


Figure 6.2: The CONSIST screen: a library of reusable car types (left), the ordered train under construction with per-car overrides (centre), and saved trains (right). The running summary reports car count, total mass, total length, and the number of powered vehicles.

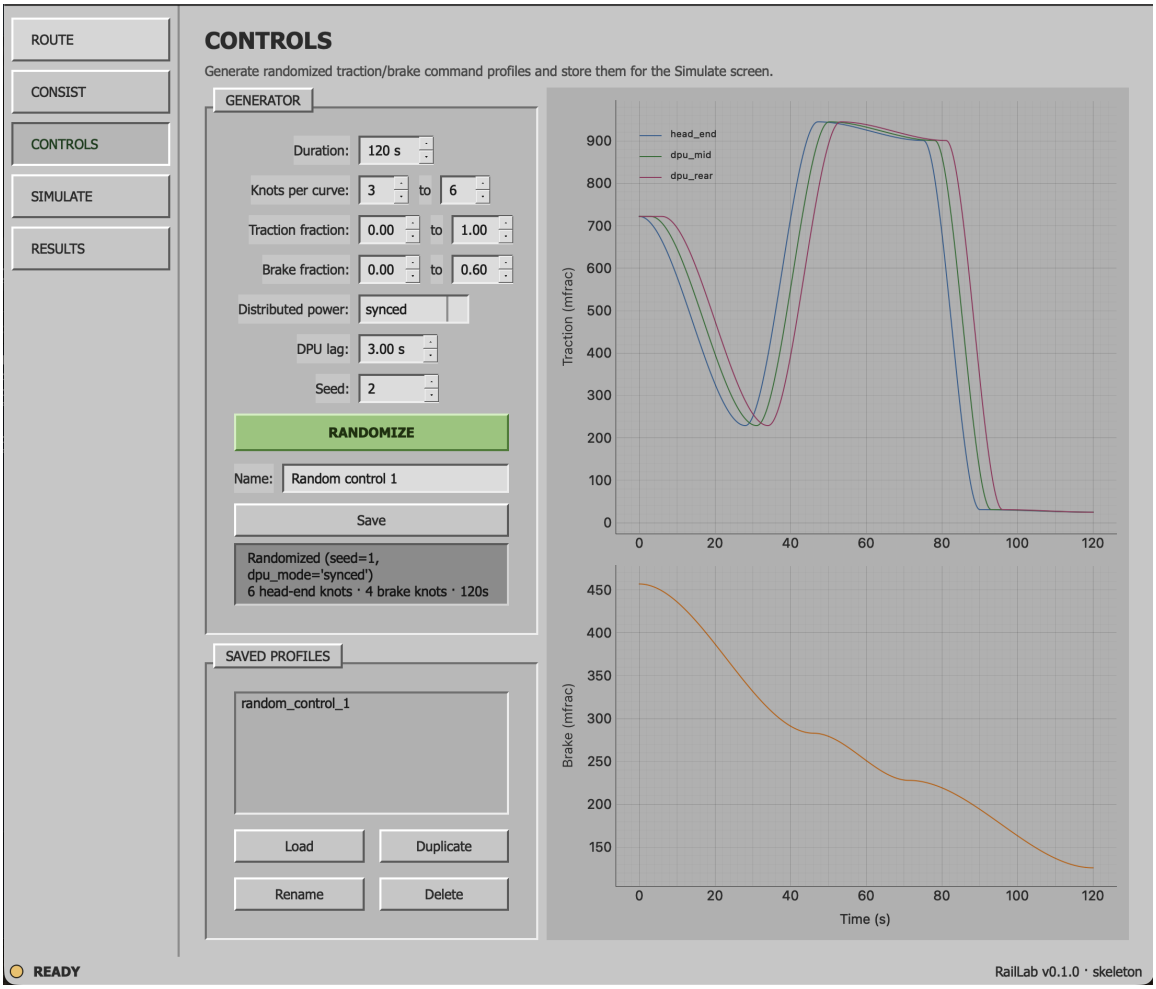


Figure 6.3: The CONTROLS screen: randomization bounds and distributed-power settings on the left, with a live preview of the generated traction command per power role (top right) and the shared brake command (bottom right). The offset between the three traction traces is the configured lag of the mid-train and rear helper units behind the head end.

Randomized commands are what the surrogate's training set needs: a model trained only on hand-designed, sensible driving would never see the transients that make coupler dynamics hard. The bounded ranges keep the randomness within a plausible envelope rather than making it arbitrary.

6.3.4 Simulate and Results

The simulate screen is deliberately spare. It selects a saved train and a route, sets duration, an optional traction ceiling override, and the curvature-term scale, picks a control profile or falls back to the default ramp, and runs. Everything it needs has already been named and saved elsewhere, which is the point: a run is fully specified by a short list of named artifacts and four numbers, and is therefore trivially repeatable.

Results are plotted as they arrive: every vehicle's speed on one axis and every coupler's force on a second, time-linked axis below it (fig. 6.4). Plotting every vehicle rather than a summary is intentional. The figure shows a 126-vehicle consist over 120 seconds under a randomized control profile, and the structure that carries the information is the *spread* between vehicles: the envelope of the speed traces widening and collapsing as slack runs in and out, and the corresponding fan of coupler forces. Neither is visible in a mean, and the qualitative behavior, oscillation that decays rather than growing, forces that stay bounded, is the sanity check that a summary statistic cannot provide.

6.4 Status and limitations

RailLab is a working instrument, not finished software, and several limitations bear on how it should be read.

Runs cannot be interrupted or monitored mid-flight, for the reason given in section 6.2: the solver call is atomic. Long runs therefore occupy the application until they complete, though not the interface thread. The results screen plots the two channels that matter most for the present work and does not yet expose the full node and edge state, nor does it export a run directly; dataset generation remains the job of the headless batch path, which is the correct division, since the interface is for looking at one run and the batch runner is for producing thousands.

Most importantly, nothing in this chapter changes the standing caveat from chapter 4: the parameters in the car library are realistic in shape but are not calibrated to any specific real rolling stock. The application makes it easy to build a train and watch it behave, and that ease should not be mistaken for validation. What is being explored is the reference simulator's behavior, which is exactly what the surrogate of the following chapters is trained to reproduce.

6.5 Chapter summary

This chapter documented RailLab, the desktop console built over the reference simulator and route pipeline. Its architecture keeps the interface strictly separated from the numerical backend through a service layer, so the backend remains headless and testable and later

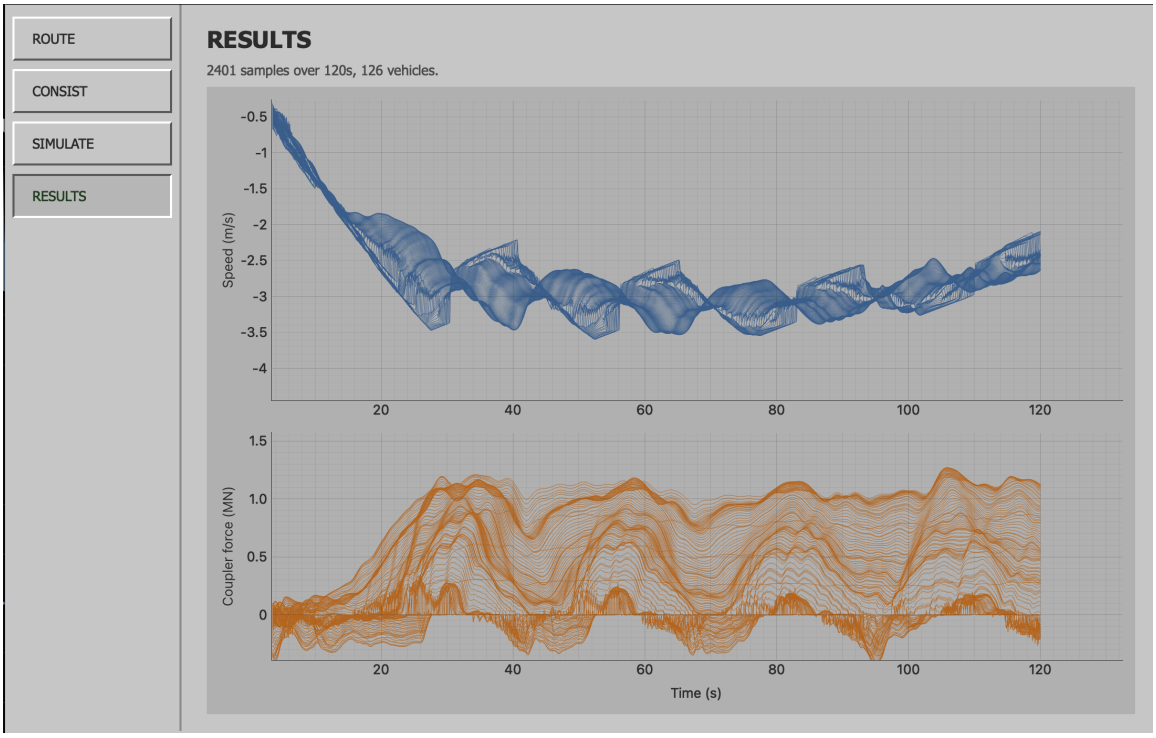


Figure 6.4: The RESULTS screen after a 120 s run of a 126-vehicle consist under a randomized control profile. Every vehicle's speed is plotted above every coupler's force, on time-linked axes. The information is carried by the spread between traces: the widening and collapsing speed envelope is slack running in and out, with the corresponding fan of coupler forces below.

components attach without disturbing existing screens. Its five screens follow the workflow the thesis needs: inspect a route, assemble a consist from a reusable car library, generate randomized driving commands, run, and look at the result. The artifacts it manipulates, car types, trains, control profiles, routes, are the same named, serializable objects the headless batch tooling uses, so interactive exploration and large-scale dataset generation share one representation. Chapter 7 uses that representation to generate the training data at scale.

Chapter 7

GPU Integrator and Dataset Build

Chapter 4 built the reference simulator and chapter 5 built the routes it runs on. Together they can produce one scenario at a time: a consist, a route and a set of driving commands go in, and a trajectory comes out. Training a surrogate needs thousands of such scenarios, spanning consists from ten to one hundred and fifty vehicles, and this chapter describes how they were produced.

The chapter has two halves. The first shows that the reference simulator, as written, could not produce a dataset of the required size, and describes the batched GPU integrator that replaced it for data generation: how it is built, how it was checked against the reference, and what it costs to run. The second describes the dataset itself: how scenarios are sampled, how they are stored, how the two corpora were built and validated, and the known problems in them.

One point about scope is worth making at the start. The GPU integrator is needed to generate the data and to measure the surrogate's speedup against a fair reference. It is not part of the surrogate. Under the formulation of section 3.1.4, the model never evaluates the physics right-hand side during training or inference; the right-hand side appears only in the data the model is trained on.

7.1 Why the reference simulator could not build the dataset

The reference simulator of chapter 4 evaluates its right-hand side in NumPy and integrates it with SciPy's adaptive `solve_ivp`. To estimate the cost of a dataset, it was run on 66 scenarios: three consist sizes, two routes, and the eleven driving regimes described in section 7.3.1, each simulating 120 s with a wall-clock budget of 30 s per run. Table 7.1 gives the result.

At $N = 130$, the size range the held-out long-consist split depends on, not one run finished. The dataset also needs runs of 300 to 600 s, long enough to cross varied terrain, which multiplies the cost again. A rough estimate for 10,000 scenarios of 300 s was 170 CPU-hours, with the large consists effectively excluded.

Two causes account for the cost. The first is that the right-hand side loops over vehicles in Python, so each evaluation at $N = 130$ performs 130 interpreted iterations, and a run

Consist size N	Runs completed	Median wall clock	Notes
11	16 of 22	4.0 s	3 regimes exceeded 30 s
60	4 of 22	28.1 s	18 of 22 exceeded 30 s
130	0 of 22	> 30 s	every run exceeded 30 s

Table 7.1: Cost of the NumPy reference simulator on 120 s scenarios, with a 30 s wall-clock budget per run. At $N = 130$ no run finished within budget.

needs tens of thousands of evaluations. The second is the coupler slack. The force law of Stage 3 changes slope abruptly at the edge of the slack, and the adaptive solver reduces its step whenever any coupler crosses that edge. On a long consist some coupler is almost always crossing. The per-regime medians show the effect directly: the smooth regimes (cruise, notch up, power to coast) took about 4 s, while dynamic-brake descent took 27.3 s and throttle modulation 15.8 s. Those slow regimes produce the largest coupler forces, which are the forces the safety constraint concerns, so they cannot be sampled less often to save time.

7.2 A batched integrator on the GPU

7.2.1 Design

The replacement is a PyTorch implementation of the same right-hand side, written so that one evaluation handles a whole batch of scenarios at once. State is held as tensors of shape $B \times N$, for B scenarios of N vehicles each, and the per-vehicle loop becomes a set of operations over whole tensors. Coupler forces, route sampling at each vehicle's position, resistance, traction and braking are all computed this way, so the cost of an evaluation no longer grows with the number of vehicles through the interpreter.

The adaptive solver is replaced by the classical fourth-order Runge–Kutta method with a fixed step of $\Delta t = 0.02$ s. This makes the cost of a run fixed in advance at $\lceil \text{duration}/\Delta t \rceil$ steps, whatever the driving regime. A regime full of slack events costs the same as a steady cruise, and a run can no longer stall. The per-scenario wall-clock limit that the NumPy benchmark required is therefore not needed. States are recorded every 0.25 s, so each stored transition spans 12.5 integration steps, or fifty evaluations of the right-hand side. That ratio is the one quoted in section 3.1.4 for the surrogate's single step.

The coupler law keeps its exact piecewise form. The port's contract allowed the slack transition to be smoothed if fixed-step error at slack crossings proved unacceptable. It did not, as the next subsection shows, so no smoothing was applied.

7.2.2 Agreement with the reference

The port was held to an acceptance contract of 60 tests, written before the port existed, which compare it against the NumPy simulator on fifteen reference scenarios. These include one scenario for each driving regime and several built to be difficult, such as a consist that cycles repeatedly in and out of slack. The main tolerance is 0.05 m/s on vehicle velocity.

Beyond velocity, the contract requires that couplers engage and release at the same times as in the reference in at least 98% of samples, and that peak coupler force agree within 5%, on every scenario; both hold. The 135 existing tests of the reference simulator were left unmodified and continue to pass.

Two properties of the NumPy reference were reproduced exactly rather than corrected, so that the two implementations describe the same model. The grade force applies a sine to a field that already stores $\sin\theta$, computing $mg\sin(\sin\theta)$ rather than $mg\sin\theta$. The relative difference between the two is $\sin^2\theta/6$: under two parts in 10^5 at a 1% grade, and 2.7×10^{-4} at the 4% grade limit, where it amounts to about 10 N on a loaded 100 t car. That is far below anything the dataset resolves. The Davis resistance also uses a hard sign function of velocity, which is discontinuous at standstill. Both are recorded here because they are properties of the data-generating model, and correcting them in one implementation but not the other would break the comparison.

7.2.3 Step size

The step was chosen by sweeping Δt against the reference scenarios in double precision. Ten of the fifteen reference trajectories were generated with a tight solver tolerance, and on those the error falls steadily as the step shrinks (table 7.2). The other five were generated at the solver's default tolerance, and on those the error stops falling below $\Delta t = 0.01$ s and rises again at 0.005 s. The remaining difference there is error in the reference, not in the port, so those five cannot justify a smaller step.

Δt (s)	Maximum velocity error (m/s)
0.05	0.0120
0.02	0.0063
0.01	0.0040
0.005	0.0025

Table 7.2: Maximum velocity error of the GPU integrator against the ten tightly integrated reference scenarios, in double precision.

The error falls roughly in proportion to the step rather than to its fourth power. This is expected: the error is dominated by the slack transition and the Davis sign discontinuity, not by the truncation error of the method, so refining the step buys less than the method's order suggests. The step of 0.02 s sits eight times inside the tolerance, and halving it would cost 1.8 times more for a 1.6-fold reduction in error that the tolerance does not require.

7.2.4 Launch overhead and CUDA graphs

A first version ran each integration step as a sequence of separate GPU operations. Profiling at $B = 64$, $N = 130$ showed 661 operation launches per step, with the GPU idle about 92% of the time: the cost was the overhead of issuing work, not the work itself. Two changes removed it (table 7.3). Redundant operations were removed first, for example by sharing one index lookup between route fields sampled on the same grid, which gave a 1.6-fold gain.

The larger gain came from recording one complete integration step as a CUDA graph, a sequence of GPU operations captured once and replayed as a single launch. Replaying the recorded step gave a further 10.1-fold gain.

Configuration	Wall clock
Separate operations, before removing redundancy	61.5 s
Separate operations, after removing redundancy	38.2 s
Recorded step replayed as a CUDA graph	3.79 s

Table 7.3: Effect of launch overhead on the GPU integrator, for $B = 64$ scenarios of $N = 130$ vehicles and 120 s of simulated time.

PyTorch's own compiler offers a mode intended for this situation, but it failed during rollouts, inside its own bookkeeping, because a rollout keeps one output tensor for every stored sample. Recording the step directly avoided the problem and has two further advantages: the state lives in fixed buffers that the recorded step updates in place, so a replay never returns control to the host, and nothing depends on a compiler toolchain at run time. The recorded path is used only for rollouts that do not need gradients. Rollouts that do use the ordinary path, since recording would fix the gradient computation into static buffers.

7.2.5 Precision

The port's specification assumed that double precision would be impractical, since the GPU used here performs double-precision arithmetic at about one sixty-fourth of its single-precision rate. The measurement contradicted this (table 7.4). Double precision cost about 20% more, not 64 times more, because the integrator is limited by per-step overhead rather than by arithmetic, so doubling the width of the arithmetic changes little.

B	N	Single / double precision (s)	Ratio
64	130	4.21 / 5.03	1.20
256	130	4.54 / 5.49	1.21
256	200	4.58 / 5.66	1.24

Table 7.4: Cost of double precision relative to single precision for 120 s of simulated time.

The accuracy margin in single precision, by contrast, was thin. On the worst reference scenario, rounding error alone used 60% of the 0.05 m/s velocity tolerance over a 40 s run, and dataset scenarios are ten times longer. Both corpora were therefore integrated in double precision and stored in single precision. Reducing the precision of the stored output is a separate decision from integrating in reduced precision, and it halves the size on disk without spending the error budget on rounding.

7.2.6 Throughput

Table 7.5 gives wall-clock time on one RTX 4090 for 120 s of simulated time, against batch size and consist size.

$N = 130$, varying B			$B = 64$, varying N	
B	Wall clock	ms per scenario	N	Wall clock
1	3.28 s	3,282	11	3.72 s
16	3.65 s	228	30	3.80 s
64	3.71 s	58	60	3.77 s
128	3.79 s	30	130	3.71 s
256	4.01 s	16	200	3.76 s

Table 7.5: Wall clock of the GPU integrator for 120 s of simulated time, in single precision, against batch size B (left) and consist size N (right).

Wall clock is nearly flat in both directions. Running 256 scenarios instead of one takes 22% longer, and going from 11 to 200 vehicles makes no measurable difference. Time is set by the number of graph replays, one per integration step, and the GPU is far from fully used even at $B = 256$ and $N = 200$. Two consequences follow. Per-scenario cost falls almost in proportion to batch size, so dataset generation should use the largest batch that fits in memory. And a consist of 150 vehicles costs no more to simulate than one of 11, which removes the obstacle that made the long-consist split impossible to generate.

Against the NumPy simulator, which needs at least 89.2 s for a 120 s scenario at $N = 130$ on one core, the GPU integrator is 1,538 times faster at $B = 64$ and 5,682 times faster at $B = 256$. The NumPy figure is a lower bound, projected from a fixed-step count; the adaptive solver it actually uses needs more evaluations than that in the regimes with frequent slack events, so the true ratio is larger.

7.3 Scenario generation

7.3.1 Driving regimes

The simplest way to generate driving commands is to draw each point of a traction and brake profile at random. Most profiles drawn that way combine commands no operator would give, such as full traction with heavy braking, and very few resemble the structured manoeuvres a controller will later produce. Commands are instead generated from eleven manoeuvre types, or regimes: startup, notch up, cruise, throttle modulation, power to coast, coast, coast to brake, dynamic-brake descent, brake release, stretch braking, and emergency braking. Each regime has a fixed shape whose timing and levels are randomized. Regimes are weighted so that steady running and throttle modulation dominate, with emergency braking rare but present, since it produces the most extreme coupler forces.

Traction and brake are never commanded together, with two intended exceptions. Stretch braking holds light brake against light traction to keep the slack stretched, a real handling

technique and exactly the kind of combined command the surrogate needs to see. In an emergency application the brake is applied while traction is still falling, so the two overlap for a few seconds. The rule is enforced by a check applied after each profile is generated rather than relied on by construction, and a sample of 5,500 profiles showed no violations.

Building this check exposed three errors that would have entered the dataset unnoticed. Random variation added to the commands of remote locomotives lifted zero commands slightly above zero, reintroducing traction during braking; the variation is now proportional, so zero stays zero. Random timing variation could reorder the points of a profile, which in one case turned an emergency application into a restoration of power under full braking; the variation is now bounded so that order cannot change. And remote locomotives could be given an entirely different manoeuvre from the lead, so a mid-train unit could pull while the head end braked; they now follow the lead's manoeuvre with a delay.

7.3.2 Randomized scenarios

Each scenario samples a consist from a library of 500, a route, a regime, and the following conditions:

- a starting position anywhere on the route with room to run, and a duration clipped so the consist cannot run off the end;
- an initial speed of up to 90% of the speed limit at the starting position;
- an initial slack state (neutral, stretched, bunched, or random), always strictly inside the slack so no coupler force is present at the start;
- actuator states already settled at the profile's initial command, so a run opens mid-manoeuvre rather than with an artificial ramp;
- an adhesion condition (dry, wet or low), applied as a multiplier on the maximum tractive force of powered vehicles.

Two degenerate cases are handled explicitly. A consist starting at rest with no brake applied rolls backward down any real grade before traction builds, so standing starts are placed only on track flatter than 0.5%. And profiles that brake within the first 20 s are given a minimum starting speed, so a run does not stop in its first seconds and remain stationary for the rest of the window.

7.4 Dataset format

Each scenario is written as one compressed array file holding the channels of the node and edge tensors of section 3.1.3, organized by whether they change over time (table 7.6).

Two decisions keep the format compact. The node tensor has eleven channels, but seven of them are constants for each vehicle, so they are stored once rather than at every time step. And route fields are stored once per corridor rather than in each scenario, since one corridor is shared by about 2,000 scenarios. The edge channels, by contrast, are stored at

Array	Shape	Contents
`state`	$[T, N, 4]$	position, velocity, and brake and traction lag states per vehicle
`node_static`	$[N, 7]$	mass, three Davis coefficients, traction flag, maximum traction and brake force
`edge_dynamic`	$[T, N-1, 3]$	coupler extension, extension rate, and force
`edge_static`	$[N-1, 6]$	nominal spacing, slack half-width, and draft and buff stiffness and damping
`edge_index`	$[2, N-1]$	chain connectivity
`u_trac`, `u_brk`	$[T, N]$	traction and brake commands per vehicle

Table 7.6: Arrays stored for each scenario. Each file also holds the initial state, the integrator constants needed to recompute the right-hand side, trip-level summaries, and a metadata record.

every step even though they can be computed from the node state, because they serve as training targets; they are marked in the file as derived.

Four files describe each build as a whole: a configuration record with the sampling ranges, random seed, integrator settings and code version; an index with one row of 43 metadata fields per scenario; normalization statistics for every channel, computed over the training split only; and an explicit list of the scenarios in each split.

7.5 The two corpora

Two corpora were built from the same seed and the same code, differing only in the curvature law (table 7.7). Because they share every scenario, the difference between them isolates the effect of curve resistance.

	`data/v1`	`data/v2`
Curvature law	off	linear (AREMA), $c_\kappa = 1$
Scenarios	10,000	10,000
Size on disk	12.4 GB	12.4 GB
Integration time	1,336 s	1,389 s

Table 7.7: The two corpora. `data/v2` is the training corpus; `data/v1` exists to measure the curvature term.

Each corpus holds 12.8 million time steps and 753 million vehicle-steps. Consists range from 10 to 150 vehicles, durations from 180 to 599 s (mean 320 s), and scenarios cover 500 consists, eleven regimes and five corridors. A full build took about 25 minutes, against the 170 CPU-hours estimated for the NumPy simulator. Batches of 256 scenarios were used; compression costs about a quarter of the build time and halves the size on disk.

Table 7.8 gives the splits. Corridor reservations are applied before the consist-level long-consist flag, so a scenario on a reserved corridor cannot enter training through another route.

Split	Scenarios	Basis
`train`	4,043	corridors `route_line0`, `route_line3`, `route_line4`
`val`	517	random 10% of eligible scenarios
`test_id`	496	random 10% of eligible scenarios
`ood_size`	955	consists flagged as long in the consist library
`ood_grade`	1,999	`route_line2`, selected as the steepest corridor
`ood_corridor`	1,990	`route_line5`, held out entirely
`control_eval`	0	not buildable with five corridors

Table 7.8: Splits of each corpus.

Comparing the two corpora shows that curve resistance cannot be left out. Over 500 scenarios compared in full, the largest velocity difference between the two versions of a scenario is 0.37 m/s at the median and 1.56 m/s at the 90th percentile. The median alone is seven times the tolerance the integrator was held to. The mechanism is accumulation, not sensitivity: a force of about 800 N on a 100 t car is an acceleration of 0.008 m/s^2, which over 600 s amounts to 4.8 m/s. The quantity the safety constraint uses moves much less. Peak coupler force changes by 0.5% at the median and 4.6% at the 90th percentile. `data/v2`, with the linear law of eq. (4.3), is therefore the training corpus.

7.6 Validation

A separate program checks each finished build in three ways, and both corpora pass. The first is consistency of the manifests: the index and the split lists describe the same scenarios, splits do not overlap, every referenced file exists, and the normalization statistics come from the training split and contain no zero divisors. The third is a set of distribution summaries (speed-limit violations, reversals, consist sizes and force ranges), so that a build whose inputs went wrong is visible without opening individual files.

The second check is the most important. The right-hand side is rebuilt from nothing but the contents of the scenario file and its route file, in a separate implementation that shares no code with the simulator, and compared with the rate of change implied by the stored trajectory. A bug shared by the simulator and the checker therefore cannot hide, and a missing constant or a reordered column in the stored data would appear as a mismatch.

The rate of change implied by the trajectory is estimated by a central difference on the 0.25 s output grid, and that estimate carries its own error. To show that the remaining mismatch is this error and not a defect in the data, one scenario was re-sampled at progressively finer output spacing (table 7.9). Each halving of the spacing reduces the mismatch about fourfold, the behavior expected of a central difference, so the mismatch comes from the estimate rather than from the stored trajectory.

This has a consequence for anything trained on the data. At the stored spacing of 0.25 s, the error of a finite-difference estimate of the rate of change is as large as the quantities

Output spacing (s)	Position rate (m/s)	Velocity rate (m/s^2)
0.25	2.1×10^{-2}	1.9×10^{-1}
0.10	4.5×10^{-3}	5.6×10^{-2}
0.05	1.1×10^{-3}	1.6×10^{-2}
0.02	1.9×10^{-4}	2.5×10^{-3}
0.01	4.6×10^{-5}	6.1×10^{-4}

Table 7.9: Maximum mismatch between the recomputed right-hand side and a central difference of the stored trajectory, for the position and velocity rates of one scenario ($N = 31$, cruise) re-sampled at finer output spacing.

a model would be asked to learn from it. Where a rate of change is needed, it should be recomputed from the stored arrays using the right-hand side, which the stored constants make possible, rather than estimated from differences between stored states.

Two defects were found and fixed while building the corpora. Both matter beyond the code.

Builds were not reproducible from their own seed. The regime generator seeded its random number generator from Python's built-in hash of the regime name, and Python randomizes string hashes in each new process. The generator was therefore consistent within one run and different in the next, although it was documented as deterministic for a given seed. The existing test could not catch this, because it called the generator twice in the same process, where the hash does not change. The seed now comes from a checksum that is stable across processes, and a new test runs the generator in separate processes with different hash seeds; it was confirmed to fail on the old code before being kept. Both corpora were built after the fix, and the recorded seed reproduces them.

The validator ignored the curvature law. The checker recomputed the right-hand side with the original curvature law regardless of the law used to build the corpus. For the linear law this is wrong by a factor of about 33 at 15 m/s, but the deliberately loose tolerance of the finite-difference comparison absorbed it, and the first build of `data/v2` passed validation without its curvature term having been checked at all. The checker now reads the law from the file. The combination of a generous tolerance and a hard-coded assumption is what allowed the error through.

7.7 Known limitations of the corpora

Overspeed. The simulator does not enforce the speed limit. The limit is information for a controller, not a physical force, so nothing stops a coasting consist from accelerating down a long grade for the whole of a 600 s run. In `data/v2`, 72.9% of scenarios never exceed the limit, 17.8% exceed it by more than 5 m/s, 2.4% by more than 20 m/s, and the worst reaches 67.8 m/s. A related 22% of scenarios roll backward faster than 1 m/s, largely because standing starts need track flatter than 0.5% and the corridors often have none. These scenarios are recorded rather than removed: the index stores each scenario's maximum excess over the limit and fraction of time over it, so they can be filtered at training

time, and keeping only scenarios within 5 m/s of the limit retains about 81% of the corpus. Whether such a scenario is bad data or a useful hard case is a modelling decision, not a question of how the data is stored.

The corridor set. Both corpora are built on the five corridors of chapter 5, with a 200 m smoothing window and a 4% grade limit. Five corridors is a small set for claims about unseen routes, and the `ood_grade` split rests on a single corridor, `route_line2`. Section 10.1 returns to that corridor and to what it means for results on that split.

No control-evaluation split. The build supports all seven splits, but with five corridors each held-out corridor costs about a fifth of the corpus, and reserving a third would leave only two corridors for training. The control-evaluation split is therefore empty. Populating it, and replacing `ood_grade` with a corridor that is genuinely steep, both require a wider corridor set, which section 13.2 builds.

No arrival time. An open-loop scenario has no destination, so the arrival-time summary is undefined throughout and the distance travelled is stored in its place. Arrival time becomes meaningful in the control scenarios, which do have one.

A rebuild on a wider, corrected corridor set is planned. Results reported in later chapters are measured on `data/v2` as described here, and would need to be re-run on a rebuilt corpus rather than carried over.

7.8 Chapter summary

This chapter replaced the NumPy reference simulator, for the purpose of data generation, with a batched GPU integrator: the same right-hand side, evaluated over whole batches of scenarios with fixed-step fourth-order Runge–Kutta and each step replayed as a single recorded GPU operation. It agrees with the reference within 0.05 m/s in velocity and 5% in peak coupler force, its cost does not grow with consist size, and it is more than a thousand times faster than the reference. Double precision proved almost free and was used for integration.

With it, two corpora of 10,000 scenarios each were built in about 25 minutes apiece, from regime-based driving commands and randomized starting conditions, stored in the node and edge layout of chapter 3, and checked by an independent re-implementation of the physics. Their known limitations, chiefly the overspeed population, the small corridor set, and the empty control-evaluation split, are stated here so that later results can be read against them.

Chapter 8

Surrogate Architecture

Chapter 3 stated the surrogate as a learned step $\mathcal{S}_\theta$ on the tensor state (eq. (3.4)), and chapter 7 produced the data it is trained on. This chapter builds $\mathcal{S}_\theta$. It answers two questions in turn. The first is what the neural network should be asked to predict: which parts of the next state it produces, and which are computed exactly from known physics. The second is how the network itself is built, so that one set of learned weights can advance a train of any length.

Both answers are driven by measurement rather than convention, and the measurements are given where each decision is made. They were taken on the training corpus `data/v2`. The decisions rest on properties of the physics and of the step size rather than on the particular corpus, so the expectation is that a different corpus would shift the reported numbers in magnitude rather than in direction. That expectation is tested in chapter 14, where the same measurements are repeated on a corpus generated with revised physics.

8.1 What a neural network contributes here

A neural network is a function with many adjustable parameters, chosen so that the function fits a set of examples. The basic building block used throughout this chapter is the multilayer perceptron (MLP). The version used here has two layers:

$$\phi(\mathbf{z}) = W_2\,\sigma(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2, \tag{8.1}$$

where $\mathbf{z}$ is the input vector, W_1 and W_2 are weight matrices, $\mathbf{b}_1$ and $\mathbf{b}_2$ are bias vectors, and σ is a fixed nonlinear function applied to each element. Without σ the two layers would collapse into a single linear map. With it, an MLP of sufficient width can approximate any continuous function on a bounded domain to arbitrary accuracy [54]. The weights and biases are the parameters θ. They start at random values and are adjusted by gradient descent: the error of the network's output on training examples is measured by a loss function, the gradient of that loss with respect to every parameter is computed by backpropagation, and the parameters are moved a small distance against it, many thousands of times.

Expressiveness is not the difficulty. An MLP can represent almost anything, which also means it assumes almost nothing, and what it has not been told it must learn from data. How well a trained network behaves on inputs unlike its training examples depends mostly

on what its structure assumes about the problem, its *inductive bias* [4]. The design problem in this chapter is therefore not how large to make the network but what to build into it. Two kinds of structure are used. The first concerns *what* the network is asked for: every quantity that follows exactly from known relations is computed rather than learned (sections 8.2 to 8.4). The second concerns *how* information may flow: the network is arranged on the chain of vehicles and couplers, so that its structure matches the mechanics (section 8.5).

A plain MLP applied to a whole train shows why the second kind matters. To predict the next state of a 150-car consist, it would take every vehicle's and coupler's features as one long input vector, about 3,900 numbers with the features used here. Its first layer alone would then need about half a million weights, every one tied to a particular position in that vector. The network would have no way to know that vehicle 3 and vehicle 40 obey the same force balance; it would have to learn the same physics separately at every position. And it could not be applied to a train of any other length at all, since the input vector would be the wrong size. The architecture of section 8.5 removes both problems by applying one small set of weights to every vehicle and every coupler.

8.2 The unit of prediction: one fixed step

There are two broad ways to use a neural network to simulate a dynamical system. The network can replace or correct the right-hand side of the differential equation, $\mathrm{d}\mathbf{s}/\mathrm{d}t$, with an ordinary solver integrating it. Or it can replace a whole step of the solution, mapping the state at one time directly to the state a fixed interval later. The first is the neural ordinary differential equation (neural ODE) of [37], of which a learned correction to a known right-hand side is a special case. The second is the graph network simulator of [5]. This thesis uses the second. An earlier statement of this work proposed the first [7], and the reasons it was not adopted are properties of this problem worth stating.

A correction to the right-hand side would be zero. A physics-residual model learns $\mathrm{d}\mathbf{s}/\mathrm{d}t = \mathcal{F}_n(\mathbf{s}) + \Delta_\theta(\mathbf{s})$, where $\mathcal{F}_n$ is the known right-hand side of eq. (3.2) and Δ_θ is a learned correction. The correction is trained to supply the difference between the true rate of change and $\mathcal{F}_n$. But the training data were generated by integrating $\mathcal{F}_n$ itself (chapter 7), so that difference is exactly zero at every point: not small, but identically zero, apart from rounding in storage. The scope of this argument matters. It holds because the model and the data generator are the same model. A residual correction is the right design when the known physics *approximates* data from somewhere else, such as field measurements or a finer simulator, and it would become the right design here if the surrogate were later calibrated against measured train data. Against its own generator it has nothing to learn.

A learned derivative would need several network calls per step. A network that learns the derivative must be integrated with a numerical solver, and the solver's step is limited by the stiffness of the couplers. As section 8.3 shows, an explicit step is stable here only at $h \leq 0.05\,\mathrm{s}$, so each stored 0.25 s transition would need at least five network evaluations. Four of the five intermediate states fall between stored samples, where no data exist to supervise them. Stiffness is a documented obstacle to training neural ODEs in

general [38], and in a coupled vehicle chain it is the normal condition rather than an edge case. A correction model would also evaluate $\mathcal{F}_n$ at every solver stage in addition to the network, so it could not be faster than the simulator it replaces.

A learned step replaces fifty evaluations with one. The reference integrator takes 12.5 fixed RK4 steps between stored samples, fifty evaluations of the right-hand side (section 7.2). A network that predicts the whole 0.25 s step replaces all fifty with a single evaluation. This is the source of the surrogate's speed, and because it follows from the integrator's own configuration it does not depend on a hardware comparison. A learned step is not bound by the stability limit of the explicit scheme it replaces [6], which is what allows it to take a step the reference integrator could not.

One caveat should be stated plainly. The rule that turns the network's output into the next state (section 8.4) is itself a fixed integration step: velocity is advanced by the predicted acceleration, and positions by the trapezoid rule. The claim is not that integration has been removed. It is that an adaptive solver calling the network many times has been replaced by a fixed update calling it once, and that the network's job has changed from supplying an instantaneous derivative to supplying the *mean* derivative over the whole step, which is what the fixed update needs to be exact.

8.3 What to anchor the step on

A step model need not predict the next state from nothing. It can start from a cheap estimate, an *anchor*, and predict only the remainder:

$$\mathbf{s}_{k+1} = \mathbf{s}_{k+1}^{\text{anchor}} + (\text{learned remainder}). \tag{8.2}$$

A good anchor makes the remainder small, and a small remainder is an easier learning target. The size of the remainder is also exactly the error of a model that outputs zero, so it is the baseline any trained network must beat. The natural anchor is one step of the physics itself. Table 8.1 and fig. 8.1(a) show what that does for velocity.

Anchor for velocity	Median	90th pct.	99th pct.
None: predict the change directly	**0.018**	**0.091**	**0.223**
Explicit Euler step, $h = 0.25$ s	0.111	0.219	0.438
RK4 step with coupler force frozen	0.102	0.197	0.391
RK4 step, $h = 0.25$ s	1.187	7.527	27.37

Table 8.1: Size of the remaining learning target for velocity under each anchor: the largest velocity change over the consist that the network would still have to supply, in m/s, over 480 transitions from 40 training scenarios of `data/v2`.

Every physics anchor makes the velocity target larger, by a factor of about six at the median for the best of them. A single RK4 step at the full 0.25 s diverges outright. The reason is stiffness. Each coupler is a stiff spring between two heavy vehicles, and the pair

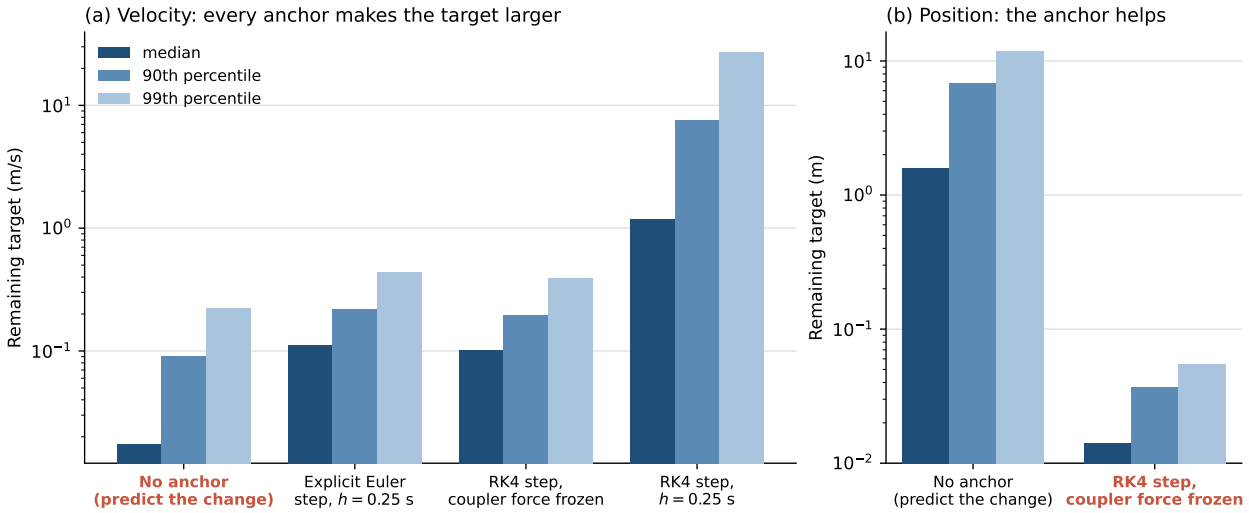


Figure 8.1: The size of the remaining learning target under each anchor, on a logarithmic scale. (a) For velocity, every physics anchor leaves a larger target than predicting the change directly. (b) For position, anchoring on the physics reduces the target more than a hundredfold. The best choice in each panel is marked in red. Data from tables 8.1 and 8.3.

has a natural frequency $\omega = \sqrt{k\,(1/m_j + 1/m_{j+1})}$, where k is the coupler stiffness and m_j, m_{j+1} the two masses. Table 8.2 gives its distribution over the sampled consists.

Coupler branch	Median ω	99th pct. ω	Shortest period
Buff (compression)	17.4 rad/s	29.8 rad/s	0.211 s
Draft (tension)	15.1 rad/s	25.8 rad/s	0.243 s

Table 8.2: Natural frequency of the two-vehicle coupler mode, over 60 sampled scenarios.

An explicit integrator is stable only while ωh stays below a limit set by the method, about 2.8 for RK4. At $h = 0.25$ s, ωh reaches 7.45. The instability disappears only at $h \leq 0.05$ s, where the 99th percentile of ωh is 1.49, and that is the limit that forces a neural ODE to five or more calls per stored step. Nor is this an occasional regime: couplers are outside their slack, and therefore stiff, 91.1% of the time. Freezing the coupler force over the step breaks the feedback loop that causes divergence, but the result is still six times worse than no anchor, because a force held constant over 250 ms is a poor estimate of its own average.

It is worth separating this from a related concern. A shortest period of 0.21 s is below the 0.25 s sampling interval, which would suggest the stored data miss the peaks of coupler force. They do not. Re-integrating six high-force scenarios at a 2 ms step and comparing peak force against the same trace sampled at 0.25 s, the peak was missed by 0.4% at the median and 3.7% at worst. The couplers are heavily damped, with a damping ratio of about 0.37, and the forces are driven slowly by the bulk motion of the train rather than ringing. The *trajectory* is smooth and well sampled at 0.25 s; it is the *explicit integrator* that is unstable there. Stability depends on ωh, not on how smooth the solution is.

For position the picture reverses (table 8.3 and fig. 8.1(b)). The relation $\dot{x} = v$ holds

exactly, and anchoring on it leaves a target 113 times smaller at the median. The stiffness lives entirely in the velocity equation.

Anchor for position	Median	90th pct.	99th pct.
None: predict the change directly	1.582	6.796	11.73
RK4 step with coupler force frozen	**0.014**	**0.037**	**0.055**

Table 8.3: Size of the remaining learning target for position under each anchor, in metres, on the same transitions as table 8.1.

The conclusion is that the anchor must be chosen per channel, not globally. The next section does that.

8.4 Channel by channel

Each vehicle carries four dynamic channels: position x_i, velocity v_i, and the brake and traction lag states $z_{\mathrm{brk},i}$ and $z_{\mathrm{trac},i}$. Each coupler carries its stretch δ_j, the change in the gap between vehicles j and $j+1$ relative to its nominal length. Figure 8.2 summarizes how each is advanced.

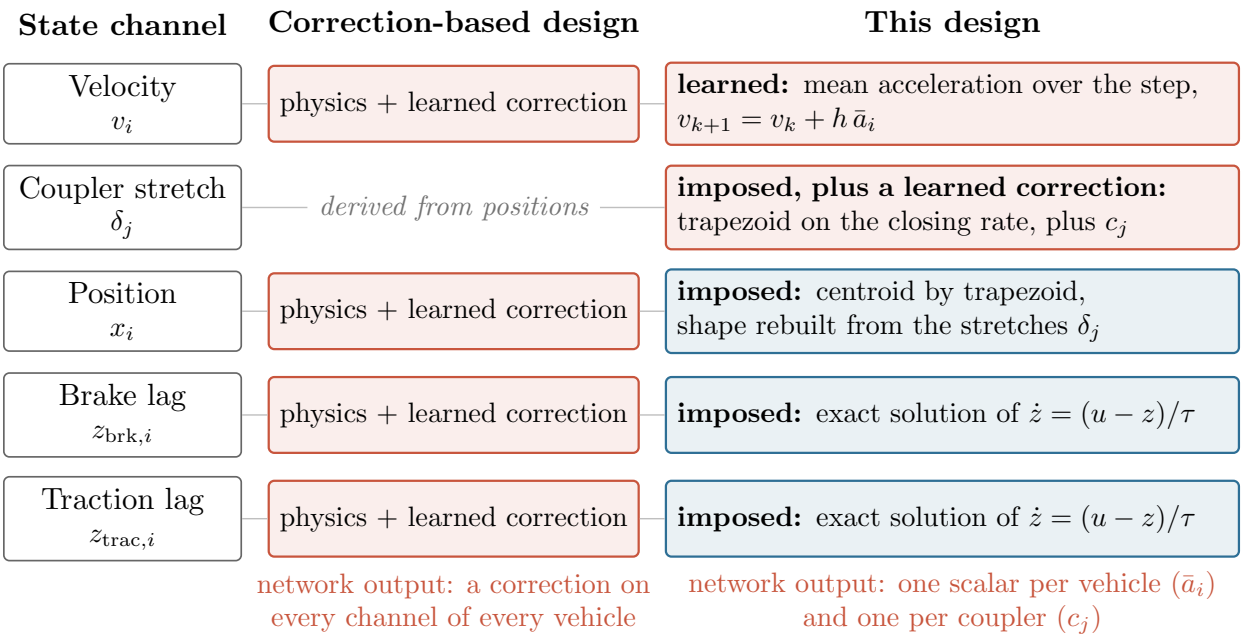


Figure 8.2: How each state channel is advanced over one step. A design that corrects the physics would learn a correction on every channel (centre). Here two channels are imposed exactly, position is imposed apart from the coupler shape, and the network's output narrows to one scalar per vehicle and one per coupler (right).

8.4.1 Actuator lags: imposed exactly

The brake and traction lag states obey $\dot{z} = (u - z)/\tau$, where u is the commanded value and τ the lag time constant (chapter 4). The command is known in advance and varies linearly between stored samples, so the equation can be solved exactly across the step. With $r = h/\tau$,

$$z_{k+1} = e^{-r} z_k + u_k \left(1 - e^{-r}\right) + (u_{k+1} - u_k) \left[1 - \frac{\tau}{h} \left(1 - e^{-r}\right)\right]. \tag{8.3}$$

This is not an approximation of the data generator; it is the generator's own actuator model, solved. Checked against 25 scenarios, it reproduces the stored lag states to within 3.6×10^{-3} N on a scale of 9.2×10^4 N, which is the rounding of single-precision storage, and a hundred times more accurately than an Euler step of the same equation. Giving these channels to the network would ask it to learn a first-order lag it can simply be told.

8.4.2 Velocity: the one learned quantity

The network's node output is the mean acceleration of each vehicle over the step, and velocity follows by definition:

$$\bar{a}_i = \frac{v_{i,k+1} - v_{i,k}}{h}, \qquad v_{i,k+1} = v_{i,k} + h\,\bar{a}_i. \tag{8.4}$$

The target is the *mean* acceleration over the step, not the instantaneous acceleration at its start. The two differ exactly where the dynamics are stiff, which is why no physics anchor helps (section 8.3).

8.4.3 Position: imposed, except for the shape of the train

With the velocities at both ends of the step known, the trapezoid rule $x_{k+1} = x_k + \frac{h}{2}(v_k + v_{k+1})$ is the natural way to advance position. Table 8.4 measures how well it does when given the *true* velocities, split into the motion of the train as a whole and its shape.

Quantity	Trapezoid error	No-change error	Ratio
Absolute position x_i	29.6 mm	3.67 m	124
Coupler stretch δ_j	42.8 mm	54.7 mm	1.3
Bulk motion (consist centroid)	0.7 mm	3.67 m	5,219
Shape about the centroid	29.2 mm	87.5 mm	3.0

Table 8.4: Largest per-step error of the trapezoid rule computed from true velocities, against predicting no change, over 30 training scenarios.

The trapezoid carries the bulk motion of the train almost exactly, to 0.7 mm per step. It fails on the shape. The error in coupler stretch, 43 mm, is 2.1 times the coupler's 20 mm half-slack, so a scheme that derived stretch from trapezoid positions would put couplers on the wrong side of their slack at every step. The cause is again stiffness: within one step, adjacent vehicles complete close to a full oscillation against each other, and the endpoints of an oscillation do not determine its integral. The kinematic relation is therefore kept, and the network supplies a small correction for the part it cannot recover.

Where that correction lives turned out to matter. Positions are stored in single precision and run to about 15 km, where the spacing between representable numbers is 0.98 mm. The correction has a standard deviation of 0.75 mm, so as a per-vehicle position target it is mostly rounding error: the ratio of signal to quantization is 0.77. A first model trained on that target learned nothing from it, which was the correct behaviour. The same physical quantity expressed as a coupler stretch, on a scale of about 66 mm, has a signal-to-quantization ratio of about 117,700. The correction is therefore predicted per coupler, and the network's edge output c_j is added to a trapezoid update of the stretch:

$$\delta_{j,k+1} = \delta_{j,k} + \frac{h}{2}\left(\dot\delta_{j,k} + \dot\delta_{j,k+1}\right) + c_j, \qquad \dot\delta_j = v_j - v_{j+1}. \tag{8.5}$$

Absolute positions are rebuilt from the two parts the analysis separated: the centroid of the consist advanced by the trapezoid rule, which is accurate to 0.7 mm, and the shape of the train given by the new stretches, walking back from the lead vehicle through $\delta_j + L_{0,j}$, where $L_{0,j}$ is the nominal coupler spacing. Absolute position is then needed only to look up grade, curvature and speed limit, which vary over metres, so its millimetre rounding does no harm there, while the geometry the coupler law reads is carried at full precision. This also explains a storage decision of section 7.4: the coupler stretch is stored even though it can in principle be computed from positions, and at the precision the model needs it cannot.

8.4.4 What the division achieves

Two of the four vehicle channels leave the network entirely, and a third is imposed apart from a small shape correction. What the network produces narrows from a correction on every channel of every vehicle to one scalar per vehicle, $\bar a_i$, and one per coupler, c_j. The model imposes more physics than a correction-based design would, not less; the physics is built into the structure of the step rather than added to the network's output. The complete update is

$$\begin{aligned}
(\bar a_i,\ c_j) &= \mathrm{GNN}_\theta\big(\mathbf{s}_k,\ \boldsymbol{\delta}_k,\ u_k,\ u_{k+1},\ \mathcal{R}\big),\\
v_{i,k+1} &= v_{i,k} + h\,\bar a_i,\\
\delta_{j,k+1} &= \delta_{j,k} + \tfrac{h}{2}\big(\dot\delta_{j,k} + \dot\delta_{j,k+1}\big) + c_j,\\
\bar x_{k+1} &= \bar x_k + \tfrac{h}{2}\big(\bar v_k + \bar v_{k+1}\big), \quad x_{i,k+1} \text{ from } \bar x_{k+1} \text{ and } \boldsymbol{\delta}_{k+1},\\
z_{k+1} &= \text{eq. (8.3)},
\end{aligned} \tag{8.6}$$

where $\bar x$ and $\bar v$ are the mean position and velocity over the consist. The rest of the chapter describes GNN_θ.

8.5 The network

The network follows the encode–process–decode pattern of graph network simulators [5]: each vehicle's and coupler's features are first mapped into a larger internal representation, information is then exchanged along the chain for a fixed number of rounds, and finally each vehicle's and coupler's representation is mapped to its output. Figure 8.3 shows the whole pipeline, including the reconstruction of eq. (8.6) and the loop by which a rollout feeds each predicted state back in as the next input.

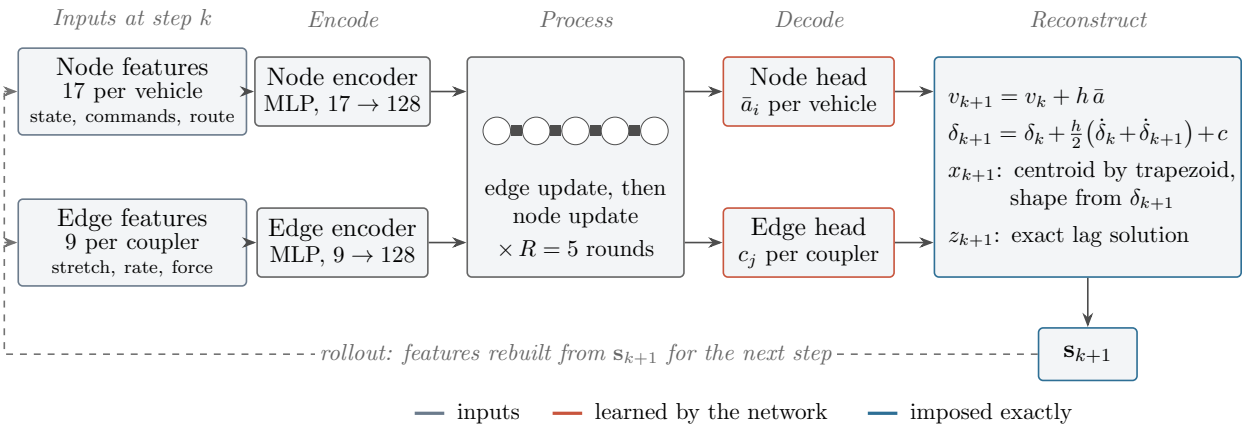


Figure 8.3: The surrogate as a whole. Node and edge features are encoded into 128-dimensional vectors, updated by five rounds of message passing along the chain, and decoded into the two learned outputs. The reconstruction step turns those into the next state exactly as in eq. (8.6). In a rollout the new state is fed back as the next input.

8.5.1 The graph and its features

The graph is the consist itself: one node per vehicle and one edge per coupler, joined in a chain. Unlike the particle and mesh systems for which graph network simulators were developed, this graph never changes during a run and needs no neighbour search. Its connectivity is given by the consist manifest.

Each node carries 17 input features: the vehicle's velocity and two lag states; the traction and brake commands at both the start and the end of the step (four values); the vehicle's seven constant parameters (mass, three Davis coefficients, whether it is powered, and its maximum traction and brake forces); and the grade, curvature and speed limit of the route sampled at the vehicle's current position. Absolute position is deliberately excluded. A chainage of 12 km means nothing that transfers to another route, and what the physics actually reads at that position, the route fields, is supplied directly.

Each edge carries 9 input features: the coupler's stretch, its rate of change, and its force, followed by six constant parameters (nominal spacing, slack half-width, and stiffness and damping in draft and in buff). The coupler force is computed from the stretch and rate with the known coupler law of chapter 4. This hands the network the slack explicitly instead of making it rediscover the discontinuity from raw positions. It is not information leakage, because at inference the force is computed in exactly the same way from the stretch the rollout carries. It is one more place where known physics enters as structure: the network is told the instantaneous force and must learn what it averages to over the step.

All features are standardized, each shifted by its mean and divided by its standard deviation over the training split, so that quantities measured in newtons, kilograms and metres per second enter on comparable scales. The two targets are standardized the same way.

8.5.2 Encoding

Two MLPs of the form of eq. (8.1), one for nodes and one for edges, map the input features into 128-dimensional vectors, written n_i for vehicle i and e_j for coupler j. These are the network's internal representation of each vehicle and coupler. They have no fixed physical meaning; training decides what each dimension encodes. The encoding lets the network describe the state in whatever combination of the inputs turns out to be useful, rather than in the units they arrive in.

8.5.3 Message passing

The core of the network is message passing [31, 32]. In each of R rounds, every coupler first updates its vector from its own vector and those of the two vehicles it joins, and every vehicle then updates its vector from its own and those of the two couplers attached to it:

$$e_j \leftarrow e_j + \mathrm{LN}\big(\phi_e^{(r)}\big[\, e_j,\ n_j,\ n_{j+1} \,\big]\big), \tag{8.7}$$

$$n_i \leftarrow n_i + \mathrm{LN}\big(\phi_n^{(r)}\big[\, n_i,\ e_{i-1},\ e_i \,\big]\big), \tag{8.8}$$

where square brackets denote concatenation of vectors, $\phi_e^{(r)}$ and $\phi_n^{(r)}$ are MLPs, and LN is layer normalization. Figure 8.4 shows one round.

The two updates mirror the physics directly. The edge update has the shape of the coupler law, $F_j = \phi_j(\delta_j, \dot{\delta}_j)$, which reads the two vehicles a coupler joins. The node update has the shape of the force balance $m_i \dot{v}_i = F_{\mathrm{ext},i} + F_{i-1} - F_i$, which combines the two coupler forces acting on a vehicle. Where the reference simulator evaluates a fixed spring law on each coupler and sums fixed forces at each vehicle, the network evaluates learned functions in the same places.

Several design choices within this pattern are specific to a train.

The same weights for every vehicle and coupler. $\phi_e^{(r)}$ is applied to every coupler and $\phi_n^{(r)}$ to every vehicle, with the same weights. This is what makes the model independent of consist length: a 12-car and a 140-car train exercise exactly the same parameters, the second simply applies them more times. It is also the network's version of the fact that every vehicle obeys the same force balance, the structural assumption a plain MLP could not express (section 8.1). Each round has its own pair of MLPs, so the five rounds can do different jobs, but within a round the weights are shared along the whole train.

Ordered messages, not a sum. General message-passing networks combine the messages a node receives by summing or averaging them, since in a general graph a node's neighbours have no natural order. A vehicle's neighbours do: one coupler is ahead of it and one behind. The coupler law is also asymmetric, with different stiffness in draft and buff, so a vehicle pulled from ahead is in a different state from one pushed from behind, and a sum would discard that distinction. The two messages are therefore concatenated in a fixed order, front then rear, as eq. (8.8) shows.

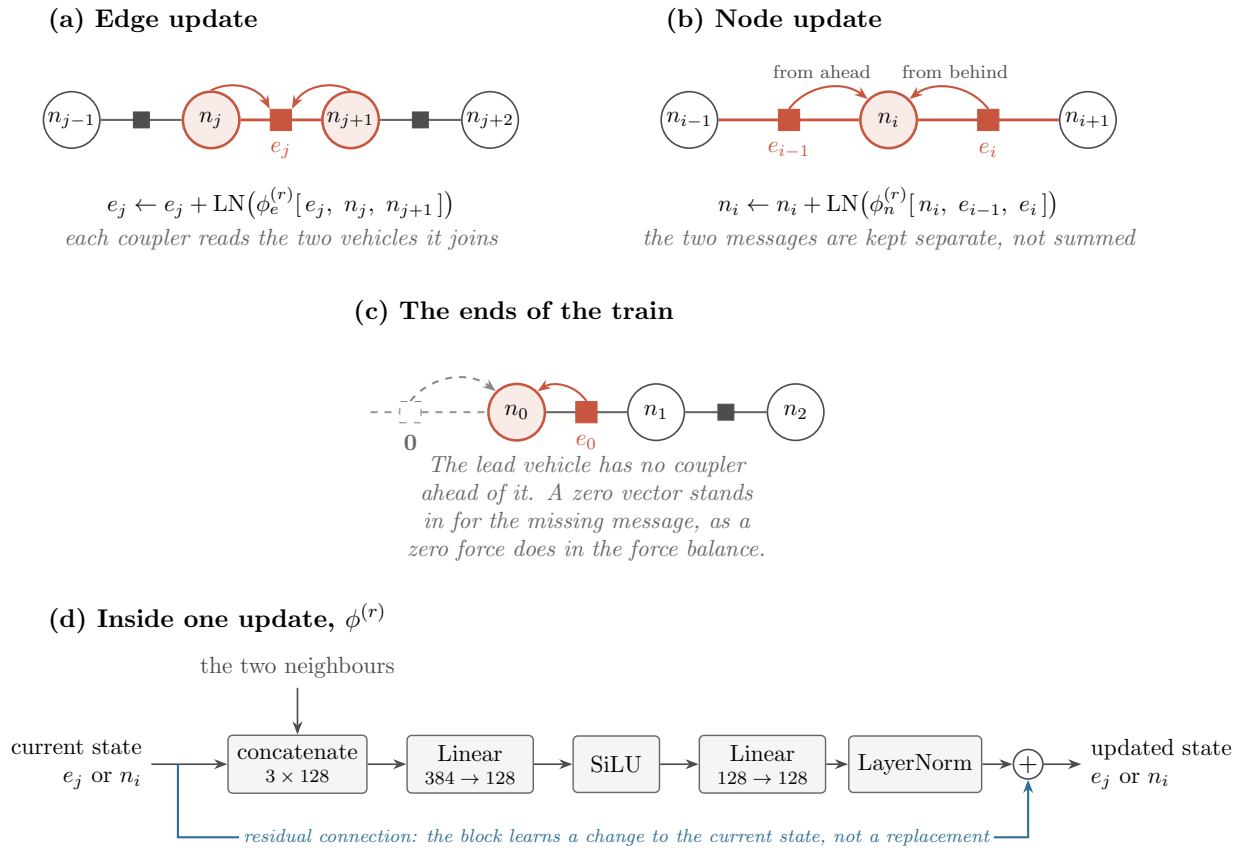


Figure 8.4: One round of message passing. (a) Each coupler updates from the two vehicles it joins. (b) Each vehicle then updates from the couplers ahead of and behind it, kept as separate inputs. (c) At the ends of the train a zero vector stands in for the missing coupler. (d) Each update is a two-layer MLP followed by layer normalization, added to the current vector through a residual connection.

The ends of the train. The lead vehicle has no coupler ahead of it and the last has none behind. A zero vector stands in for the missing coupler, which matches the reference simulator's own convention that the end forces F_{-1} and F_{N-1} are zero (fig. 8.4(c)).

Residual connections and layer normalization. Each update adds a learned change to the current vector rather than replacing it. This residual form [55] makes the identity, leaving a vector unchanged, the easiest thing for a round to do, so rounds that have nothing useful to add do no harm, and it keeps gradients well behaved through the stack of rounds during training. It also matches the nature of the task: each round refines a description of the train rather than rebuilding it. Layer normalization [56] rescales each update to a standard range using only that vector's own elements. Unlike batch normalization, it does not depend on the other examples in a batch or on the number of vehicles, so the network behaves the same on a short train as on a long one.

A smooth activation. The nonlinearity σ is the sigmoid-weighted linear unit, $\sigma(z) = z/(1 + e^{-z})$, often called SiLU [57]. Unlike the common rectified linear unit, which has a corner at zero, it is smooth everywhere. The surrogate will later be differentiated with respect to the control plan to choose driving commands, and a smooth network gives smooth gradients for that purpose.

8.5.4 How many rounds: depth from the physics

In one round, information moves one vehicle along the chain. After r rounds a vehicle's vector depends on vehicles up to r couplers away, and on nothing further (fig. 8.5). The number of rounds is therefore the distance over which the network can relate cause and effect within one step. That distance has a physical counterpart: a disturbance travels along a train as a force wave through the couplers, at a speed set by coupler stiffness and vehicle mass. Measured from the consist parameters, the wave travels at 167 to 193 m/s (median by coupler branch), against a car pitch of about 17 m. In one 0.25 s step it therefore reaches about three vehicles in each direction at the median and five at the 99th percentile.

Five rounds cover that reach, and the network uses $R = 5$. The depth is fixed by physics rather than tuned, a line of argument also used for graph network simulators of structural dynamics [41]. It does not grow with the length of the train: a 150-car consist needs no more rounds than a 20-car one, because within one step no vehicle can be affected by anything further than the wave travels. Effects across the whole train build up over successive steps of a rollout, just as they do in the physical system.

8.5.5 Decoding

Two further MLPs read the final vectors. The node head maps each n_i to the mean acceleration $\bar{a}_i$, and the edge head maps each e_j to the stretch correction c_j. The two outputs are different quantities in different coordinates, one per vehicle and one per coupler, so they get separate decoders reading from the shared representation that message passing has built.

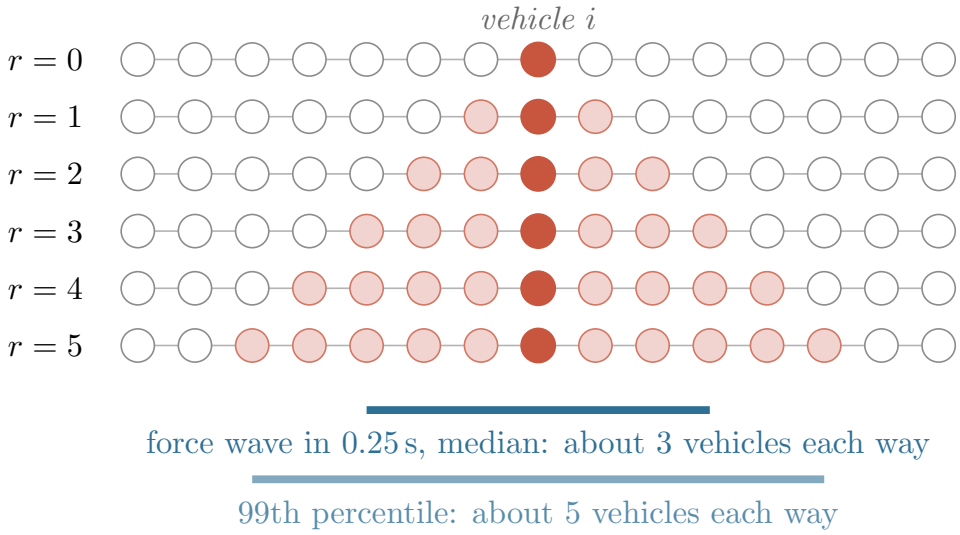


Figure 8.5: Receptive field of vehicle i after r rounds of message passing. After r rounds it has received information from vehicles up to r couplers away, and no further. A coupler force wave travels 167–193 m/s, about three vehicles each way in one 0.25 s step at the median and five at the 99th percentile over a 17 m car pitch, so five rounds cover the physical reach of one step.

8.5.6 Size, and independence from train length

Table 8.5 counts the parameters. The network has 730,370 in total, and none of the counts depends on the number of vehicles. The same trained weights apply to a consist of any length, and the computation grows only linearly with it, one application of each MLP per vehicle or coupler per round. For comparison, the first layer alone of the plain MLP of section 8.1 would need about half a million weights, and would work only for trains of exactly 150 cars.

Component	Structure	Parameters
Encoders	two MLPs, 17 → 128 → 128 and 9 → 128 → 128	36,608
Message passing	five rounds, each an edge and a node MLP (384 → 128 → 128) and two layer normalizations	660,480
Decoders	two MLPs, 128 → 128 → 1	33,282
Total		730,370

Table 8.5: Parameter count of the network. None of the counts depends on the number of vehicles.

8.5.7 Implementation

Because the graph is a chain, message passing needs no graph library. Every coupler reads its two vehicles by offsetting the vehicle array by one position, and every vehicle gathers its two couplers by padding the coupler array with a zero at one end or the other. This is the same pattern the GPU integrator uses to compute coupler forces (section 7.2), in the same batch-by-vehicle array layout. Training batches are drawn from one consist size at a time, so every batch is rectangular and needs no padding or masking, even though consist size ranges from 10 to 150. The model itself never sees the difference.

8.6 Training objective

The network is trained to predict its two targets, the mean acceleration of eq. (8.4) and the stretch correction defined by eq. (8.5), from a stored state. The loss is the sum of the mean squared errors of the two standardized outputs,

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i}\left(\hat{a}_i - \bar{a}_i\right)^2 + \frac{1}{N-1}\sum_{j}\left(\hat{c}_j - c_j\right)^2, \tag{8.9}$$

with hats denoting predictions. Because both targets are standardized, this weights them equally relative to their own spread rather than by their physical magnitudes, which differ by orders of magnitude. The imposed channels contribute nothing: they are exact, so there is no error in them to fit. This is the concrete form of the objective of eq. (3.15). Parameters are updated with the Adam optimizer [58], with a learning rate decaying from 10^{-3} along a cosine schedule and gradients clipped to a norm of 1. A model trained only to predict one step from true states does not remain stable when fed its own predictions; the techniques that make it stable over long rollouts are the subject of chapter 9.

8.7 Relation to graph network simulators

The model belongs to the graph network simulator family and borrows its structure, its name, and its central idea that a learned step is advanced by a fixed update. It departs from the original in ways that follow from the problem. Table 8.6 sets the two side by side.

8.8 Chapter summary

The surrogate predicts one fixed 0.25 s step, replacing fifty evaluations of the reference right-hand side with one network evaluation. A learned correction to the right-hand side was rejected because, against data generated by that right-hand side, the correction is identically zero, and a learned derivative because coupler stiffness would force at least five network calls per step between unsupervised states. Measurement showed that no physics anchor helps velocity, since the couplers are stiff at this step size, while the kinematic anchor helps position a hundredfold. The step is therefore built channel by channel: lag states exact, velocity learned as a mean acceleration, position by the trapezoid rule on the bulk motion

	Graph network simulator [5]	This model
Nodes and edges	particles; edges between particles within a radius	vehicles; edges are the couplers
Graph	rebuilt at every step by neighbour search	fixed chain, given by the consist
Combining messages	sum over neighbours	ordered concatenation, front and rear
Network output	acceleration per particle	mean acceleration per vehicle and a stretch correction per coupler
Update rule	fixed integrator on all channels	velocity from the output; position by trapezoid with learned shape; lag states exact
Depth	tuned	set by force-wave travel in one step

Table 8.6: The original graph network simulator and the model of this chapter.

with a learned correction to the shape of the train, predicted per coupler because only there does it survive single-precision storage.

The network that supplies those two outputs is a message-passing graph network on the consist chain: encoders into 128-dimensional vectors, five rounds of edge and node updates with residual connections and layer normalization, and one decoder for each output. Its weights are shared across every vehicle and every coupler, so its 730,370 parameters apply unchanged to a train of any length, and its depth is set by how far a force wave travels in one step rather than by tuning. The next chapter describes how it is trained to stay stable when run on its own predictions over hundreds of steps.

Chapter 9

Training and First Results

Chapter 8 built a network that advances a consist by one fixed step of 0.25 s. This chapter trains it and asks three questions of the result. Does it learn the step, so that one prediction from a true state is accurate? Does it stay accurate when it runs on its own predictions for hundreds of steps, which is how a controller will use it? And does it extrapolate, to consists longer than any it was trained on and to corridors it has not seen? The third question is the one the graph formulation was chosen to answer (section 3.1.3).

Everything here is measured on `data/v2`, the corpus of chapter 7, generated by the simulator of chapter 4 on the five corridors of chapter 5. The model is the network of chapter 8 and is called GNS-1 where it must be told apart from the revised model of Part III. The chapter closes with a list of what these results claim. Part II then examines what those claims rest on.

9.1 Experimental setup

9.1.1 Data

Table 9.1 lists the splits of `data/v2` that this chapter uses. Training and the three in-distribution splits share three corridors and consists of 10 to 80 vehicles. The three held-out splits each change one thing. `ood_size` keeps the training corridors but uses only consists of 120 to 150 vehicles, so there is no overlap in length with training. `ood_corridor` and `ood_grade` are whole corridors never seen in training, but they hold every consist size from 10 to 150. A score on either therefore mixes a change of corridor with a change of length. To keep the two apart, both are also scored restricted to consists of at most 80 vehicles, the trained range.

The training split is larger than fits in GPU memory alongside the model, so each run loads a random subset that fills a 5 GiB budget: about 1,950 of the 4,043 scenarios, or 2.5 million transitions. The subset is drawn independently for each random seed. Evaluation sets are subsampled the same way within a 3 GiB budget.

Split	Scenarios	Consist size	Corridors
`train`	4,043	10–80	`route_line0`, `route_line3`, `route_line4`
`val`	517	10–80	same three
`test_id`	496	10–80	same three
`ood_size`	955	120–150	same three
`ood_corridor`	1,990	10–150	`route_line5`
`ood_grade`	1,999	10–150	`route_line2`

Table 9.1: Splits of `data/v2` used in this chapter (table 7.8). No consist in the library has between 81 and 119 vehicles.

9.1.2 Batches of one consist size

Consist size varies from 10 to 150, and padding every batch to the longest consist would spend most of each batch on empty positions, since short consists dominate the corpus. Instead, scenarios are grouped by consist size, 70 groups in the training split, and each batch is drawn from a single group, chosen with probability proportional to its number of transitions. Every batch is then a rectangular array with no padding and no mask, and the network, whose weights do not depend on consist size (section 8.5), never sees the difference.

Training across all sizes at once also proved better than training on one. A development run on 42-vehicle consists alone reached 3.4 times the zero-output baseline on velocity; training on every size from 10 to 80 with the same architecture, and a third more updates, reached 7.8 times on a validation set spanning that range. The second run saw more data, 2.5 million transitions against 155,000, and more varied data, and it is the configuration used throughout.

9.1.3 Optimization

The loss is eq. (8.9). Each run takes 6,000 updates with the Adam optimizer, batches of 256 transitions, a learning rate decaying from 10^{-3} along a cosine schedule, and gradients clipped to a norm of 1. A run takes about three minutes on one RTX 4090, plus about 40 s to load the data. Every result in section 9.3 is reported over three runs with different random seeds, each with its own training subset, as a mean with the spread between seeds where it matters.

9.1.4 Metrics

Two kinds of error are reported.

One-step error is measured from a true state: the model makes one prediction, and it is compared with the stored next state. For velocity the quantity is the root-mean-square error of the predicted mean acceleration $\bar{a}_i$, in m/s^2, over every vehicle and transition. For shape it is the root-mean-square error of the stretch correction c_j, in millimetres, over every coupler and transition.

Rollout error is measured after the model has run on its own output for k steps from one true state. At each horizon the error is the largest over the consist, of $|v_i - v_i^{\text{true}}|$ for

velocity and of $|\delta_j - \delta_j^{\text{true}}|$ for stretch, and the reported figure is the median of that maximum over many starting points. The maximum over the consist is used because the quantities a controller acts on, the worst coupler force and the fastest vehicle, are maxima. The median over starts keeps a handful of extreme scenarios from setting the headline.

The shape correction needs one further remark. It is sparse: on 64% of transitions every coupler's correction is below 50 μm, and almost all of the signal lies in slack run-in and run-out events, with the largest per-transition correction reaching 4.2 mm at the 90th percentile and 23.5 mm at the 99th. A median statistic would measure only the quiet majority, where predicting exactly zero is unbeatable. An early version of the metric did exactly that and reported the model as nine times worse than zero while its root-mean-square error was 4.6 times better. Shape is therefore reported by root-mean-square error and tail percentiles, never by a median of the one-step correction.

9.1.5 Baselines

Three baselines need no training, and every model result is reported beside them.

- **Zero output** predicts no change over the step: mean acceleration zero and no stretch correction. Over 0.25 s a freight consist barely changes state, so this is a strong bar; its one-step error is the size of the learning target itself.
- **Hold** freezes the starting state for the whole of a rollout. It is the rollout counterpart of zero output.
- **Constant acceleration** repeats the previous step's true mean acceleration and stretch correction, one step or many. It uses one stored state more than the model is given, the state a step before the start, so it is slightly favoured.

One baseline is missing on purpose. A single step of the physics itself, fourth-order Runge–Kutta at 0.25 s, diverges (table 8.1), so there is no physics-only model to compare with at this step size. The residual MLP and the transformer ablation named in chapter 1 were not run in Part I and are reported in chapter 15.

9.2 Keeping a rollout stable

A model trained only to predict one step from true states sees, during training, only states the simulator produced. In a rollout it is fed its own output, and its small errors move it into states it has never seen, where it predicts worse and moves further off. This is the standard failure of learned step models (section 2.4.3), and the first model trained here showed it plainly (table 9.2). Within 20 steps, five seconds of simulated time, its velocity error exceeded that of simply freezing the state, and by 50 steps it had converged to that baseline. Its stretch error at 20 steps, 40.4 mm, was more than five times that of holding the state.

Steps	Time	No noise	With noise	Hold
1	0.25 s	0.0052	0.0044	0.0155
5	1.25 s	0.0517	0.0131	0.0669
20	5.00 s	**0.2272**	0.0317	0.2002
50	12.50 s	0.4447	0.0622	0.4523

Table 9.2: Rollout velocity error, the largest over the consist in m/s (median over starts), with and without noise injection during training. Development runs of 3,000 updates, one seed, validation split. Without noise the model is worse than holding the state after about 20 steps.

9.2.1 Noise injection

The remedy introduced with graph network simulators [5] is to corrupt the inputs during training. Each training state has Gaussian noise added to every vehicle's velocity and every coupler's stretch, and the target is then recomputed from the *perturbed* state to the *true* next state. The label is therefore the correction that carries a drifted state back onto the trajectory, not the step the model would have taken from a clean one, and training spends its effort learning to correct drift. Positions are not perturbed, since absolute position enters the model only through route lookup on a 10 m grid, and the actuator states are exact and never drift. A unit test checks that a perturbed state advanced by its recomputed targets lands exactly on the true next state.

Table 9.3 sweeps the size of the velocity noise. The best value, 0.005 m/s, is close to the model's own one-step velocity error, which is the prescription of [5]: perturb by about as much as the model will drift. Stretch noise is set the same way, at 1 mm. With noise, velocity error at 20 and 50 steps falls about sevenfold, and the model beats holding the state at every horizon in both channels.

Velocity noise (m/s)	1 step	5	20	50	One-step RMSE (m/s^2)
0	0.0052	0.0517	0.2272	0.4447	0.0207
0.005	**0.0044**	**0.0131**	**0.0317**	**0.0622**	0.0216
0.01	0.0053	0.0147	0.0372	0.0709	0.0222
0.02	0.0069	0.0206	0.0529	0.0909	0.0249
0.04	0.0104	0.0326	0.0861	0.1488	0.0352

Table 9.3: Rollout velocity error in m/s at four horizons, and one-step velocity error, against the size of the injected velocity noise. Development runs of 3,000 updates, one seed. Differences under about 20% are within seed variation.

The stability is paid for in one-step accuracy. With noise the one-step velocity error rises from 0.0207 to 0.0216 m/s^2 and the one-step shape error from 0.25 to 0.32 mm: the model is slightly blunter from a clean state and far more stable on its own output. A table of one-step errors alone would favour the model trained without noise while hiding that it diverges, which is why both are reported in what follows.

9.2.2 Pushforward

Gaussian noise is a guess at what the model's errors look like. The pushforward method of [34] uses the real thing. Before each supervised step the model takes a few steps on its own output with no gradient, and only the step that follows is supervised. The perturbation the model learns to correct is then drawn from its own error distribution, and it tightens as the model improves. Because no gradient is taken through the unrolled steps, memory does not grow with their number, and each pushed step costs one extra forward pass. Pushforward is switched on after 500 plain updates, since before then the model's own steps are close to noise.

Configuration	10 steps	50	100	200	Shape, 200 (mm)	Time
noise only	0.0188	0.0581	0.0987	0.1614	12.3	57 s
push 1 + noise	0.0185	0.0531	0.0914	0.1413	10.3	68 s
push 3 + noise	0.0151	0.0424	0.0709	0.1219	10.4	90 s
push 8 + noise	0.0169	0.0425	0.0644	0.0944	14.9	135 s
push 16 + noise	0.0248	0.0501	0.0742	0.1129	9.8	224 s
push 1, no noise	0.0236	0.0837	0.1311	0.2229	25.0	69 s

Table 9.4: Rollout velocity error in m/s at four horizons, stretch error at 200 steps, and training time, for pushforward with 1 to 16 unsupervised steps. Development runs of 3,000 updates, one seed.

Table 9.4 gives two results. Pushforward alone is worse than noise alone, 0.22 against 0.16 m/s at 200 steps, so the two are complements rather than substitutes. A plausible reason is that early in training the model's own one-step error is small, so pushforward supplies much less perturbation than the fixed noise does. And the gains flatten: velocity improves up to about eight pushed steps, while the shape column moves non-monotonically, which at a single seed is variation rather than a trend. Three pushed steps, with noise, is used from here on, the cheapest point in the useful range, at 1.6 times the training time for about 25% less velocity error at 200 steps.

9.2.3 Checking the rollout itself

Rollout error is only meaningful if the rollout code is exact. A test feeds the rollout the true targets in place of the model's output and checks that it reproduces the stored trajectory over 30 steps, to within 10^{-3} m/s, the accumulation of single-precision rounding. Without that check, part of every rollout error could be an error in the harness, and there would be no way to tell.

9.3 Results

The results in this section come from three training runs with seeds 0, 1 and 2, at the settings of section 9.1 with noise injection and three pushforward steps. For each split,

about a thousand starting points are drawn across the consist-size groups, each with room for 200 steps, and the model and all three baselines are scored from the same starts. One-step errors are those of the first step from each start. Figures are means over the three seeds. Seed 0 reproduces the development measurements exactly, so training is deterministic for a given seed.

9.3.1 One step from a true state

Table 9.5 gives the one-step errors. In distribution, on `val` and `test_id`, the model predicts the mean acceleration with a root-mean-square error of 0.015 to 0.019 $\mathrm{m/s^2}$, 7.3 to 7.5 times better than predicting no change and 3.7 to 3.8 times better than constant acceleration. The shape correction is harder. Its error of 0.43 to 0.45 mm is 2.1 to 2.6 times better than zero output. This is a smaller margin than for velocity, and it is the price of the stretch noise injected during training (section 9.2).

		Mean acceleration			Stretch correction		
Split	N	RMSE ($\mathrm{m/s^2}$)	vs zero	vs const.	RMSE (mm)	vs zero	vs const.
`val`	10–80	0.0154	7.5×	3.7×	0.428	2.1×	3.5×
`test_id`	10–80	0.0186	7.3×	3.8×	0.446	2.6×	4.0×
`ood_size`	121–150	0.0216	8.5×	3.5×	0.576	2.2×	3.4×
`ood_corridor`	10–80	0.0148	9.2×	3.7×	0.405	2.5×	3.6×
`ood_corridor`	10–150	0.0239	5.2×	2.2×	0.393	2.1×	3.5×
`ood_grade`	10–80	0.0326	4.8×	3.2×	0.787	2.2×	3.5×
`ood_grade`	10–150	0.0350	4.8×	3.2×	0.825	2.3×	3.5×

Table 9.5: One-step error of GNS-1, mean over three seeds, with the factor by which it improves on zero output and on constant acceleration (the baseline's error divided by the model's).

On `ood_size` the acceleration error rises to 0.022 $\mathrm{m/s^2}$, but the gain over zero output rises too, to 8.5 times, because longer consists carry more dynamics and the zero-output target is larger. Ratios are the fair comparison across splits and absolute errors are not. On `ood_corridor` at the trained lengths the model is as accurate as in distribution. On `ood_grade` its error is roughly twice the in-distribution value at every consist size.

9.3.2 Rolled out

Table 9.6 and fig. 9.1 give the errors after the model has run on its own output. On the in-distribution splits, velocity error after 200 steps, 50 s of simulated time, is 0.065 to 0.066 m/s, 26 to 28 times smaller than that of holding the starting state; the range over seeds on `val` is 0.060 to 0.068 m/s. Stretch error after 200 steps is 4.1 to 4.7 mm, 6.8 to 7.6 times smaller than holding. Both errors grow steadily with horizon, with no sign of the divergence of table 9.2.

		Velocity (m/s)				Stretch (mm)			
Split	N	10	50	200	vs hold	10	50	200	vs hold
`val`	10–80	0.010	0.028	0.065	28×	1.3	3.0	4.7	6.8×
`test_id`	10–80	0.009	0.026	0.066	26×	1.2	2.5	4.1	7.6×
`ood_size`	121–150	0.021	0.050	0.099	18×	5.3	22.0	**40.5**	**1.2**×
`ood_corridor`	10–80	0.009	0.022	0.055	17×	1.2	2.6	3.8	10.5×
`ood_corridor`	10–150	0.011	0.028	0.066	14×	1.8	6.5	9.3	4.4×
`ood_grade`	10–80	0.024	0.060	0.139	18×	4.1	11.2	23.4	2.3×
`ood_grade`	10–150	0.028	0.065	0.148	17×	5.3	12.1	27.0	2.2×

Table 9.6: Rollout error of GNS-1 after 10, 50 and 200 steps (2.5, 12.5 and 50 s): the largest error over the consist, median over starts, mean over three seeds. "vs hold" is the factor by which the model improves on holding the starting state at 200 steps.

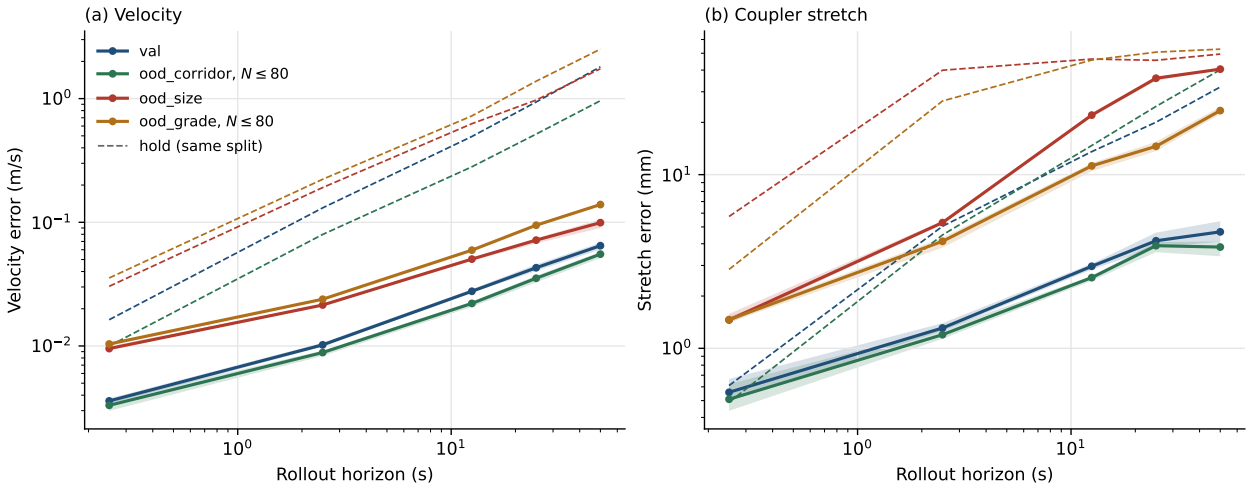


Figure 9.1: Rollout error against horizon on four splits, on logarithmic axes. Solid lines are the model (mean over three seeds, with the range over seeds shaded). Dashed lines are holding the starting state on the same split. (a) Velocity error stays more than an order of magnitude below holding on every split. (b) Stretch error does too on `val` and on the unseen corridor, but on long consists (`ood_size`) it rises to meet the holding baseline.

9.3.3 Longer consists

The test the graph formulation was chosen for is `ood_size`: consists of 121 to 150 vehicles, 1.5 to 15 times longer than anything in training, with no overlap. For velocity the result is clear. One-step error rises from 0.015 to 0.022 m/s^2 while the gain over zero output holds at 8.5 times, and after 200 steps the velocity error of 0.099 m/s (0.090 to 0.106 over seeds) is 18 times smaller than holding the state. Nothing in the model knows how many vehicles there are. Message passing applies the same weights to every vehicle and coupler, and the velocity result is that property working as intended.

For coupler shape the result is the opposite. The one-step correction is still 2.2 times better than zero output, but after 200 steps the stretch error is 40.5 mm against 49.5 mm for holding, a gain of only 1.2 times, against 6.8 times on `val`. The three seeds agree to within 0.2 mm, so this is a property of the model, not of one run. Section 9.4 examines why.

9.3.4 An unseen corridor, and a steep one

On `ood_corridor` at the trained consist lengths, the model is as accurate as in distribution in every column of tables 9.5 and 9.6, and slightly better on stretch: 3.8 mm after 200 steps, against 4.7 mm on `val`. Generalization to an unseen corridor holds here. Its evidence is one corridor, `route_line5`, whose grades are similar to the training corridors (table 5.1). Including the long consists of the same corridor raises the stretch error after 200 steps to 9.3 mm, which is the length effect of the previous subsection appearing in a different split. One seed's one-step acceleration error on the full split, 0.032 m/s^2 against 0.020 and 0.019 for the other two, comes from those long consists as well.

On `ood_grade` the errors are about twice the in-distribution values at the trained lengths: 0.139 m/s and 23 mm after 200 steps, still 18 and 2.3 times better than holding. This corridor is the steepest of the five, with an rms grade of 2.46% against 0.83 to 1.03% for the others. The model extrapolates to it, but with clearly degraded accuracy, and its stretch error is closer to the long-consist result than to the in-distribution one.

9.3.5 Against constant acceleration

Constant acceleration is a stronger baseline than its one-step root-mean-square error suggests. On the median of the largest velocity error after one step, it beats the model in distribution by about four times: 0.0009 against 0.0036 m/s on `val`. Over most 0.25 s steps a freight consist's acceleration barely changes, and repeating the last one is very nearly exact. The model's one-step error on those quiet steps sits at about the level of the noise it was trained with. Its advantage lies in the tail, in notch changes, brake applications and slack events, where constant acceleration is badly wrong, and that is why the root-mean-square comparison favours the model by 3.7 times. On long consists, where the tail is larger, the model wins even on the one-step median; on the steep corridor at the trained lengths the two are close.

Rolled out, the comparison is not close. By 10 steps the model's velocity error is three times smaller in distribution, and by 200 steps 21 times smaller. Constant acceleration repeats the last stretch correction as well, and that integrates without bound: its stretch error

reaches about 130 mm after 50 steps on `val` and about 2 m after 200. Constant acceleration is a good predictor of the next quarter-second and useless beyond a few seconds, which is the horizon a controller needs.

9.4 Coupler shape on long consists

The one place the model fails to extrapolate is coupler shape on long consists over long horizons. Because the control problem constrains coupler force on long consists over long horizons, this failure matters more than its place in a table suggests, and it was examined in detail. Three hypotheses were tested and rejected. The tests were development runs with one to three seeds and are reported as such.

9.4.1 What the error is

Table 9.7 decomposes the signed per-coupler stretch error over a 200-step rollout on one validation consist of 38 vehicles and one `ood_size` consist of 150. Three features matter. The error grows with the number of steps as $n^{0.66}$ on the long consist, faster than the $n^{0.5}$ of independent errors, so successive errors are correlated. Its mean is a quarter of its root-mean-square value. And it is spread evenly along the consist, if anything slightly lower at the rear.

	`val`, $N = 38$	`ood_size`, $N = 150$
Growth exponent (rms $\propto n^p$)	0.37	**0.66**
Bias at 200 steps	+0.15 mm (2.8% of rms)	**+3.91 mm (26%)**
Standard deviation at 200 steps	5.47 mm	14.73 mm
Mean absolute error by quarter (mm)	1.28 / 1.25 / 1.39 / 1.90	11.4 / 8.8 / 10.8 / 6.8

Table 9.7: The signed per-coupler stretch error after a 200-step rollout, on a short and a long consist. The last row gives the mean absolute error in each quarter of the consist, front to rear.

Not a question of reach. If five rounds of message passing were too few to coordinate a long consist, the error would grow towards the far end of the train. It does not, so adding rounds for long consists, the first remedy considered, would not have helped.

9.4.2 A constraint that should have helped, and did not

The stretches of a consist are not independent. Their sum is fixed by the positions of the end vehicles, which the trapezoid carries accurately (table 8.4), so the true consist-mean of the stretch corrections is tiny: a standard deviation of 0.026 mm on training data and 0.008 mm on `ood_size`, against 1.04 mm per coupler. It is tightest on long consists, exactly where the drift appeared, and the bias in table 9.7 suggested that constraining the edge head to a zero consist-mean would remove it.

Seed	`val`, constrained	`val`, free	`ood_size`, constrained	`ood_size`, free
0	10.12	5.84	41.06	40.96
1	6.48	4.42	41.49	40.41
2	5.31	4.60	35.42	36.65
Mean	7.30	**4.95**	39.32	39.34

Table 9.8: Stretch error at 200 steps in mm, with the edge head constrained to a zero consist-mean and without. Three seeds.

It did not (table 9.8). The constraint made validation error worse by about half in every seed and left `ood_size` unchanged. The mistake was to read a bias of 26% of the root-mean-square error as 26% of the problem. The root-mean-square error is $\sqrt{3.91^2 + 14.73^2} = 15.24$ mm, and removing the bias entirely takes it to 14.73 mm, a 3% improvement; the dominant term is variance, and that was visible in the decomposition before any code was written. Why the constraint actively hurts is less certain. Centring couples every coupler's gradient to every other, which plausibly costs more than the freed drift mode gains.

9.4.3 Velocity error is not the cause

Stretch integrates the difference of neighbouring velocities, so a spatially correlated component of velocity error would accumulate directly into shape error, and would also explain the correlated growth. That hypothesis was tested causally rather than by correlation: one channel was forced to the truth at every step of a rollout, and the error of the other was measured.

Rollout	`val`, $N = 38$	`ood_size`, $N = 150$
Model on its own output	1.27 mm	44.18 mm
True velocity forced every step	1.73 mm	40.07 mm

Table 9.9: Stretch error at 200 steps with and without the true velocity field forced into the rollout.

A perfect velocity field removes 9% of the shape error on the long consist, and on the short one it makes shape *worse* (table 9.9). The reverse holds too: forcing the true stretch raises the velocity error on `ood_size` from 0.083 to 0.139 m/s. The two channels' errors are anti-correlated. The model has learned a self-consistent effective dynamics in which its velocity and stretch errors partly cancel, and forcing the truth into either breaks that compensation. One general consequence follows: a test that scores one channel against a teacher-forced other understates how good the joint state is.

9.4.4 A failure of stability, not accuracy

The per-step correction error on `ood_size` is only 17% larger than on validation, yet after 200 steps the stretch error is about 35 times larger in the same comparison. The difference is in

how errors accumulate. On validation the rollout accumulates less error than a random walk of the same per-step size would, about 1.3 mm against 5.7 mm, so errors correct themselves. On `ood_size` it accumulates more, about 44 mm against 22 mm, so errors reinforce one another. The learned dynamics is a stable attractor within the training range of consist sizes and not outside it.

Two hypotheses about the model have been rejected and a third, more message passing, was ruled out by the error profile before it was tried. Without an idea aimed specifically at stability, further changes to the model were not pursued in Part I. The practical consequence is a separation of claims: velocity extrapolates to unseen consist lengths, coupler shape over long horizons does not, and a controller that needs coupler force on long consists should be given a model trained on the consist sizes it will control.

9.5 Cost of a step

Section 8.2 gave the surrogate's structural saving: one network evaluation replaces the fifty right-hand-side evaluations the reference integrator makes between two stored states. That ratio is fixed by the integrator's configuration. Wall clock is a separate question, and it was measured on the workload of table 7.5: 120 s of simulated time, 480 output steps, for consists of 130 vehicles from `ood_size`. The timing covers the whole rollout loop, including feature construction, route lookup and the state update.

B	GNS-1	Integrator, single	Integrator, double	GNS-1 per scenario
1	1.78 s	3.28 s	—	1,780 ms
64	1.68 s	3.71 s	5.03 s	26 ms
256	3.27 s	4.01 s	5.49 s	13 ms

Table 9.10: Wall clock for 120 s of simulated time at $N = 130$ on one RTX 4090: GNS-1 rolled out in plain PyTorch, against the GPU integrator replayed as a CUDA graph in single and in double precision (tables 7.4 and 7.5). The corpora were integrated in double precision.

The fifty-to-one ratio does not become a fifty-fold saving in time (table 9.10). The surrogate is 1.2 to 2.2 times faster than the integrator in single precision, and 1.7 to 3.0 times faster than the double-precision integration the corpora were built with. The reason is the one section 7.2.6 found for the integrator itself: on this GPU both are limited by the overhead of issuing each step, not by arithmetic. The integrator replays each step as one recorded CUDA graph; the surrogate, as measured, launches every operation of every step separately. Recording the surrogate's step the same way is the obvious next optimization and was not done in Part I. Against the NumPy reference simulator, which needs at least 89.2 s for the same scenario on one core, a single surrogate rollout is about 50 times faster.

The speed claim of Part I is therefore the structural one: fifty evaluations become one. Measured wall clock confirms a saving, but a modest one, and it depends more on implementation than on the formulation.

9.6 What Part I claims

Table 9.11 lists what this chapter's results claim, each with its evidence. The third column names what the claim rests on beyond the measurement itself. Chapter 10 examines those foundations and marks each claim as standing, qualified or withdrawn, and chapter 15 marks them again for the rebuilt system.

	Claim	Evidence	Rests on
C1	The network learns the 0.25 s step.	Mean acceleration RMSE 0.015–0.019 m/s^2 in distribution, 7.3–7.5× zero output, 3.7–3.8× constant acceleration (table 9.5).	The simulator as the definition of truth.
C2	With noise injection and pushforward, rollouts stay accurate over 200 steps (50 s).	0.065 m/s and 4.7 mm after 200 steps on `val`, 28× and 6.8× better than holding (table 9.6).	Development sweeps on one seed (tables 9.3 and 9.4).
C3	Velocity extrapolates to consists 1.5–15× longer than any trained on.	0.099 m/s after 200 steps on `ood_size`, 18× better than holding; one-step gain 8.5× zero output.	Consist library and couplers of the legacy simulator.
C4	Coupler shape does *not* extrapolate to long consists over long horizons.	40.5 mm against 49.5 mm for holding after 200 steps, 1.2×, consistent over seeds; diagnosed as instability (section 9.4).	The Kelvin–Voigt coupler with deadband.
C5	The model generalizes to an unseen corridor.	At trained lengths, `ood_corridor` matches in-distribution accuracy (0.055 m/s, 3.8 mm after 200 steps).	One corridor, of grades similar to training.
C6	The model extrapolates to steeper terrain, with degraded accuracy.	`ood_grade` errors about twice in distribution; 18× and 2.3× better than holding.	The grade field of `route_line2`.
C7	One network evaluation replaces fifty right-hand-side evaluations.	Integrator configuration (section 7.2); measured wall clock 1.2–3.0× faster than the GPU integrator (table 9.10).	Integrator step of 0.02 s.

Table 9.11: What Part I claims. Chapter 10 examines the third column.

All seven claims are about the surrogate's fidelity to the simulator that generated its data, not about fidelity to a real train. That distinction, stated in chapter 4, is where Part II begins.

9.7 Chapter summary

The graph network simulator of chapter 8 was trained on `data/v2` across every consist size from 10 to 80, in batches of a single size so that no padding is needed. Trained only to predict one step, it diverged within about 20 steps of a rollout. Injecting noise into its training inputs, with targets recomputed to the true next state, made it stable, and adding three pushforward steps improved it further; the two work together and neither replaces the other.

Over three seeds, the trained model predicts the step 7.5 times better than zero output and stays accurate over 200 steps, 50 s of simulated time, at 0.065 m/s in velocity and 4.7 mm in coupler stretch. Its velocity predictions extrapolate to consists up to 15 times longer than any it was trained on, and to an unseen corridor at full accuracy. Its coupler shape predictions do not extrapolate to long consists over long horizons, a failure traced to the stability of the learned dynamics rather than to its per-step accuracy or to velocity error. On the steepest corridor its errors double. A rollout costs one network evaluation per stored step instead of fifty, which on this hardware and implementation is a 1.2 to 3 times saving in wall clock.

These are the results of the first experiment, and table 9.11 states them as claims. Every one of them is measured against the simulator that produced the data. The next part of the thesis asks how far that simulator, and the routes and data built on it, can be trusted.

Part II

Interrogating the Simulator

Chapter 10

Critique: What the First Experiment Rests On

10.1 A corridor that was not on the track

The first weakness to surface was in the routes rather than the simulator. Once the first results of chapter 9 were in, the grade field of `route_line2` was examined more closely. Its rms grade was 2.46%, against 0.83–1.03% on the other four corridors, and its grade sat at the 4% limit on 22.7% of its length (table 5.1).

A grade field can be wrong in two ways. The elevation can be correct but too noisy for the chosen smoothing window, in which case a wider window fixes it. Or the elevation can be wrong at the source, because the samples do not lie on the track: the mapped centreline has drifted off the real alignment, or the elevation model does not resolve a cutting or embankment and reports the ground beside it. No smoothing window repairs the second case. Smoothing incorrect values only averages them, and widening the window far enough to hide the error also flattens the real terrain on every healthy corridor.

The two cases can be told apart before any smoothing is done, by looking at the raw elevation directly. Table 10.1 gives three measures of raw roughness for the five corridors. The key one is the share of 10 m steps whose elevation changes by more than a metre. A one-metre change over ten metres is a 10% grade, far steeper than any freight main line, so a corridor where this happens on a large share of steps is not following the track.

The expected explanation for the poor grade field on `route_line2` was too little smoothing. The table shows that it was not. The raw elevation is already wrong before smoothing, and a search for a window wide enough to hide it settled on 1200 m, a value that would have erased real terrain on the other four corridors to rescue one broken one.

The corridor had passed unnoticed because the 4% grade limit is well above the grades freight main lines are built to. The limit clipped the broken corridor's grade without drawing attention to it.

This matters for Part I in two ways. Much of the overspeed population of section 7.7 comes from the corridor's false grades: in a pilot build, 39% of its scenarios exceeded the limit by more than 5 m/s, against 2% on `route_line5`. And the `ood_grade` split, selected as the steepest corridor, is built on it. That split therefore tests extrapolation to false grades

Corridor	Median $\lvert\Delta z\rvert$ per step	Steps with $\lvert\Delta z\rvert > 1$ m	Second-difference roughness
`route_line0`	0.077 m	0.7%	0.021 m
`route_line2`	0.234 m	**19.3%**	0.143 m
`route_line3`	0.047 m	1.5%	0.040 m
`route_line4`	0.039 m	1.1%	0.036 m
`route_line5`	0.045 m	1.0%	0.041 m

Table 10.1: Roughness of the raw elevation, before any smoothing, for the five acquired corridors on a 10 m grid. `route_line2` exceeds a one-metre step on nearly a fifth of its length, against 0.7–1.5% for the others. Its bridge and tunnel share is under 1%, so structure correction does not explain the difference.

rather than to real steep terrain, and results on it should be read in that light. The repair, a check on the raw elevation before any smoothing, is described in section 13.1.

Chapter 11

In-Train Forces: Draft Gear, Traction, and the International Benchmark

Planned chapter. The friction draft gear, notch traction and the dynamic brake, and their comparison with the nine simulators of the international longitudinal dynamics benchmark.

Chapter 12

Air Brakes, Resistance, and the v3 Simulator

Planned chapter. The brake-pipe model calibrated to measured data, the remaining physics corrections, the speed planner, and the assembled v3 configuration.

Part III

The Rebuilt System

Chapter 13

Corridors at Scale and the v3 Corpus

13.1 Repairing the grade field

The pipeline now checks the raw elevation first. A corridor on which more than 5% of steps change by more than a metre is rejected, and the smoothing window is chosen only over the corridors that pass. `route_line2` fails the check (section 10.1) and is removed, leaving four corridors. The window is then set to the narrowest value, 800 m, at which the grade limit stops binding on every remaining corridor.

The grade limit was lowered from 4% to 2.5% at the same time, and its role changed. At 2.5%, reaching the limit is itself a sign of an error in the data, and the fraction of a corridor that reaches it is reported. Table 13.1 gives the resulting grade statistics, to be compared with table 5.1.

Corridor	rms grade	95th percentile	Maximum	Share at limit
`route_line0` (49 km)	0.86%	1.45%	2.50%	1.0%
`route_line3` (29 km)	0.72%	1.25%	1.47%	0.0%
`route_line4` (27 km)	0.39%	0.75%	1.23%	0.0%
`route_line5` (22 km)	0.58%	0.87%	1.17%	0.0%

Table 13.1: Grade statistics for the four corridors that pass the raw-elevation check, with an 800 m smoothing window and a 2.5% grade limit.

`route_line0` still reaches the limit on 1.0% of its length, in short runs with a median length of 135 m. A real ruling grade is held for kilometres, so these runs are residual artifacts rather than terrain. They fall within the tolerance, and a filter on the minimum length of a grade excursion would be a more physical fix than further smoothing.

The effect on simulated trains is direct. In scenarios run on the old grade field, the fastest overspeed reached 67.8 m/s, a freight train travelling at about 245 km/h, and these extreme cases came from the false grades of `route_line2`. On the corrected corridors the worst case falls to 25.7 m/s. The share of scenarios exceeding the speed limit by more than 5 m/s does not change: compared on the same four corridors, it is 13.3% before and 13.5% after. That shows it has a separate cause, which lies in how driving commands are generated rather than in the terrain. Chapter 12 addresses it.

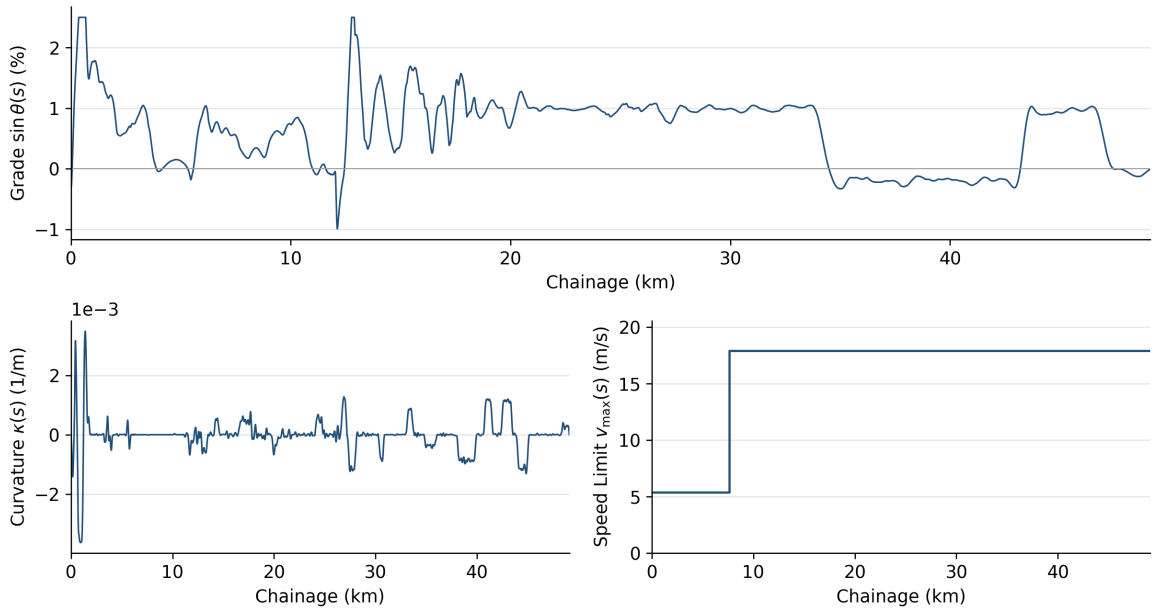


Figure 13.1: The same fields as fig. 5.7, for `route_line0` after the raw-elevation check, with an 800 m smoothing window and a 2.5% grade limit. The urban approach over the first 20 km is still the noisiest stretch, but the grade reaches the 2.5% limit only in two short runs, near the start and near 12.7 km, which are residual artifacts rather than terrain.

13.2 Scaling beyond a handful of corridors

Five corridors are enough to build and test the pipeline, but not enough to support a strong claim about generalization to unseen routes, or to hold out a genuinely steep corridor for evaluation. The next step was therefore a larger, regional corridor set, and the pipeline was extended for it.

13.2.1 The batch crawl

For more than a few routes, a headless batch runner replaces the interactive application. It takes a curated list of seed points, runs the area-first acquisition and bounded outward crawl of section 5.1 from each, and merges the results into one dataset. Lines pulled from overlapping seeds are deduplicated by their shared map identifiers, keeping the longest version of any line pulled more than once, and the run caches each seed so it can resume after an interruption. Several measures keep individual queries to the Overpass service from timing out: service track and industrial, tourism, and military spurs are excluded at query time, the initial query for each seed is split into a grid of small requests, and every request retries with exponential backoff and can rotate across mirror servers.

A regional run of 22 seeds within 900 km of Pueblo, Colorado, chosen to include steep crossings such as Raton Pass and Tennessee Pass, did not complete. It failed on five consecutive seeds with server-busy errors over 4.5 hours. The choice of server had been based on the time to answer a single test query, and a single query does not predict a sustained crawl: a server that answers one request quickly may throttle heavily once hundreds arrive. Retries

and backoff do not help when the limit is the server's tolerance for the workload itself.

13.2.2 The North American Rail Network

The crawl was replaced by a different source of track geometry. The U.S. Department of Transportation's Bureau of Transportation Statistics publishes the North American Rail Network, a national dataset of rail line segments covering the United States, Canada, and Mexico, as a queryable feature service [59]. It removes the two hardest parts of the OSM path.

- **Topology is explicit.** Every segment records the node it starts from and the node it ends at, so the network is already a graph. The OSM path had to reconstruct connectivity from fragment endpoints, which is where its gaps and branch-ordering problems came from.
- **Track is already classified.** Each of the 302,771 segments carries a network class assigned by the data's maintainers. Main-line track accounts for 95,936 segments; the rest are yard tracks, industrial leads, passing sidings, and out-of-service or abandoned line. The mainline filter that the OSM path approximated with tag rules is given directly.

It also carries attributes OSM did not reliably provide, including the owning railroad, the subdivision name, and the number of tracks, so corridors can be identified by their real names rather than by labels such as `route_line2`. No rate limiting was observed. The full main-line network, 95,936 segments and 274,145 km of track, was downloaded in 13.5 minutes.

The network does not include line speeds, so speed limits still come from OSM, queried per corridor rather than crawled. Elevation still comes from 3DEP. With the batched elevation queries of section 5.3, the rest of the pipeline is fast enough for the whole network: resampling, elevation, and profile derivation for all 274,145 km at 10 m spacing, about 27 million points, are projected to take roughly 36 minutes, almost all of it in elevation queries.

13.2.3 From network to corridors

The network is a graph of segments averaging 2.9 km in length, joined at 92,787 nodes, not a set of routes. The simulator needs continuous corridors with a single, increasing chainage, so a traversal stage was added before resampling. Network segments are adapted to the same way objects the OSM path produces, so the corridor walk of section 5.1 applies unchanged. Because connectivity is recorded in the data rather than inferred from endpoints, the gap-bridging and coordinate-fallback heuristics of the OSM path never fire.

Corridors are grown within each railroad subdivision, identified by owner and subdivision name together since names are reused across railroads. Long chains are cut at 60 km and chains shorter than 15 km are dropped. The traversal of the whole continent takes 93 s and needs no network access. It yields 5,621 corridors (4,350 in the United States, 935 in Canada and 336 in Mexico), with a median length of 47.6 km, retaining 248,892 of the 274,145 km of main-line track. Corridors carry real identities, such as `bnsf_raton_p0_56km`

or `rrrr_tennessee_pass_p0_39km`, and corridor supply is no longer the constraint on the size of the evaluation splits.

Chapter 14

The Surrogate, Revised

Planned chapter. What the v3 physics changes for the model: an edge state for the draft gear, new features, the Chapter 8 measurements repeated on v3, and training at scale.

Chapter 15

Results II

Planned chapter. The protocol of chapter 9 on the v3 corpus, all held-out splits, the full baseline set, and the comparison of training on illustrative and sourced physics.

Chapter 16

Fuel-Optimal Control Demonstration

Planned chapter. Model-predictive control through the surrogate's gradients on held-out corridors, with speed-limit and coupler-force constraints.

Chapter 17

Discussion and Conclusion

Planned chapter. What the critique changed, the lessons that transfer, and the limitations that remain.

Bibliography

[1] Qing Wu, Colin Cole, and Maksym Spiryagin. "Parallel Computing Enables Whole-Trip Train Dynamics Optimizations". In: *Journal of Computational and Nonlinear Dynamics* 11.4 (2016), p. 044503. DOI: 10.1115/1.4032075.

[2] Qing Wu and Colin Cole. "Computing Schemes for Longitudinal Train Dynamics: Sequential, Parallel and Hybrid". In: *Journal of Computational and Nonlinear Dynamics* 10.6 (2015), p. 064502. DOI: 10.1115/1.4029716.

[3] Shiyang Zhou et al. "A Machine-Learning-based Surrogate Modeling Methodology for Submodel Integration in the Holistic Railway Digital Twin Platform". In: *Procedia CIRP (33rd CIRP Design Conference)*. Vol. 119. Elsevier, 2023, pp. 345–350. DOI: 10.1016/j.procir.2023.02.141.

[4] Peter W. Battaglia et al. "Relational Inductive Biases, Deep Learning, and Graph Networks". In: *arXiv preprint arXiv:1806.01261* (2018).

[5] Alvaro Sanchez-Gonzalez et al. "Learning to Simulate Complex Physics with Graph Networks". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. Vol. 119. 2020, pp. 8459–8468.

[6] Kimberly Stachenfeld et al. "Learned Coarse Models for Efficient Turbulence Simulation". In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2112.15275. URL: https://arxiv.org/abs/2112.15275.

[7] George James et al. "A Differentiable Simulation Testbed for Longitudinal Train Dynamics: Reference Simulator, Real-Terrain Route Pipeline, and Research Console for a Graph Neural Surrogate". Manuscript submitted to the Transportation Research Board 106th Annual Meeting. 2026.

[8] Colin Cole. "Longitudinal Train Dynamics". In: *Handbook of Railway Vehicle Dynamics*. Ed. by Simon Iwnicki. Boca Raton, FL: CRC Press, 2006. Chap. 9, pp. 239–278. ISBN: 9780849333217.

[9] Qing Wu, Maksym Spiryagin, and Colin Cole. "Longitudinal train dynamics: an overview". In: *Vehicle System Dynamics* 54.12 (2016), pp. 1688–1714. DOI: 10.1080/00423114.2016.1228988.

[10] Ahmed A. Shabana. *Dynamics of Multibody Systems*. 5th. Cambridge, UK: Cambridge University Press, 2020. ISBN: 9781108485647.

[11] Maksym Spiryagin, Qing Wu, and Colin Cole. “International benchmarking of longitudinal train dynamics simulators: benchmarking questions”. In: *Vehicle System Dynamics* 55.4 (2017), pp. 450–463. DOI: 10.1080/00423114.2016.1270457.

[12] L. Cantone et al. “TrainDy, a new UIC simulator for the longitudinal dynamics of trains”. In: *Computers in Railways XI: Computer System Design and Operation in the Railway and Other Transit Systems*. Vol. 103. WIT Transactions on The Built Environment. COMPRAIL 2008, Toledo, Spain, 15–17 September 2008. Southampton, UK: WIT Press, 2008, pp. 735–744. ISBN: 9781845641269. DOI: 10.2495/CR080711.

[13] Ralph Schorr. *Vehicle-Track Modeling and Simulation*. Presentation PC07, Principles Course, Wheel/Rail Interaction Conference (WRI ’18), Rosemont, IL. Accessed 2026-08-18. Amsted Rail, May 2018. URL: https://archive.wheel-rail-seminars.com/archives/2018/pc-papers/presentations/PC07.pdf.

[14] Rachel Anaya and Shad Pate. *Fully Scalable Train Braking Simulation Environment*. Tech. rep. DOT/FRA/ORD-23/19. Available from the FRA eLibrary; accessed 2026-08-18. Washington, DC: U.S. Department of Transportation, Federal Railroad Administration, July 2023.

[15] Federal Railroad Administration. *Train Energy and Dynamics Simulator (TEDS)*. Web page, Research & Development, Rolling Stock, Current Projects. Last updated 22 October 2019; accessed 2026-08-18. Oct. 2019. URL: https://railroads.dot.gov/rolling-stock/current-projects/train-energy-and-dynamics-simulator-teds.

[16] Sharma & Associates, Inc. *Validation of the Train Energy and Dynamics Simulator (TEDS)*. Tech. rep. DOT/FRA/ORD-15/01. Accessed 2026-08-18. Washington, DC: U.S. Department of Transportation, Federal Railroad Administration, Office of Railroad Policy and Development, Jan. 2015. DOI: 10.21949/1503647. URL: https://railroads.dot.gov/sites/fra.dot.gov/files/fra_net/14343/TEDS_Validation_final.pdf.

[17] Universal Mechanism. *UM Train: Module for Train Dynamics Simulation*. Web page, Universal Mechanism module catalogue. Accessed 2026-08-18. 2024. URL: https://www.universalmechanism.com/en/plugins/catalogue/index.php?id=3.

[18] Qing Wu, Shihui Luo, and Colin Cole. “Longitudinal dynamics and energy analysis for heavy haul trains”. In: *Journal of Modern Transportation* 22.3 (2014), pp. 127–136. DOI: 10.1007/s40534-014-0055-x.

[19] Yinyu Nie et al. “A data-driven dynamics simulation framework for railway vehicles”. In: *Vehicle System Dynamics* 56.3 (2018), pp. 406–427. DOI: 10.1080/00423114.2017.1381981.

[20] Sheng Zhang, Pu Huang, and Wenyi Yan. “A data-driven approach for railway in-train forces monitoring”. In: *Advanced Engineering Informatics* 59 (2024), p. 102258. DOI: 10.1016/j.aei.2023.102258.

[21] Sheng Zhang and Wenyi Yan. “In-trainNet: A Two-Step Data-Driven Framework for Enhancing Railway In-Train Forces Monitoring”. In: *Advanced Engineering Informatics* 65 (2025), p. 103352. DOI: 10.1016/j.aei.2025.103352.

[22] Ruiguang Liu et al. "A Data-Driven Surrogate Modeling for Train Rescheduling in High-Speed Railway Networks Under Wind-Caused Speed Restrictions". In: *IEEE Transactions on Automation Science and Engineering* 21.2 (2024), pp. 1107–1121. DOI: 10.1109/TASE.2023.3338695.

[23] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, pp. 5998–6008. arXiv: 1706.03762 [cs.CL].

[24] Haoyi Zhou et al. "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12. 2021, pp. 11106–11115. DOI: 10.1609/aaai.v35i12.17325. arXiv: 2012.07436 [cs.LG].

[25] Haixu Wu et al. "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting". In: *Advances in Neural Information Processing Systems 34 (NeurIPS)*. 2021, pp. 22419–22430. arXiv: 2106.13008 [cs.LG].

[26] Bryan Lim et al. "Temporal Fusion Transformers for Interpretable Multi-Horizon Time Series Forecasting". In: *International Journal of Forecasting* 37.4 (2021), pp. 1748–1764. DOI: 10.1016/j.ijforecast.2021.03.012. arXiv: 1912.09363 [cs.LG].

[27] Zongyi Li et al. "Fourier Neural Operator for Parametric Partial Differential Equations". In: *International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2010.08895 [cs.LG].

[28] Zongyi Li et al. "Physics-Informed Neural Operator for Learning Partial Differential Equations". In: *arXiv preprint arXiv:2111.03794* (2021). arXiv: 2111.03794 [cs.LG].

[29] Shuhao Cao. "Choose a Transformer: Fourier or Galerkin". In: *Advances in Neural Information Processing Systems 34 (NeurIPS)*. 2021. arXiv: 2105.14995 [cs.LG].

[30] Zijie Li, Kazem Meidani, and Amir Barati Farimani. "Transformer for Partial Differential Equations' Operator Learning". In: *Transactions on Machine Learning Research* (2023). arXiv: 2205.13671 [cs.LG].

[31] Peter W. Battaglia et al. "Interaction Networks for Learning about Objects, Relations and Physics". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 29. 2016. arXiv: 1612.00222 [cs.AI]. URL: https://arxiv.org/abs/1612.00222.

[32] Justin Gilmer et al. "Neural Message Passing for Quantum Chemistry". In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. Proceedings of Machine Learning Research. 2017, pp. 1263–1272. URL: https://proceedings.mlr.press/v70/gilmer17a.html.

[33] Tobias Pfaff et al. "Learning Mesh-Based Simulation with Graph Networks". In: *International Conference on Learning Representations (ICLR)*. 2021.

[34] Johannes Brandstetter, Daniel E. Worrall, and Max Welling. "Message Passing Neural PDE Solvers". In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2202.03376. URL: https://arxiv.org/abs/2202.03376.

[35] Kiwon Um et al. “Solver-in-the-Loop: Learning from Differentiable Physics to Interact with Iterative PDE-Solvers”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020. arXiv: 2007.00016. URL: https://proceedings.neurips.cc/paper/2020/hash/43e4e6a6f341e00671e123714de019a8-Abstract.html.

[36] Dmitrii Kochkov et al. “Machine Learning-Accelerated Computational Fluid Dynamics”. In: *Proceedings of the National Academy of Sciences* 118.21 (2021), e2101784118. DOI: 10.1073/pnas.2101784118.

[37] Ricky T. Q. Chen et al. “Neural Ordinary Differential Equations”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018, pp. 6571–6583.

[38] Suyong Kim et al. “Stiff Neural Ordinary Differential Equations”. In: *Chaos: An Interdisciplinary Journal of Nonlinear Science* 31.9 (2021), p. 093122. DOI: 10.1063/5.0060697.

[39] Zhanjun Shao et al. “A New Three-Dimensional Model of Train-Track-Bridge Coupled System Based on Meshless Method and Its Graph Neural Network-Based Surrogate Model”. In: *Computers & Structures* 315 (2025), p. 107786. DOI: 10.1016/j.compstruc.2025.107786.

[40] Zhirong Zhao et al. “Spatio-Temporal Joint Network for Coupler Anomaly Detection Under Complex Working Conditions Utilizing Multi-Source Sensors”. In: *Sensors* 26.9 (2026), p. 2661. DOI: 10.3390/s26092661.

[41] Alessandro Lucchetti et al. *Graph Network-based Structural Simulator: Graph Neural Networks for Structural Dynamics*. 2025. arXiv: 2510.25683 [cs.LG]. URL: https://arxiv.org/abs/2510.25683.

[42] Roger Koenker and Gilbert Bassett. “Regression Quantiles”. In: *Econometrica* 46.1 (1978), pp. 33–50. DOI: 10.2307/1913643.

[43] Charles Blundell et al. “Weight Uncertainty in Neural Networks”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. PMLR, 2015, pp. 1613–1622. arXiv: 1505.05424 [stat.ML].

[44] Yarin Gal and Zoubin Ghahramani. “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning”. In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. PMLR, 2016, pp. 1050–1059. arXiv: 1506.02142 [stat.ML].

[45] Chuan Guo et al. “On Calibration of Modern Neural Networks”. In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1321–1330. arXiv: 1706.04599 [cs.LG].

[46] Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. “Accurate Uncertainties for Deep Learning Using Calibrated Regression”. In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 2796–2804. arXiv: 1807.00263 [cs.LG].

[47] Glenn Shafer and Vladimir Vovk. “A Tutorial on Conformal Prediction”. In: *Journal of Machine Learning Research* 9.12 (2008), pp. 371–421. URL: https://www.jmlr.org/papers/v9/shafer08a.html.

[48] Yaniv Romano, Evan Patterson, and Emmanuel J. Candès. "Conformalized Quantile Regression". In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, pp. 3538–3548. arXiv: 1905.03222 [stat.ME].

[49] Tilmann Gneiting and Adrian E. Raftery. "Strictly Proper Scoring Rules, Prediction, and Estimation". In: *Journal of the American Statistical Association* 102.477 (2007), pp. 359–378. DOI: 10.1198/016214506000001437.

[50] OpenStreetMap contributors. *OpenStreetMap*. https://www.openstreetmap.org. Accessed July 2026. 2026.

[51] U.S. Geological Survey. *3D Elevation Program (3DEP) Standards and Specifications*. Tech. rep. Reston, VA: U.S. Geological Survey, 2022.

[52] Abraham Savitzky and Marcel J. E. Golay. "Smoothing and Differentiation of Data by Simplified Least Squares Procedures". In: *Analytical Chemistry* 36.8 (1964), pp. 1627–1639.

[53] Federal Railroad Administration. *Track Safety Standards, 49 CFR Part 213*. 2023.

[54] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. "Multilayer Feedforward Networks Are Universal Approximators". In: *Neural Networks* 2.5 (1989), pp. 359–366. DOI: 10.1016/0893-6080(89)90020-8.

[55] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[56] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. *Layer Normalization*. 2016. arXiv: 1607.06450 [stat.ML].

[57] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning". In: *Neural Networks* 107 (2018), pp. 3–11. DOI: 10.1016/j.neunet.2017.12.012.

[58] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6980.

[59] Bureau of Transportation Statistics. *North American Rail Network Lines*. National Transportation Atlas Database, U.S. Department of Transportation. https://services.arcgis.com/xOi1kZaI0eWDREZv/arcgis/rest/services/NTAD_North_American_Rail_Network_Lines/FeatureServer/0. Accessed September 2026. 2026.